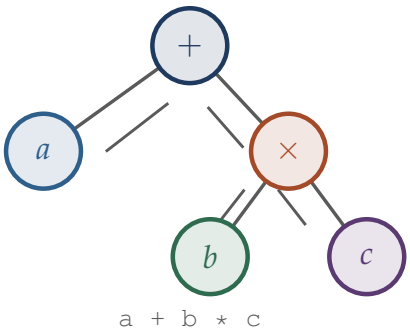
Informatik von Grund auf — Band 6

Die Verwandlung des Textes

Eine vollständige Einführung in Programmiersprachen und Übersetzer



Von der Zeichenkette bis zum Maschinenbefehl

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 6
Die Verwandlung des Textes · Programmiersprachen und Übersetzerbau

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 6: Die Verwandlung des Textes*. Fassung 1.0.1. `https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 188 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Vom Text zur Struktur 9

1 Was ein Übersetzer tut 11

2 Lexikalische Analyse 21

3 Syntaktische Analyse 31

4 Namen und Bedeutung 41

II Die Sprachen selbst 51

5 Typsysteme 53

6 Speicher und Lebensdauer 63

7 Abstraktionsmittel 73

III Vom Baum zum Befehl 83

8 Zwischendarstellungen 85

9 Optimierung 95

10 Codeerzeugung 107

11 Binden und Laden 117

IV Andere Wege 125

12 Interpreter und virtuelle Maschinen 127

13 Übersetzen zur Laufzeit 139

14 Übersetzer in der Praxis 149

15 Der Stand im Jahr 2026 **161**

V Anhang **169**

A Zahlen und Kenngrößen **171**

B Landkarte des Bandes **175**

C Glossar **177**

D Zeittafel **179**

E Kommentierte Quellen **181**

Vorwort: Warum es zunächst niemand glaubte

Ein Übersetzer nimmt einen Text entgegen, den ein Mensch geschrieben hat, und erzeugt daraus Befehle für eine Maschine. *Das klingt heute selbstverständlich. Es war es lange nicht.*

Vier Zeilen und ihr Ergebnis

```
int summe(int *a, int n) {
  int s = 0;
  for (int i = 0; i < n; i++) s +=
a[i];
  return s; }
```

Ein heutiger Übersetzer macht daraus, je nach eingestellter Stufe, sehr verschiedene Dinge.

Stufe	Befehle	Was geschieht
`-O0`	≈ 24	wörtliche Umsetzung
`-O1`	≈ 12	Werte in Registern gehalten
`-O2`	≈ 9	Schleife entrollt
`-O3`	≈ 7	**acht Werte auf einmal addiert**

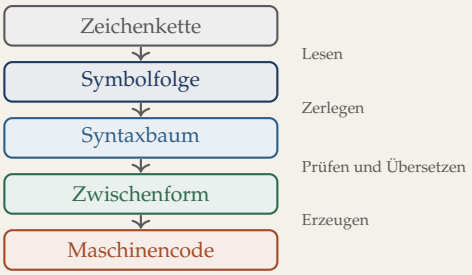
Bei der letzten Zeile hat der Übersetzer erkannt, dass die Additionen unabhängig voneinander sind, und benutzt Befehle, die acht Zahlen gleichzeitig verarbeiten (Band 4, Kapitel 10). Der Programmtext sagt davon kein Wort.

Stufe	Laufzeit	Verhältnis
`-O0`	1,00 ms	1,0
`-O1`	0,31 ms	3,2
`-O2`	0,22 ms	4,5
`-O3`	0,06 ms	**17**

Faktor siebzehn, ohne dass am Programm etwas geändert wurde. Ein erheblicher Teil dieses Bandes handelt davon, wie das zugeht und wo die Grenzen liegen.

Die sechs Verwandlungen

Der Weg vom Text zum Befehl führt über eine Reihe von Zwischenformen, die jede etwas anderes darstellen.



Zeichenkette	Der Text, wie er in der Datei steht
Symbolfolge	Wörter statt Zeichen: Bezeichner, Zahlen, Operatoren
Syntaxbaum	Die Struktur: Was gehört zu was
Zwischenform	Einfache Anweisungen, unabhängig vom Zielrechner
Maschinencode	Befehle für einen bestimmten Prozessor

> *Jeder Schritt wirft Angaben weg, die er nicht mehr braucht, und fügt andere hinzu. Am Ende steht ein Text, der von dem, womit man begonnen hat, nichts mehr erkennen lässt.*

Warum so viele Zwischenschritte?
Man könnte den Text auch unmittelbar in Befehle übersetzen, und die ersten Übersetzer taten genau das. Die Aufteilung hat drei Gründe.

Prüfbarkeit	Jede Stufe hat eine klare Aufgabe und lässt sich für sich testen
Wiederverwendung	m Sprachen und n Rechner brauchen $m + n$ Teile statt $m \cdot n$
Optimierung	In der Zwischenform lässt sich umformen, ohne Rücksicht auf Sprache oder Zielrechner

Die zweite Zeile ist der Grund, warum es Projekte wie LLVM gibt. Wer eine neue Sprache erfindet, schreibt nur den vorderen Teil und erbt damit die Unterstützung für zwanzig Prozessorarchitekturen.

	ohne Zwischenform	**mit Zwischenform**
5 Sprachen, 4 Rechner	20 Übersetzer	**9** Teile
10 und 10	100	**20**

Was dieses Buch will

Erstens: Zeigen, dass ein Übersetzer ein gewöhnliches Programm ist.
Übersetzerbau gilt als schwieriges Gebiet, und das liegt teilweise an der Darstellung. In Wahrheit ist ein Übersetzer eine Kette von Umformungen auf Datenstrukturen, und jede einzelne davon ist überschaubar.

> *Ich gehe deshalb jede Stufe an einem durchgehenden Beispiel durch, von der Zeichenkette bis zum fertigen Befehl, und rechne die Zwischenergebnisse vollständig aus.*

Zweitens: Die Sprachen selbst mitbehandeln.
Ein Übersetzer setzt eine Sprache um, und die Frage, warum eine Sprache so aussieht, wie sie aussieht, ist mindestens so interessant wie die Umsetzung. Typsysteme, Speichermodelle und Abstraktionsmittel bekommen deshalb einen eigenen Teil.

Drittens: Den Selbstbezug ernst nehmen.
Ein Übersetzer für eine Sprache lässt sich in dieser Sprache schreiben. Das klingt nach einem Zirkelschluss und ist keiner, sondern gängige Praxis.

> *Kapitel 1 zeigt, wie man aus diesem scheinbaren Zirkel herauskommt, und Kapitel 14 zeigt, welche unangenehme Möglichkeit sich daraus ergibt. Es ist dieselbe Art von Kreis, die in Band 2 beim Halteproblem auftauchte.*

Drei Warnungen

Erstens: Vieles ist unentscheidbar.
Die Frage, ob eine Optimierung zulässig ist, führt oft auf Probleme, die im Sinne von Band 2 nicht lösbar sind. Übersetzer arbeiten deshalb mit Näherungen, die im Zweifel die vorsichtigere Antwort geben.
Zweitens: Optimierung ist selten das Wichtigste.

> *Ein Übersetzer, der falsch übersetzt, ist unbrauchbar, auch wenn er schnellen Code erzeugt. Und ein Übersetzer mit schlechten Fehlermeldungen kostet mehr Arbeitszeit, als seine Optimierungen je einsparen.*

Drittens: Die Beispiele sind vereinfacht.
Ein vollständiger C-Übersetzer hat einige hunderttausend Zeilen. Ich zeige jeweils den Kern des Verfahrens und sage dazu, was in einer vollständigen Umsetzung noch dazukäme.

Zum Aufbau

- **Teil I** (Kap. 1–4): Vom Text zur geprüften Struktur. Lexikalische und syntaktische Analyse, Namen und Typen.
- **Teil II** (Kap. 5–7): Die Sprachen selbst. Typsysteme, Speicher und Lebensdauer, Abstraktionsmittel.
- **Teil III** (Kap. 8–11): Vom Baum zum Befehl. Zwischenformen, Optimierung, Codeerzeugung, Binden.
- **Teil IV** (Kap. 12–15): Andere Wege. Interpreter, Übersetzung zur Laufzeit, Praxis, und der Stand im Jahr 2026.

Der Übersetzer, den niemand für möglich hielt

Im Jahr 1953 war Programmieren gleichbedeutend mit dem Schreiben von Maschinenbefehlen. Wer eine Formel berechnen wollte, zerlegte sie von Hand in Additionen, Multiplikationen und Sprünge.
JOHN BACKUS arbeitete bei IBM und hielt das für Verschwendung. Er schlug ein System vor, das mathematische Formeln entgegennimmt und den Maschinencode selbst erzeugt.

> *Der Einwand seiner Zeitgenossen war nicht, dass es unmöglich sei. Der Einwand war, dass ein automatisch erzeugtes Programm niemals so gut sein könne wie ein von Hand geschriebenes, und dass Rechenzeit viel zu teuer sei, um sie zu verschwenden.*

Der Einwand war berechtigt. Ein Rechner der IBM 704 kostete etwa eine Million Dollar, und ein Programmierer verdiente einige tausend im Jahr. Es lohnte sich, Menschenzeit gegen Maschinenzeit zu tauschen, und zwar in dieser Richtung.

	1957	**2026**
Rechner	$\approx$ 1 Mio. $	einige hundert $
Programmierer	einige tausend $	sechsstellig
Knapp ist	**Rechenzeit**	**Menschenzeit**

Backus wusste, dass er diesen Einwand entkräften musste, und setzte darauf alles. Das Team von etwa einem Dutzend Leuten brauchte zweieinhalb Jahre, deutlich länger als geplant.

> *Das Ergebnis erschien 1957 als FORTRAN. Der Übersetzer umfasste etwa 25 000 Zeilen Maschinencode und war das mit Abstand größte Programm, das bis dahin geschrieben worden war.*

Er war gut. Der erzeugte Code kam an handgeschriebenen heran, in manchen Fällen war er besser, weil er die Register geduldiger verwaltete als ein müder Mensch. Das war der Punkt, an dem der Widerstand zusammenbrach.

1957	FORTRAN: der Nachweis, dass es geht
1960	ALGOL 60: die saubere Sprachbeschreibung
1960	LISP: Programme als Daten
1972	C: Systemprogrammierung in einer Hochsprache

Backus hat später gesagt, ein erheblicher Teil des Aufwands sei in die Registerzuteilung und in die Behandlung von Feldindizes gegangen, also genau in die Stellen, an denen ein Mensch üblicherweise Fehler macht.

Er erhielt 1977 den Turing-Award. In seiner Dankesrede kritisierte er die Sprachen, die auf FORTRAN gefolgt waren, ausführlich und schlug etwas ganz anderes vor. Das ist eine ungewöhnliche Art, einen Preis entgegenzunehmen.

Siebzig Jahre Übersetzerbau

1951	Hopper	erste Programme, die andere Programme erzeugen
1957	**Backus u. a.**	**FORTRAN**
1958	McCarthy	LISP; Programme als Listen
1960	**ALGOL 60**	**BNF, Blockstruktur, Rekursion**
1964	Kemeny, Kurtz	BASIC
1965	Knuth	LR-Analyse
1968	Dijkstra	„Go To Statement Considered Harmful“
1970	Wirth	Pascal
1971	Wirth	schrittweise Verfeinerung
1972	**Ritchie**	**C**
1973	Hoare	Hinweise zum Sprachentwurf
1975	Steele, Sussman	Scheme
1978	Milner	Typinferenz für ML
1979	`yacc`, `lex`	Werkzeuge statt Handarbeit
1982	Chaitin	Registerzuteilung durch Graphfärbung
1983	Stroustrup	C++
1987	Ferrante u. a.	Programmabhängigkeitsgraph

1991	**Cytron u. a.**	**SSA-Form**
1991	van Rossum	Python
1994	Hölzle, Ungar	Aufrufe zur Laufzeit optimieren
1995	Java	Bytecode und virtuelle Maschine
1999	HotSpot	Übersetzung zur Laufzeit im Massengebrauch
2003	**Lattner**	**LLVM**
2008	V8	JavaScript wird schnell
2010	Rust beginnt	Eigentum im Typsystem
2011	Yang u. a.	Csmith findet Übersetzerfehler
2012	Go 1.0	schnelles Übersetzen als Entwurfsziel
2015	Rust 1.0	
2017	WebAssembly	tragbares Zwischenformat
2020	Sprachserver	Übersetzertechnik im Editor
2026	—	Übersetzer als Prüfwerkzeug

Zwei Zeilen verdienen eine Erläuterung.
1960, ALGOL 60. Der Bericht über diese Sprache war der erste, der die Syntax vollständig und formal beschrieb, in einer Notation, die heute Backus-Naur-Form heißt. Vorher waren Sprachbeschreibungen Prosa mit Beispielen, und Streitfragen wurden durch Nachsehen im Übersetzer geklärt.
1991, SSA-Form. Eine Zwischendarstellung, in der jede Variable genau einmal geschrieben wird. Das klingt nach einer technischen Einzelheit und hat die Optimierung so gründlich verändert, dass heute praktisch jeder ernsthafte Übersetzer damit arbeitet.

Ein Programm ist eine Botschaft an einen Menschen
und nur nebenbei eine Anweisung an eine Maschine.

— sinngemäß ein Gedanke, den DONALD KNUTH
unter dem Namen Literate Programming *ausgearbeitet hat.*
Wer je fremden Code gelesen hat, weiß, warum.

Part I

Vom Text zur Struktur

1 Was ein Übersetzer tut

Vorderteil und Hinterteil

Ein Übersetzer zerfällt in zwei Hälften, die über die Zwischenform miteinander sprechen.

	Vorderteil	Hinterteil
Kennt	die Sprache	den Zielrechner
Kennt nicht	den Zielrechner	die Sprache
Erzeugt	Zwischenform	Maschinencode
Fehler meldet	**es**	fast nie

> *Die letzte Zeile ist wichtig. Alle Meldungen, die ein Programmierer je zu sehen bekommt, entstehen im Vorderteil. Wenn das Hinterteil einen Fehler meldet, ist meist der Übersetzer selbst schuld.*

Dazwischen liegt der Mittelteil, in dem die Optimierungen arbeiten. Er kennt weder die Sprache noch den Rechner und ist deshalb der Teil, der sich am besten wiederverwenden lässt.



Wie viel Arbeit wo steckt.

Teil	Zeilen	Übersetzungszeit
Lexik	≈ 2 %	≈ 5 %
Syntax	≈ 5 %	≈ 10 %
Semantik	≈ 15 %	≈ 15 %
Optimierung	≈ 50 %	≈ 50 %
Erzeugung	≈ 28 %	≈ 20 %

Die vordersten Stufen, denen Lehrbücher traditionell den meisten Raum widmen, machen zusammen weniger als ein Zehntel aus. Das liegt daran, dass sie gelöst sind: Man kann sie aus einer Grammatik erzeugen lassen.

Übersetzen oder ausführen

Es gibt nicht nur einen Weg vom Text zur Ausführung.

Verfahren	Vorgehen	Beispiel
Übersetzen	einmal, vorab	C, Rust, Go
Interpretieren	Anweisung für Anweisung	Shell, frühes BASIC
Bytecode	übersetzen, dann interpretieren	Python, Java
Zur Laufzeit	interpretieren, Heißes übersetzen	JavaScript, Java
Transpilieren	in eine andere Hochsprache	TypeScript

	Startzeit	Dauerlauf	Werkzeuge
Übersetzt	sofort	schnell	Neustart nötig
Interpretiert	sofort	10–100-fach	sehr gut
Bytecode	gut	3–10-fach	gut
Zur Laufzeit	Anlauf nötig	fast wie übersetzt	gut

Die Grenzen verschwimmen. Java übersetzt zu Bytecode und diesen zur Laufzeit weiter. Python hat einen Bytecode-Übersetzer und seit 2023 zusätzlich eine Übersetzung zur Laufzeit. Die Frage „übersetzt oder interpretiert“ ist eine Eigenschaft der Umsetzung, nicht der Sprache.

Ein verbreiteter Irrtum

Falsch	„Python ist eine interpretierte Sprache“
Richtig	„Die verbreitete Umsetzung von Python arbeitet mit Bytecode und einem Interpreter“

Es gibt Übersetzer für Python, die Maschinencode erzeugen, und es gibt Interpreter für C. Beides ist möglich, weil eine Sprache nur festlegt, was *ein Programm bedeutet, nicht* wie *es ausgeführt wird.*

Was allerdings stimmt: Manche Spracheigenschaften machen eine schnelle Übersetzung schwer. Wenn ein Programm zur Laufzeit neue Namen einführen und Typen ändern kann, muss der erzeugte Code darauf gefasst sein, und das kostet.

Der Übersetzer, der sich selbst übersetzt

Ein Übersetzer für eine Sprache lässt sich in eben dieser Sprache schreiben. Das wirft eine Frage auf: Womit wird der erste übersetzt?

1. Man schreibt einen kleinen Übersetzer für einen **Teil** der Sprache, in einer anderen Sprache.
2. Mit ihm übersetzt man einen größeren Übersetzer, geschrieben in diesem Teil.
3. Mit dem größeren übersetzt man einen noch größeren.
4. Am Ende übersetzt der Übersetzer sich selbst.

Der Kreis ist damit keiner. Er ist eine Treppe, deren unterste Stufe außerhalb liegt. Danach kann man die Hilfssprache wegwerfen, weil sie nicht mehr gebraucht wird.

Sprache	Übersetzer in	seit
C	C	1973
Go	C, dann Go	2015
Rust	OCaml, dann Rust	2011
TypeScript	TypeScript	

Warum man das tut.

Prüfung	Der Übersetzer ist das erste ernsthafte Programm in der neuen Sprache; Schwächen fallen sofort auf
Glaubwürdigkeit	Wer seine Sprache nicht selbst benutzt, hat ein Argumentationsproblem
Wartbarkeit	Wer die Sprache beherrscht, kann am Übersetzer arbeiten

Es gibt auch einen Einwand. Ein selbsttragender Übersetzer lässt sich nur mit einer älteren Fassung seiner selbst bauen, und diese Kette reicht bis zum Anfang zurück. Wer heute Rust von Grund auf bauen will, braucht eine lange Reihe von Zwischenfassungen.

Und es gibt eine unangenehmere Folgerung, auf die Ken Thompson 1984 aufmerksam gemacht hat. Kapitel 14 kommt darauf zurück.

Aufgabe 1.1.

(a) Verfolgen Sie einen Ausdruck durch alle Stufen des Übersetzers.
(b) Rechnen Sie aus, wann sich eine Zwischenform lohnt.
(c) Führen Sie den Selbstaufbau eines Übersetzers durch.

Lösung Schritt für Schritt.

(a) Ein Ausdruck durch alle Stufen.
Wir verfolgen die Zeile

```
x = a + b * 2;
```

Stufe 1 – Die Zeichenkette.

```
'x',' ','=',' ','a',' ','+',' ','b','
','*',' ','2',';'
```

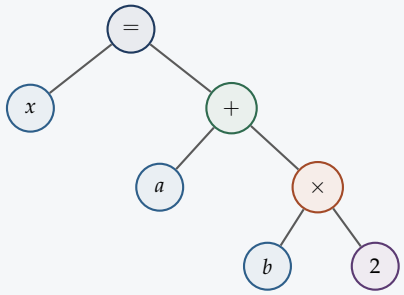
Vierzehn Zeichen ohne jede Struktur.

Stufe 2 – Die Symbolfolge.

Text	Art	Wert
`x`	Bezeichner	`x`
`=`	Zuweisung	—
`a`	Bezeichner	`a`
`+`	Operator	—
`b`	Bezeichner	`b`
`*`	Operator	—
`2`	Zahl	2
`;`	Semikolon	—

Die Leerzeichen sind verschwunden. Aus vierzehn Zeichen sind acht Symbole geworden.

Stufe 3 – Der Syntaxbaum.



Jetzt steht die Struktur da. Dass `b * 2` zusammengehört und nicht `a + b`, ergibt sich aus dem Vorrang der Operatoren und ist im Baum festgehalten. Klammern werden nicht mehr gebraucht.

Stufe 4 – Die Zwischenform.

Jeder Knoten wird zu einer einfachen Anweisung mit höchstens einem Operator.

```
t1 = b * 2
t2 = a + t1
x = t2
```

Stufe 5 – Nach der Optimierung.
Der Übersetzer erkennt, dass `t2` nur einmal benutzt wird, und spart die Zuweisung.

```
t1 = b * 2      wird zu      t1 = b + b   oder
t1 = b « 1

x = a + t1
```

Die Multiplikation mit zwei wird durch eine Verschiebung ersetzt. Das ist die einfachste aller Optimierungen und heißt Stärkeminderung: Ein teurer Operator wird durch einen billigeren ersetzt, der dasselbe leistet.

Stufe 6 – Der Maschinencode.

```
mov eax, [b]
add eax, eax      schneller als shl eax, 1 auf manchen Prozessoren
add eax, [a]
mov [x], eax
```

Stufe	Umfang	enthält
Zeichen	14	alles, auch Formatierung
Symbole	8	keine Leerzeichen mehr
Baumknoten	7	Struktur, keine Klammern
Zwischen-anweisungen	3	keine Verschachtelung
Maschinenbefehle	4	Register statt Namen

(b) Wann sich eine Zwischenform lohnt.
Der Ansatz. Sei m die Zahl der Sprachen, n die der Zielrechner, V der Aufwand für einen Vorderteil, H für einen Hinterteil.

Ohne Zwischenform	$m \cdot n$ vollständige Übersetzer
Mit Zwischenform	m Vorderteile $+$ n Hinterteile

Mit Zahlen. Nehmen wir $V = 30$ Personenmonate, $H = 50$, ein vollständiger Übersetzer 80.

m	n	**ohne**	**mit**
1	1	80	80
2	2	320	160
5	4	1600	**350**
10	10	8000	**800**

Schon bei zwei Sprachen und zwei Rechnern halbiert sich der Aufwand. Bei zehn und zehn ist es Faktor zehn. Deshalb baut heute kaum jemand einen Übersetzer ohne Zwischenform.

Was die Rechnung verschweigt.

Die Zwischenform muss passen	Sie muss alles ausdrücken können, was die Sprachen brauchen, ohne für den Rechner zu speziell zu sein
Verluste an der Grenze	Was der Vorderteil weiß und nicht weitergibt, ist verloren
Ein gemeinsamer Nenner	Eine Zwischenform für C und für Haskell passt zu keiner von beiden gut

Deshalb haben viele Übersetzer heute mehrere *Zwischenformen hintereinander: eine hohe, die noch nah an der Sprache ist, und eine niedrige, die nah am Rechner ist. Rust hat drei, Swift ebenfalls.*

(c) Der Selbstaufbau.
Die Ausgangslage. Wir wollen einen Übersetzer für eine Sprache L haben, geschrieben in L, und besitzen zunächst nur einen C-Übersetzer.
Die Bezeichnung. Wir schreiben $\ddot{U}_L^S$ für einen Übersetzer, der L übersetzt und in S geschrieben ist.

1. Wir wählen $L_0 \subset L$, eine kleine Teilsprache, und schreiben $\ddot{U}^{C}_{L_0}$ von Hand.
2. Mit dem C-Übersetzer entsteht daraus ein laufendes Programm.
3. Wir schreiben $\ddot{U}^{L_0}_{L_1}$: einen Übersetzer für mehr, geschrieben in wenig.
4. Schritt 2 mit dem Ergebnis aus Schritt 2 statt des C-Übersetzers.
5. Wiederholen, bis $L_k = L$.

Ein Zahlenbeispiel.

Stufe	**übersetzt**	**geschrieben in**	**Zeilen**
0	Zuweisungen, `if`	C	1500
1	+ Schleifen, Felder	L_0	4000
2	+ Prozeduren, Typen	L_1	12 000
3	vollständig	L_2	40 000

Die entscheidende Probe.
Man übersetzt den fertigen Übersetzer zweimal mit sich selbst.

Durchgang	**übersetzt mit**	**Ergebnis**
1	Stufe 2	Übersetzer A
2	A	Übersetzer B
3	B	Übersetzer C

B und C müssen **Bit für Bit** gleich sein.

Die Begründung: B und C entstanden aus demselben Quelltext, übersetzt von zwei Programmen (A und B), die ihrerseits denselben Quelltext haben. Ein Unterschied wäre ein Hinweis auf einen Fehler.

Warum diese Probe oft fehlschlägt

Sie verlangt, dass die Übersetzung reproduzierbar ist, und das ist sie erstaunlich selten.

Zeitstempel	Manche Übersetzer betten Datum und Uhrzeit ein
Pfade	Absolute Verzeichnisnamen landen in Fehlermeldungen
Reihenfolge	Bei paralleler Übersetzung entscheidet der Zufall über die Anordnung
Hashwerte	Streuwerte über Zeigeradressen, die sich bei jedem Lauf ändern

Die letzte Zeile ist besonders tückisch. Wenn eine Symboltabelle nach Speicheradressen sortiert wird und der Speicher zufällig verteilt ist (Band 5, Kapitel 5), ändert sich die Reihenfolge bei jedem Lauf, und damit möglicherweise der erzeugte Code.

Seit etwa 2015 gibt es eine breitere Bewegung, Übersetzungen reproduzierbar zu machen. Der Gewinn geht über die Selbstaufbau-Probe hinaus: Wenn zwei unabhängige Personen aus demselben Quelltext dasselbe Programm erzeugen, kann man prüfen, ob ein ausgeliefertes Programm wirklich aus dem veröffentlichten Quelltext stammt.

Aho und Mitautoren[1] sind die klassische Gesamtdarstellung; Cooper und Torczon[6] sind moderner und zugänglicher; Nystrom[23] baut zwei vollständige Übersetzer vor den Augen des Lesers.

2 Lexikalische Analyse

Von Zeichen zu Wörtern

Die erste Stufe fasst Zeichen zu Wörtern zusammen und wirft weg, was für die Bedeutung ohne Belang ist.

Erkennen	Bezeichner, Schlüsselwörter, Zahlen, Zeichenketten, Operatoren
Verwerfen	Leerzeichen, Zeilenumbrüche, Kommentare
Merken	Zeile und Spalte, für die Fehlermeldungen

> *Die dritte Zeile wird gern vergessen und ist die wichtigste für den Benutzer. Ein Übersetzer, der nur „Syntaxfehler“ sagt, ist erheblich weniger wert als einer, der Zeile und Spalte nennt und die Stelle unterstreicht.*

Warum das eine eigene Stufe ist.
Man könnte die Zerlegung in Wörter auch der Syntaxanalyse überlassen. Die Trennung hat praktische Gründe.

Einfachheit	Wörter lassen sich mit regulären Ausdrücken beschreiben, Sätze nicht (Band 2)
Geschwindigkeit	Der Scanner liest jedes Zeichen genau einmal und ist der einzige Teil, der das tut
Sonderregeln	Leerzeichen und Kommentare würden jede Grammatikregel verunstalten

Die erste Zeile verweist auf Band 2. Die Menge der gültigen Bezeichner ist eine reguläre Sprache und lässt sich von einem endlichen Automaten erkennen. Die Menge der gültigen Programme ist es nicht.

Reguläre Ausdrücke und Automaten

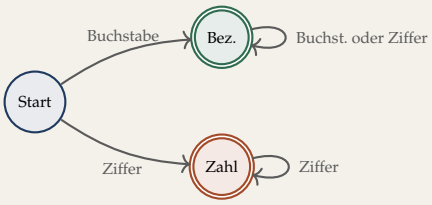
Jede Symbolart wird durch einen regulären Ausdruck beschrieben.

Art	Muster	Beispiel
Bezeichner	`[A-Za-z_][A-Za-z0-9_]*`	`summe1`
Ganzzahl	`[0-9]+`	`4711`
Gleitkomma	`[0-9]+\.[0-9]*`	`3.14`
Zeichenkette	`"[^"]*"`	`"abc"`
Kommentar	`//[^\n]*`	

Aus diesen Mustern lässt sich mechanisch ein endlicher Automat erzeugen. Das ist der Grund, warum niemand einen Scanner mehr von Hand schreibt, außer aus Geschwindigkeitsgründen.

Der Weg vom Muster zum Automaten.

1. Aus jedem regulären Ausdruck einen nichtdeterministischen Automaten bauen (Thompson-Konstruktion).
2. Alle zu einem einzigen zusammenfassen, mit einem gemeinsamen Startzustand.
3. Den nichtdeterministischen in einen deterministischen umwandeln (Teilmengenkonstruktion).
4. Den deterministischen minimieren.



Zwei Regeln bei Mehrdeutigkeit.

Längster Treffer	Bei `index` wird nicht `in` erkannt, sondern das ganze Wort
Erste Regel gewinnt	Passen zwei Muster gleich lang, so entscheidet die Reihenfolge in der Beschreibung

Die zweite Regel ist der Grund, warum Schlüsselwörter in der Beschreibung vor den Bezeichnern stehen müssen. Sonst wäre `while` ein gewöhnlicher Name, denn es passt auf beide Muster.

Wo die längste Übereinstimmung schadet

In älteren C-Fassungen wurde `a--b` als `a - - b` zerlegt, weil der Scanner gierig ist. Gemeint war vielleicht `a - - b`.

> *Ein bekannteres Beispiel steckt in C++ vor 2011: Bei `vector<vector<int>>` las der Scanner » als Schiebeoperator, und der Programmierer musste ein Leerzeichen einfügen. Der Standard von 2011 hat das ausdrücklich repariert.*

Das ist bemerkenswert, weil die Reparatur eine Ausnahme von einer sonst durchgängigen Regel ist. Man hat die Sprachbeschreibung verkompliziert, um den Benutzern ein Ärgernis zu ersparen.

Wo Sprachen es dem Scanner schwer machen

Fortran	Leerzeichen sind bedeutungslos; `DO 10 I = 1,5` ist eine Schleife, `DO 10 I = 1.5` eine Zuweisung an `DO10I`
Python	Einrückung erzeugt Symbole; der Scanner muss einen Stapel führen
C	Ob `a * b` eine Multiplikation oder eine Zeigerdeklaration ist, hängt davon ab, ob `a` ein Typname ist
Perl	Ob `/` eine Division oder der Beginn eines Musters ist, hängt vom Zusammenhang ab

Die dritte und vierte Zeile durchbrechen die saubere Trennung: Der Scanner müsste wissen, was die Syntaxanalyse bisher herausgefunden hat. Man nennt das den Rückkanal, und er gilt als hässlich und ist gelegentlich unvermeidlich.

Die zweite Zeile ist eleganter gelöst. Der Scanner merkt sich die Einrückungstiefen auf einem Stapel und erzeugt beim Einrücken ein Symbol „Block auf“, beim Ausrücken so viele „Block zu“, wie nötig sind.

Zeile	**Tiefe**	**erzeugt**
`if x:`	0	—
`  y = 1`	4	**Block auf**
`    z = 2`	8	**Block auf**
`w = 3`	0	**Block zu, Block zu**

Damit sieht die Syntaxanalyse dieselbe Struktur wie bei geschweiften Klammern, und die Grammatik bleibt einfach. Der ganze Sonderfall ist im Scanner eingesperrt.

Aufgabe 2.1.

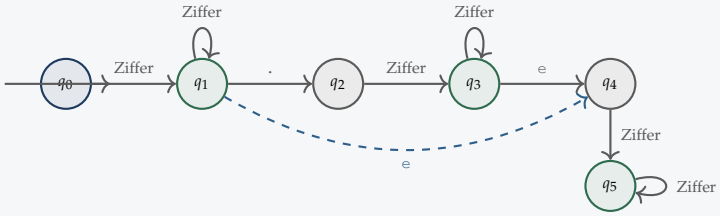
(a) Bauen Sie einen Automaten für Zahlen und führen Sie ihn aus.
(b) Zerlegen Sie ein Programmstück vollständig in Symbole.
(c) Vergleichen Sie erzeugte und handgeschriebene Scanner.

Lösung Schritt für Schritt.

(a) Ein Automat für Zahlen.
Die Aufgabe. Erkannt werden sollen Ganzzahlen, Dezimalzahlen und Zahlen mit Exponent: `42`, `3.14`, `6.02e23`, `1e-9`.
Der reguläre Ausdruck.

```
[0-9]+ ( \.[0-9]+ )?  ( [eE] [+-]?  [0-9]+ )?
```

Der Automat.



Doppelt umrandet gedacht sind die annehmenden Zustände q_1, q_3, q_5.
Der Lauf auf `6.02e23`.

Zeichen	Zustand	annehmend?	letzter Treffer
`6`	q_1	ja	`6`
`.`	q_2	nein	`6`
`0`	q_3	ja	`6.0`
`2`	q_3	ja	`6.02`
`e`	q_4	nein	`6.02`
`2`	q_5	ja	`6.02e2`
`3`	q_5	ja	**`6.02e23`**

Der Scanner merkt sich stets die Stelle des letzten annehmenden Zustands. Läuft er in eine Sackgasse, kehrt er dorthin zurück. Genau das bedeutet „längster Treffer“ in der Umsetzung.

Warum das Rücksetzen nötig ist.
Bei der Eingabe `6.02ex` läuft der Automat bis q_4 und steckt dann fest.

Ohne Rücksetzen	Fehler	falsch
Mit Rücksetzen	`6.02` als Zahl, dann `ex` als Bezeichner	richtig

(b) Ein Programmstück zerlegen.
Die Eingabe.

```
while (i <= n) { // Schleife
  s += a[i++];
}
```

Das Ergebnis.

Text	Art	Zeile:Sp.	Bemerkung
`while`	Schlüsselwort	1:1	vor Bezeichner geprüft
`(`	Klammer auf	1:7	
`i`	Bezeichner	1:8	
`<=`	Operator	1:10	**zwei Zeichen**
`n`	Bezeichner	1:13	
`)`	Klammer zu	1:14	
`{`	Block auf	1:16	
—	—	1:18	Kommentar verworfen
`s`	Bezeichner	2:3	
`+=`	Operator	2:5	
`a`	Bezeichner	2:8	
`[`	Index auf	2:9	
`i`	Bezeichner	2:10	
`++`	Operator	2:11	
`]`	Index zu	2:13	
`;`	Semikolon	2:14	
`}`	Block zu	3:1	

Größe	Wert	
Zeichen der Eingabe	48	
davon verworfen	18	Leerraum, Kommentar
Erzeugte Symbole	16	
Verhältnis	3,0	Zeichen je Symbol

Etwas mehr als ein Drittel der Zeichen sind Leerraum und Kommentar. Bei gut kommentiertem Code liegt der Anteil höher, und das ist kein Grund zur Beunruhigung, sondern ein Zeichen für lesbaren Text.

(c) Erzeugt oder von Hand geschrieben.

	erzeugt (`lex`)	**von Hand**
Aufwand	≈ 100 Zeilen Muster	≈ 1500 Zeilen
Geschwindigkeit	gut	2–5-mal schneller
Fehlermeldungen	unbeholfen	beliebig gut
Sonderfälle	schwierig	einfach
Wartung	Muster ändern	Code ändern

Warum ernsthafte Übersetzer von Hand schreiben.
Die Tabelle sieht nach einem knappen Ergebnis aus. In der Praxis haben fast alle großen Übersetzer einen handgeschriebenen Scanner.

Übersetzer	Scanner
GCC	von Hand
Clang	von Hand
Rust	von Hand
Go	von Hand

Der Grund ist die dritte Zeile der Vergleichstabelle. Ein erzeugter Scanner meldet „unerwartetes Zeichen“, und mehr ist ihm schwer zu entlocken. Ein handgeschriebener kann sagen, dass eine Zeichenkette nicht geschlossen wurde und wo sie begonnen hat.

Was ein guter Scanner an Fehlern erkennt.

Offene Zeichenkette	„Anführungszeichen in Zeile 12 nie geschlossen“
Offener Kommentar	„`/*` in Zeile 40 ohne `*/`“; sonst verschwindet der halbe Rest der Datei
Unsichtbare Zeichen	Geschütztes Leerzeichen sieht aus wie ein Leerzeichen und ist keines
Zahlenüberlauf	`99999999999999999999` passt in keinen Ganzzahltyp

Das Zeichen, das aussieht wie ein anderes

Unicode enthält viele Zeichen, die einander ähneln. Ein kyrillisches Kleinbuchstaben-A sieht in fast jeder Schrift genau aus wie ein lateinisches „a“.

Damit lassen sich zwei verschiedene Bezeichner erzeugen, die auf dem Bildschirm gleich aussehen. In einem Programm mit zwei solchen Variablen sucht man den Fehler lange.

Noch unangenehmer sind Zeichen, die die Anzeigerichtung umkehren. Damit lässt sich Quelltext so gestalten, dass er für einen Menschen anders aussieht, als der Übersetzer ihn liest.

Mensch sieht	einen Kommentar
Übersetzer liest	eine Anweisung

Diese Angriffsart wurde 2021 unter dem Namen „Trojan Source“ beschrieben. Die meisten Übersetzer warnen seither vor solchen Zeichen, und viele Editoren zeigen sie sichtbar an. Es ist ein Problem der lexikalischen Stufe, das dreißig Jahre lang niemand ernst genommen hat.

Aho und Mitautoren[1] behandeln die Konstruktion der Automaten vollständig; Cooper und Torczon[6] sind bei der praktischen Umsetzung ausführlicher; Nystrom[22] schreibt einen Scanner Zeile für Zeile.

3 Syntaktische Analyse

Grammatiken

Eine kontextfreie Grammatik beschreibt, welche Symbolfolgen gültig sind (Band 2, Kapitel 5). Für Ausdrücke sieht sie so aus:

Ausdruck	$\rightarrow$	*Ausdruck* + *Term* $\mid$ *Term*
Term	$\rightarrow$	*Term* $\star$ *Faktor* $\mid$ *Faktor*
Faktor	$\rightarrow$	(*Ausdruck*) $\mid$ *Zahl* $\mid$ *Name*

Der Vorrang der Operatoren steckt in der Schichtung. Weil ein Term nur aus Faktoren und Multiplikationen bestehen kann, bindet die Multiplikation stärker. Das ist keine Zusatzregel, sondern folgt aus dem Aufbau.

Zwei Zerlegungswege.

	Von oben	**Von unten**
Beginnt bei	Startsymbol	Eingabesymbolen
Baut	abwärts	aufwärts
Verfahren	LL, rekursiver Abstieg	LR, LALR
Kann	weniger Grammatiken	mehr Grammatiken
Verständlichkeit	sehr gut	Tabellen
Fehlermeldungen	gut	schwieriger

Die Theorie bevorzugt LR, weil es mehr Grammatiken bewältigt. Die Praxis bevorzugt seit etwa 2005 wieder den rekursiven Abstieg, aus demselben Grund wie beim Leser: Fehlermeldungen und Sonderfälle.

Rekursiver Abstieg

Für jedes Nichtterminal schreibt man eine Funktion. Die Grammatik wird damit unmittelbar zu Code.

```
Knoten* ausdruck() {
  Knoten* links = term();
  while (schau() == PLUS) {
    nimm(PLUS);
    links = neu(PLUS, links, term());
}
  return links; }
```

```
Knoten* term() {
  Knoten* links = faktor();
  while (schau() == STERN) {
    nimm(STERN);
    links = neu(STERN, links,
faktor()); }
  return links; }
```

Bemerkenswert ist, wie wenig das ist. Zwölf Zeilen, und der Vorrang der Operatoren stimmt bereits, weil `ausdruck` den `term` aufruft und nicht umgekehrt.

Warum die Schleife und nicht Rekursion.
Die Grammatik oben ist linksrekursiv: *Ausdruck* → *Ausdruck*+*Term*. Würde man das wörtlich übersetzen, riefe `ausdruck` als Erstes sich selbst auf und käme nie zurück.

Regel	wörtlich	als Schleife
$A \to A\,\alpha \mid \beta$	Endlosrekursion	erst β, dann α wiederholt

Die Umformung ist mechanisch und erhält die Linksassoziativität: `a - b - c` wird als `(a - b) - c` gelesen, wie es sein soll. Bei einer rechtsrekursiven Grammatik käme `a - (b - c)` heraus, und das wäre falsch.

Vorrang ohne Grammatikschichten

Bei fünfzehn Vorrangstufen wie in C bräuchte man fünfzehn Funktionen, die fast gleich aussehen. Das Verfahren von VAUGHAN PRATT kommt mit einer aus.

```
Knoten* ausdruck(int minVorrang) {
  Knoten* links = einfach();
  while (istOperator(schau()) &&
      vorrang(schau()) >= minVorrang) {
    Symbol op = nimm();
    int naechst = vorrang(op) +
          (linksbindend(op) ?  1 :  0);
    Knoten* rechts =
ausdruck(naechst);
    links = neu(op, links, rechts); }
  return links; }
```

Operator	Vorrang	Bindung
`=`	1	rechts
`||`	2	links
`==`	4	links
`+ -`	6	links
`* /`	7	links
`**`	8	**rechts**

Der Kunstgriff steckt in der Zeile mit `+ (linksbindend ?  1 : 0)`. Bei einem linksbindenden Operator wird die Schranke um eins erhöht, sodass ein gleichstarker Operator rechts nicht mehr aufgenommen wird und links gruppiert wird. Bei einem rechtsbindenden bleibt sie gleich.

Ein neuer Operator kostet damit eine Zeile in einer Tabelle statt einer neuen Grammatikschicht. Deshalb benutzen fast alle handgeschriebenen Zerteiler heute dieses Verfahren.

Der ALGOL-Bericht von 1960

Vor 1960 wurden Programmiersprachen in Prosa beschrieben, mit Beispielen und guten Absichten. Wenn zwei Übersetzer ein Programm verschieden auffassten, gab es keine Instanz, die entscheiden konnte, wer recht hatte.
Eine internationale Gruppe von dreizehn Personen traf sich 1959 und 1960, um eine gemeinsame Sprache zu entwerfen. JOHN BACKUS schlug eine Notation vor, mit der sich die Syntax vollständig aufschreiben ließ; PETER NAUR, der Herausgeber des Berichts, vereinfachte sie und wandte sie durchgehend an.

> *Das Ergebnis war der „Report on the Algorithmic Language ALGOL 60". Er umfasste etwa vierzig Seiten und beschrieb die Syntax so genau, dass sich jede Streitfrage durch Nachlesen klären ließ.*

BNF	Eine Notation, in der sich Syntax vollständig aufschreiben lässt
Blockstruktur	Namen gelten in dem Block, in dem sie vereinbart wurden
Rekursion	Prozeduren dürfen sich selbst aufrufen
Verbundanweisung	Mehrere Anweisungen zählen als eine

Die letzten drei Punkte gehören heute zur Grundausstattung jeder Sprache. Damals waren sie neu, und die Rekursion war umstritten, weil sie einen Stapel voraussetzt und damit Laufzeit kostet.

> TONY HOARE *hat berichtet, die Rekursion sei fast beiläufig in den Bericht geraten, weil niemand ausdrücklich widersprach. FORTRAN erlaubte sie erst 1990.*

ALGOL 60 selbst wurde außerhalb Europas wenig benutzt. Seine Wirkung lief anders: Es wurde die Sprache, in der man Algorithmen *aufschrieb*. Wer in den 1960er und 1970er Jahren einen Algorithmus veröffentlichte, tat es in ALGOL, auch wenn er ihn in FORTRAN umsetzte.

Nachkommen	Pascal, C, Modula, Ada, und über C fast alles Weitere
Geblieben ist	Die Vorstellung, dass eine Sprache eine formale Beschreibung hat

Hoare hat über ALGOL 60 gesagt, es sei eine erhebliche Verbesserung gegenüber seinen Vorgängern und gegenüber fast allen seinen Nachfolgern. Das Urteil ist pointiert und nicht ganz unfair.

Aufgabe 3.1.

(a) Zerlegen Sie einen Ausdruck vollständig von Hand.
(b) Untersuchen Sie Mehrdeutigkeiten und ihre Auflösung.
(c) Vergleichen Sie LL und LR an einem Beispiel.

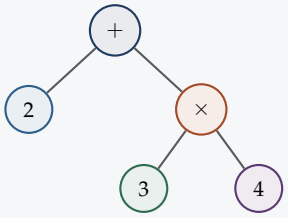
Lösung Schritt für Schritt.

(a) Eine vollständige Zerlegung.
Die Eingabe. `2 + 3 * 4`
Der Ablauf beim rekursiven Abstieg.

Tiefe	Funktion	sieht	tut
1	`ausdruck`	`2`	ruft `term`
2	`term`	`2`	ruft `faktor`
3	`faktor`	`2`	nimmt, liefert Blatt 2
2	`term`	`+`	**kein** `*`, gibt 2 zurück
1	`ausdruck`	`+`	nimmt `+`, ruft `term`
2	`term`	`3`	ruft `faktor`
3	`faktor`	`3`	liefert Blatt 3
2	`term`	`*`	**nimmt** `*`, ruft `faktor`
3	`faktor`	`4`	liefert Blatt 4
2	`term`	Ende	liefert 3×4
1	`ausdruck`	Ende	liefert $2 + (3 \times 4)$

Der entscheidende Schritt steht in Zeile 4 und Zeile 8. Beim ersten Mal sieht `term` *ein* `+` *und gibt auf, weil Additionen nicht seine Sache sind. Beim zweiten Mal sieht es ein* `*` *und greift zu. Daraus ergibt sich der Vorrang.*

Der entstandene Baum.



Die Probe durch Auswerten.

Baum	Wert	richtig?
$2 + (3 \times 4)$	14	**ja**
$(2 + 3) \times 4$	20	nein

(b) Mehrdeutigkeiten.
Der Fall. Eine Grammatik ohne Schichtung:

$$A \to A + A \mid A * A \mid \mathit{Zahl}$$

Für `2 + 3 * 4` gibt es damit zwei Ableitungen, und die Grammatik sagt nicht, welche gilt.

Lesart	Wert	
$2 + (3 \times 4)$	14	üblich
$(2 + 3) \times 4$	20	auch ableitbar

Zwei Wege, das zu beheben.

Grammatik ändern	Schichten einführen, wie zu Beginn des Kapitels
Regeln daneben-schreiben	Grammatik mehrdeutig lassen und Vorrang gesondert angeben

Der zweite Weg ist bei Werkzeugen wie `yacc` üblich. Man schreibt `%left '+'` und `%left '*'` in aufsteigender Reihenfolge, und der Erzeuger löst die Konflikte danach auf. Die Grammatik bleibt kurz und lesbar.

Der berühmteste Fall: das hängende `else`.

```
if (a) if (b) x(); else y();
```

Lesart 1	`else` gehört zum inneren `if`
Lesart 2	`else` gehört zum äußeren `if`

Alle verbreiteten Sprachen wählen Lesart 1: Ein `else` gehört zum nächstgelegenen offenen `if`. Die Regel ist willkürlich, aber einheitlich, und deshalb funktioniert sie.

Wie man das Problem ganz vermeidet.

Pflichtklammern	Go und Rust verlangen geschweifte Klammern; die Mehrdeutigkeit kann nicht entstehen
Abschlusswort	Ada und Ruby schließen mit `end if` beziehungsweise `end`
Einrückung	Python entscheidet über die Spaltenzahl

Das ist ein wiederkehrendes Muster im Sprachentwurf: Ein Problem, das man mit einer Sonderregel lösen kann, verschwindet oft ganz, wenn man die Syntax leicht ändert.

(c) LL und LR im Vergleich.
Was die Namen bedeuten.

	erstes L	**zweiter Buchstabe**
LL	Eingabe von links	**Linksableitung**
LR	Eingabe von links	**Rechtsableitung, rückwärts**

Der praktische Unterschied.
Ein LL-Zerteiler muss sich nach dem ersten Symbol entscheiden, welche Regel er anwendet. Ein LR-Zerteiler sammelt Symbole und entscheidet erst, wenn er die ganze rechte Seite gesehen hat.

Grammatik	$A \to \mathtt{x}\,B \mid \mathtt{x}\,C$	
LL(1)	scheitert	sieht `x`, weiß nicht welche Regel
LR(1)	gelingt	entscheidet später

Für LL muss man die Grammatik umformen, hier durch Ausklammern des gemeinsamen `x`. Das ist mechanisch möglich und macht die Grammatik weniger lesbar.

Die Mächtigkeit.

Klasse	Umfang	Werkzeug
LL(1)	am kleinsten	ANTLR (mit Erweiterungen)
LALR(1)	mittel	`yacc`, `bison`
LR(1)	groß	selten, große Tabellen
GLR	**alle kontextfreien**	mehrdeutige Ergebnisse

Warum die Praxis trotzdem LL bevorzugt

Seit etwa 2005 sind fast alle neuen Übersetzer zum rekursiven Abstieg zurückgekehrt, obwohl er die schwächere Methode ist.

Fehlermeldungen	Bei einer Funktion `parseIfAnweisung` weiß man, was fehlt: „erwartet `)` nach der Bedingung“
Wiederaufsetzen	Nach einem Fehler bis zum nächsten Semikolon lesen und weitermachen
Fehlersuche	Ein Zerteiler aus Funktionen lässt sich im Fehlersuchprogramm durchgehen; eine Tabelle nicht
Werkzeuge	Editoren brauchen einen Baum auch für unvollständigen Text

Die letzte Zeile ist neu und wiegt schwer. Ein Sprachserver, der beim Tippen Vorschläge macht, bekommt nie gültigen Quelltext zu sehen. Er braucht einen Zerteiler, der aus Bruchstücken etwas Brauchbares macht, und dafür ist ein handgeschriebener besser geeignet.

Übersetzer	Zerteiler	Bemerkung
GCC	handgeschrieben	seit 2004, vorher `bison`
Clang	handgeschrieben	von Anfang an
Rust	handgeschrieben	
Go	handgeschrieben	seit 2016

Die zweite Zeile in dieser Tabelle ist bemerkenswert: GCC hatte einen erzeugten Zerteiler und hat ihn ersetzt, in erster Linie wegen der Fehlermeldungen.

Knuth[15] begründete die LR-Analyse; der ALGOL-Bericht[22] führte die BNF ein; Grune und Mitautoren[10] behandeln alle Zerlegungsverfahren gründlich.

4 Namen und Bedeutung

Was die Syntax nicht prüfen kann

Ein syntaktisch einwandfreies Programm kann trotzdem unsinnig sein.

`x = y + 1;`	`y` wurde nie vereinbart
`f(1, 2);`	`f` erwartet drei Argumente
`int x = "hallo";`	Typen passen nicht
`return 5;`	in einer Prozedur ohne Rückgabewert

Keiner dieser Fehler lässt sich mit einer kontextfreien Grammatik ausschließen. Das liegt nicht an mangelnder Sorgfalt beim Sprachentwurf, sondern daran, dass die Bedingung „jeder benutzte Name wurde vereinbart“ nicht kontextfrei ist (Band 2, Kapitel 5).

Die semantische Analyse erledigt deshalb alles, was Kontext braucht. Sie zerfällt in zwei Aufgaben, die oft in einem Durchgang erledigt werden.

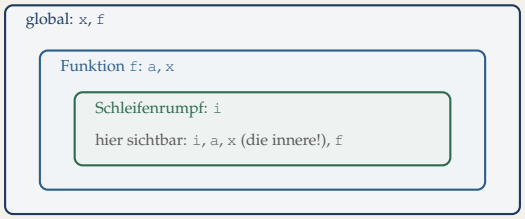
Namensauflösung	Zu jedem Vorkommen eines Namens die zugehörige Vereinbarung finden
Typprüfung	Feststellen, ob die Operationen zu den Typen passen

Die Symboltabelle

Sie bildet Namen auf das ab, was über sie bekannt ist.

Name	Art	Typ	Ort
`n`	Parameter	`int`	Stapel +8
`summe`	lokal	`int`	Stapel −4
`tabelle`	global	`int[100]`	fest
`f`	Funktion	`int(int)`	Marke

Blöcke und Gültigkeit.
Namen gelten in dem Block, in dem sie vereinbart wurden, und in allen darin enthaltenen. Ein innerer Name verdeckt einen äußeren.



Beim Nachschlagen sucht man von innen nach außen und nimmt den ersten Treffer. Das ist genau das Verhalten, das die Verdeckung erzeugt, und es kostet nichts Zusätzliches.

Die Umsetzung.

Verfahren	Nachschlagen	Blockende
Stapel von Tabellen	innen nach außen	oberste wegwerfen
Eine Tabelle mit Listen	erster Eintrag	Einträge entfernen
Unveränderliche Ketten	Zeiger folgen	nichts zu tun

Die dritte Zeile ist bei Werkzeugen beliebt, die den Zustand an einer bestimmten Stelle im Quelltext wiederherstellen wollen, etwa für Vervollständigungsvorschläge. Man hebt die alte Kette einfach auf.

Wann ein Name gilt

Die Frage, ab welcher Stelle eine Vereinbarung wirkt, wird von Sprachen verschieden beantwortet.

Sprache	gilt ab	Folge
C, C++	der Vereinbarung	Vorwärtsdeklarationen nötig
Java, C#	**dem ganzen Block**	Reihenfolge egal
Python	Zuweisung, zur Laufzeit	überraschend
JavaScript (`var`)	Funktionsanfang	berüchtigt

Die vierte Zeile hat JavaScript einen schlechten Ruf eingetragen. Eine mit `var` vereinbarte Variable ist im ganzen Funktionsrumpf sichtbar, auch vor der Vereinbarung, und hat dort den Wert `undefined`.

Die Sprache hat deshalb 2015 mit `let` eine zweite Vereinbarungsform bekommen, die sich vernünftig verhält. Die alte blieb erhalten, weil Millionen Zeilen bestehenden Codes davon abhängen.

Das ist ein wiederkehrendes Muster: Ein Sprachfehler lässt sich später nicht mehr beheben, sondern nur noch umgehen. Wer eine Sprache entwirft, sollte über Namensregeln früh sehr genau nachdenken.

Typprüfung

Der Übersetzer läuft über den Baum und bestimmt für jeden Knoten einen Typ, von unten nach oben.

Blatt	Der Typ steht in der Symboltabelle oder ergibt sich aus der Schreibweise
Operator	Aus den Typen der Kinder ergibt sich der Typ des Knotens
Zuweisung	Der Typ rechts muss zum Typ links passen

Was „passen“ bedeutet.

	streng	**mit Umwandlung**
`int` nach `double`	Fehler	automatisch
`double` nach `int`	Fehler	automatisch, verlustbehaftet
`int` nach `bool`	Fehler	in C automatisch

Die zweite Zeile ist die gefährliche. In C wird aus 3,9 *stillschweigend eine* 3*, ohne Warnung, wenn man nicht ausdrücklich danach fragt. Neuere Sprachen verlangen an dieser Stelle eine ausdrückliche Umwandlung.*

Aufgabe 4.1.

(a) Führen Sie Namensauflösung und Typprüfung an einem Programm durch.
(b) Untersuchen Sie die impliziten Umwandlungen in C.
(c) Analysieren Sie, was gute Fehlermeldungen ausmacht.

Lösung Schritt für Schritt.

(a) Ein Programm vollständig prüfen.

```
1      int x = 10;
2      double f(int a) {
3        double x = 0.5;
4        for (int i = 0; i < a; i++)
5          x = x + i;
6        return x; }
```

Schritt 1 – Die Symboltabellen je Block.

Block	Name	Typ	Zeile
global	`x`	`int`	1
global	`f`	`double(int)`	2
`f`	`a`	`int`	2
`f`	`x`	`double`	3
Schleife	`i`	`int`	4

Schritt 2 – Auflösung der Vorkommen.

Zeile	Name	gefunden in	Typ
4	`a`	`f`	`int`
4	`i`	Schleife	`int`
5	`x`	**`f`, nicht global**	`double`
5	`i`	Schleife	`int`
6	`x`	`f`	`double`

Zeile 3 der Tabelle ist die interessante. Das globale `x` vom Typ `int` ist innerhalb von `f` nicht erreichbar, weil das lokale es verdeckt. Der Übersetzer merkt das ohne Zusatzaufwand, weil er von innen nach außen sucht.

Schritt 3 – Typen im Ausdruck `x + i` bestimmen.

Knoten	Typ	Begründung
`x`	`double`	aus der Tabelle
`i`	`int`	aus der Tabelle
`+`	**`double`**	`int` wird umgewandelt

Der Übersetzer fügt an dieser Stelle einen Umwandlungsknoten in den Baum ein. Aus `x + i` wird `x + (double)i`, obwohl im Quelltext nichts davon steht.

Schritt 4 – Die Zuweisung und die Rückgabe.

Zeile	links	rechts	OK?
5	`double`	`double`	✓
6	`double` (Rückgabe)	`double`	✓

Das Programm ist korrekt. Hätte Zeile 6 `return "x";` gelautet, wäre der Fehler hier aufgefallen.

(b) Implizite Umwandlungen in C.
Die Regeln in Kurzform.

1. Alles, was kleiner ist als `int`, wird zu `int` befördert.
2. Bei verschiedenen Typen wird zum „größeren“ umgewandelt.
3. Bei gleicher Größe gewinnt **vorzeichenlos**.

Der berühmte Fall.

```
int i = -1;
unsigned int u = 1;
if (i < u) printf("kleiner");
else printf("nicht kleiner");
```

Was geschieht.

Schritt	Wert	Bemerkung
`i` als `int`	-1	
Regel 3 greift		`i` wird vorzeichenlos
`i` als `unsigned`	$4\,294\,967\,295$	Bitmuster unverändert
Vergleich	$4\,294\,967\,295 < 1$	falsch

Das Programm gibt „nicht kleiner“ aus. Minus eins ist nicht kleiner als eins. Das Bitmuster hat sich nicht geändert, nur seine Deutung, und -1 als vorzeichenlose 32-Bit-Zahl ist die größte darstellbare (Band 0, Kapitel 8).

Wie oft das vorkommt.

```
for (int i = 0; i < strlen(s); i++)
```

Der Rückgabetyp von `strlen` ist `size_t`, also vorzeichenlos. Bei einer leeren Zeichenkette und einer Schleife, die rückwärts läuft, entsteht daraus ein Fehler, den man beim Lesen nicht sieht.

Die Gegenmaßnahmen.

Warnungen	`-Wsign-compare` meldet solche Vergleiche
Sprachentwurf	Rust und Go lehnen gemischte Arithmetik ohne ausdrückliche Umwandlung ab
Vorzeichenlose meiden	Manche Projekte verbieten sie außer für Bitmuster

Die dritte Zeile ist die Empfehlung der Entwickler von C++ selbst. BJARNE STROUSTRUP *hat gesagt, die Verwendung vorzeichenloser Typen für Größen sei im Rückblick ein Fehler gewesen, den man nicht mehr beheben könne.*

(c) Was eine gute Fehlermeldung ausmacht.
Dieselbe Ursache, drei Meldungen.

Schlecht	`syntax error at line 42`
Besser	`Zeile 42: erwartet ';' vor '}'`
Gut	Zeile, Spalte, die betroffene Zeile mit Markierung, der Grund und ein Vorschlag

Woraus eine gute Meldung besteht.

Ort	Datei, Zeile, Spalte, und der Ausschnitt mit einem Zeiger darauf
Was erwartet wurde	Nicht nur, dass etwas falsch ist
Warum	Die Regel, gegen die verstoßen wurde
Der Gegenort	Bei einer nicht geschlossenen Klammer: wo sie geöffnet wurde
Ein Vorschlag	Wenn er sicher ist; sonst besser keiner

Der vierte Punkt ist der wertvollste und wird am häufigsten vergessen. Bei einer fehlenden schließenden Klammer meldet ein schlechter Übersetzer einen Fehler am Dateiende, hunderte Zeilen von der Ursache entfernt.

Folgefehler eindämmen.
Nach dem ersten Fehler weiß der Zerteiler nicht mehr, wo er ist, und meldet oft eine Kaskade.

Bis zum Anker lesen	Bis zum nächsten Semikolon oder zur schließenden Klammer überspringen
Einfügen und weiter	So tun, als wäre das fehlende Symbol da
Abbrechen	Nach zwanzig Fehlern aufhören

Die zweite Zeile ist bei fehlenden Semikola üblich und sehr wirksam. Der Zerteiler meldet den Fehler, tut so, als hätte er das Semikolon gelesen, und liest weiter. Alle folgenden Fehler sind dann echt.

Fehlermeldungen sind eine Benutzeroberfläche

Der Rust-Übersetzer hat einen erheblichen Teil seines guten Rufs den Fehlermeldungen zu verdanken, nicht dem Typsystem.

Fehlernummern	Zu jedem Fehler eine ausführliche Erklärung mit Beispielen, abrufbar über die Nummer
Mehrere Orte	Die Meldung zeigt beide beteiligten Stellen mit Beschriftung
Vorschläge	Ein konkreter Änderungsvorschlag, den Werkzeuge einspielen können

> *Der Aufwand dafür ist erheblich. Ein guter Teil des Vorderteils dient nur dazu, im Fehlerfall genug Zusammenhang zu haben, um etwas Verständliches sagen zu können.*

Clang begann diese Entwicklung um 2008, und GCC hat nachgezogen. Wer beide Übersetzer aus der Zeit davor kennt, wird bestätigen, dass hier mehr Arbeitszeit eingespart wurde als durch manche Optimierung.

Aho und Mitautoren[1] behandeln Symboltabellen und Typprüfung; Scott[25] vergleicht die Namensregeln vieler Sprachen; Hoare[11] sagt, worauf es beim Sprachentwurf ankommt.

Part II

Die Sprachen selbst

5 Typsysteme

Wozu Typen da sind

Ein Typ ist eine Zusicherung darüber, welche Werte eine Stelle annehmen kann und was man mit ihnen tun darf.

Fehler finden	Vor dem Lauf statt währenddessen
Beschreiben	Eine Signatur sagt oft mehr als ein Absatz Dokumentation
Optimieren	Wer die Größe kennt, kann Speicher sparen und Register nutzen
Überladen	Aus dem Typ ergibt sich, welche Operation gemeint ist

Der dritte Punkt wird oft übersehen. Eine Sprache ohne statische Typen muss zur Laufzeit bei jeder Addition nachsehen, was da eigentlich addiert wird. Das kostet, und Kapitel 13 zeigt, wie man es zurückgewinnt.

Zwei unabhängige Unterscheidungen.

	statisch	**dynamisch**
Geprüft wird	vor dem Lauf	während des Laufs
stark	Java, Rust, Haskell	Python, Ruby
schwach	C	JavaScript, PHP

Die beiden Achsen werden häufig verwechselt. „Statisch“ heißt, wann geprüft wird. „Stark“ heißt, wie ernst die Prüfung genommen wird, also ob sich Typen stillschweigend ineinander verwandeln.

Ausdruck	Python	JavaScript
`"1" + 1`	Fehler	`"11"`
`[] + {}`	Fehler	`"[object Object]"`
`1 < "2"`	Fehler	`true`

Beide Sprachen prüfen zur Laufzeit. Python bricht ab, JavaScript wandelt um. Das ist der Unterschied zwischen stark und schwach, und er hat mit dem Zeitpunkt der Prüfung nichts zu tun.

Typinferenz

Man kann statische Typen haben, ohne sie hinschreiben zu müssen. Der Übersetzer erschließt sie aus dem Gebrauch.

```
let doppelt f x = f (f x)
```

1. `x` hat einen unbekannten Typ, nennen wir ihn α.
2. `f` wird auf `x` angewandt, hat also die Form $\alpha \to \beta$.
3. Das Ergebnis β wird wieder an `f` übergeben, also $\beta = \alpha$.
4. Damit ist `f` vom Typ $\alpha \to \alpha$.

$$\texttt{doppelt} : (\alpha \to \alpha) \to \alpha \to \alpha.$$

Der Übersetzer hat den allgemeinsten Typ gefunden, ohne dass irgendwo ein Typ hingeschrieben wurde. `doppelt` lässt sich damit auf Zahlen, Zeichenketten und alles andere anwenden, solange die Funktion einen Typ auf sich selbst abbildet.

Das Verfahren.
ROBIN MILNER beschrieb es 1978 für die Sprache ML. Es besteht aus zwei Teilen.

Gleichungen sammeln	Aus jedem Ausdruck ergeben sich Bedingungen an die Typen
Unifikation	Ein Verfahren, das die allgemeinste Lösung des Gleichungssystems findet

Die Unifikation stammt aus der Logik und findet zu zwei Ausdrücken mit Variablen die allgemeinste Belegung, die sie gleichmacht. Sie ist auch die Grundlage von Prolog und taucht in Band 12 wieder auf.

Wie weit die Inferenz reicht

Sprache	Umfang	Bemerkung
ML, Haskell	**vollständig**	Signaturen nur zur Dokumentation
Rust	innerhalb einer Funktion	an Funktionsgrenzen Pflicht
C++, Java	`auto`, `var`	nur lokale Variablen
C	keine	alles hinschreiben

Die zweite Zeile ist eine bewusste Entscheidung, keine technische Grenze. Rust könnte die Typen von Funktionen erschließen und verlangt sie trotzdem, aus zwei Gründen.

Lesbarkeit	Wer eine Funktion benutzt, soll ihre Signatur sehen, ohne den Rumpf zu lesen
Fehlermeldungen	Ohne Signaturen wandert ein Typfehler durch das ganze Programm, bis er auffällt

Der zweite Punkt ist bei Haskell ein bekanntes Ärgernis. Ein Tippfehler in einer Hilfsfunktion kann eine Fehlermeldung in einer ganz anderen Datei erzeugen, weil der Übersetzer erst dort auf einen Widerspruch stößt.

Die Faustregel, die sich herausgebildet hat: Inferenz innerhalb, ausdrückliche Angaben an den Grenzen. Sie verbindet die Bequemlichkeit mit brauchbaren Meldungen.

Was ein Typsystem verspricht

Ein Typsystem ist ein Beweisverfahren. Es zeigt, dass bestimmte Fehler nicht auftreten können.

Zuverlässig	Wenn es kein Fehler meldet, kann der Fehler auch nicht auftreten
Vollständig	Wenn kein Fehler auftreten kann, meldet es keinen

Beides zugleich ist unmöglich, denn die Frage, ob ein Programm einen bestimmten Fehler erzeugt, ist unentscheidbar (Band 2, Kapitel 7). Jedes Typsystem muss sich also entscheiden, und alle entscheiden sich für Zuverlässigkeit.

Folge	**Beispiel**	
Manche gültigen Programme werden abgelehnt	Der Programmierer weiß mehr als der Übersetzer	
Es gibt Fluchtwege	`unsafe`, Typumwandlung	auf eigene Gefahr

Ein Beispiel für eine berechtigte Ablehnung.

```
if (aufwendigeBerechnung() == 42)
  x = 5;
print(x);      ist x gesetzt?
```

Der Übersetzer kann nicht wissen, ob die Bedingung je zutrifft, und lehnt ab. Selbst wenn die Berechnung nachweislich immer 42 liefert, bleibt es dabei: Der Prüfer arbeitet mit einer Näherung, die im Zweifel ablehnt.

Die Alternative wäre, im Zweifel durchzulassen. Dann wäre die Zusicherung wertlos, denn eine Zusicherung, die manchmal nicht gilt, ist keine.

Aufgabe 5.1.

(a) Führen Sie eine Typinferenz vollständig von Hand durch.
(b) Vergleichen Sie die Kosten statischer und dynamischer Typprüfung.
(c) Untersuchen Sie algebraische Datentypen und die Vollständigkeitsprüfung.

Lösung Schritt für Schritt.

(a) Typinferenz von Hand.
Der Ausdruck.

```
let anwenden f x = f x + 1
```

Schritt 1 – Variablen einführen.

Größe	Typvariable	
`f`	τ_f	
`x`	τ_x	
`f x`	τ_1	Ergebnis der Anwendung
Gesamtausdruck	τ_2	

Schritt 2 – Gleichungen aufstellen.

(1)	$\tau_f = \tau_x \to \tau_1$	aus der Anwendung `f x`
(2)	$\tau_1 = \texttt{int}$	linkes Argument von +
(3)	$\texttt{int} = \texttt{int}$	rechtes Argument, die `1`
(4)	$\tau_2 = \texttt{int}$	Ergebnis von +

Schritt 3 – Auflösen.

Aus	folgt	
(2)	$\tau_1 = \texttt{int}$	
(1) mit (2)	$\tau_f = \tau_x \to \texttt{int}$	
τ_x	**bleibt frei**	nichts schränkt es ein

Das Ergebnis.

$$\texttt{anwenden} : (\alpha \to \texttt{int}) \to \alpha \to \texttt{int}.$$

*Die Funktion ist **polymorph**: Sie arbeitet für jedes α. Der Übersetzer hat das erschlossen, ohne dass jemand es hingeschrieben hätte, und er hat den allgemeinsten solchen Typ gefunden.*

Die Probe an zwei Anwendungen.

Aufruf	α	gültig?
`anwenden laenge "abc"`	`string`	✓
`anwenden quadrat 5`	`int`	✓
`anwenden zeigeAn 5`	—	nein, liefert nicht `int`

(b) Was Typprüfung kostet.
Zur Übersetzungszeit.

Sprache	Anteil	Bemerkung
C	≈ 5 %	einfaches System
Java	≈ 15 %	
Rust	≈ 25 %	mit Lebensdauer-prüfung
Haskell	≈ 35 %	vollständige Inferenz

Zur Laufzeit.
Hier liegt der eigentliche Unterschied. Eine dynamisch geprüfte Addition muss nachsehen, was sie addiert.

1. Typkennzeichen beider Werte lesen.
2. Entscheiden, welche Additionsroutine zuständig ist.
3. Falls nötig, umwandeln.
4. Addieren.
5. Ergebnis mit Typkennzeichen versehen und ablegen.

Fall	Takte	Verhältnis
C, `int + int`	1	1
Java, `int + int`	1	1
Python, ohne JIT	≈ 80	**80**
JavaScript, mit JIT	≈ 1–3	je nach Vorhersagbarkeit

Die letzte Zeile ist die interessante. Wenn an einer Stelle immer nur Zahlen addiert werden, kann eine Übersetzung zur Laufzeit den Typtest herausziehen und den Rest so schnell machen wie in C. Kapitel 13 zeigt, wie.

Der Speicherbedarf.

Eine Zahl	Bytes	
C, `int`	4	
Java, `int`	4	
Java, `Integer`	16	Objektkopf
Python, `int`	28	Kopf, Zähler, Ziffern

Eine Liste von einer Million Zahlen belegt in C vier Megabyte und in Python etwa vierzig, weil jede Zahl ein eigenes Objekt mit Verwaltungsdaten ist. Deshalb gibt es Bibliotheken, die Felder von Rohwerten anbieten.

(c) Algebraische Datentypen.
Die Idee. Ein Typ ist entweder das eine oder das andere, und der Übersetzer prüft, ob man alle Fälle behandelt hat.

```
enum Form {
  Kreis { r:  f64 },
  Rechteck { b:  f64, h:  f64 },
  Dreieck { a:  f64, b:  f64, c:  f64
},
}
```

```
fn flaeche(f:  &Form) -> f64 {
  match f {
    Form::Kreis { r } => 3.14159 * r *
r,
    Form::Rechteck { b, h } => b * h,
    Form::Dreieck { a, b, c } => { ...
}
  } }
```

Was der Übersetzer daran prüft.

Vollständigkeit	Fehlt ein Fall, gibt es einen Übersetzungsfehler
Zugriff	Auf `r` kommt man nur im Kreis-Zweig; anderswo gibt es das Feld nicht
Erweiterung	Kommt später ein vierter Fall hinzu, meldet der Übersetzer **jede** Stelle, die ihn behandeln muss

Die dritte Zeile ist der praktische Hauptgewinn. Bei einer Aufzählung mit `switch` in C übersieht man die Stellen, und der Fehler zeigt sich erst im Betrieb. Hier bekommt man eine Liste aller Fundstellen.

Der wichtigste Anwendungsfall: Werte, die fehlen dürfen.

	Nullzeiger	**Optionstyp**
Darstellung	`T*`	`Option<T>`
Vergessen möglich	**ja**	nein
Fehler zeigt sich	zur Laufzeit	bei der Übersetzung

TONY HOARE *hat die Einführung des Nullzeigers 1965 in ALGOL W später seinen „Milliardenfehler“ genannt. Sein Argument: Er tat es, weil es einfach umzusetzen war, und die Folgekosten in Abstürzen und Sicherheitslücken sind kaum zu beziffern.*

Warum das lange nicht üblich war

Algebraische Datentypen gibt es seit den 1970er Jahren in ML und Hope. Sie brauchten vierzig Jahre, um in verbreiteten Sprachen anzukommen.

Sprache	Jahr	Bezeichnung
ML	1973	`datatype`
Haskell	1990	`data`
Scala	2004	`case class`
Rust	2015	`enum`
Swift	2014	`enum`
Java	2021	versiegelte Schnittstellen
C++	2017	`std::variant`, umständlich

Der Grund für die Verzögerung war die objektorientierte Denkweise. Dort löst man dasselbe Problem mit einer Basisklasse und Unterklassen, und der Übersetzer kann dann nicht mehr prüfen, ob alle Fälle behandelt sind, weil jederzeit eine weitere Unterklasse hinzukommen könnte.

	Unterklassen	Aufzählung
Neuer Fall	leicht	alle Stellen anpassen
Neue Operation	alle Klassen anpassen	leicht

Diese Spannung hat einen Namen: das **Ausdrucksproblem**. Es gibt keine Lösung, die beides leicht macht, sondern nur die Wahl, welche Richtung man häufiger braucht. Bei Formen kommen selten neue Arten hinzu, bei Benutzeroberflächen dauernd.

Pierce[24] ist die maßgebliche Darstellung der Typtheorie; Milner[19] führte die Inferenz ein; Scott[25] vergleicht die Typsysteme verbreiteter Sprachen.

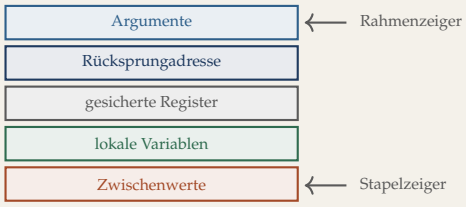
6 Speicher und Lebensdauer

Drei Orte für Daten

	Statisch	Stapel	Halde
Lebensdauer	Programmlauf	Blockende	**unklar**
Zuteilung	Übersetzer	Zeiger verschieben	Suche
Kosten	null	1–2 Takte	≈ 50 Takte
Größe	fest	fest	beliebig
Freigabe	—	automatisch	**das Problem**

Die letzte Zeile ist der Grund, warum es dieses Kapitel gibt. Alles, was auf dem Stapel liegt, verschwindet von selbst, wenn der Block endet. Bei der Halde muss jemand entscheiden, wann ein Speicherstück nicht mehr gebraucht wird.

Der Aktivierungsblock.
Bei jedem Aufruf legt das Programm einen Block auf den Stapel.



Der Rahmenzeiger ist ein fester Bezugspunkt, von dem aus alle lokalen Variablen mit einem konstanten Abstand erreichbar sind. Optimierende Übersetzer sparen ihn oft ein und rechnen alles vom Stapelzeiger aus, was ein Register frei macht und die Fehlersuche erschwert.

Zeiger auf Stapelvariablen

```
int* falsch() {
  int x = 42;
  return &x;      Zeiger auf etwas, das gleich verschwindet
}
```

Nach der Rückkehr ist der Block ungültig. Der Zeiger zeigt auf Speicher, der beim nächsten Aufruf überschrieben wird. Das Tückische ist, dass es oft scheinbar funktioniert, weil der Wert noch dasteht.

C	Warnung, wenn man Glück hat; sonst undefiniertes Verhalten
C++	dasselbe
Rust	Übersetzungsfehler durch die Lebensdauerprüfung
Java, Python	kann nicht auftreten; alles liegt auf der Halde

Der Fall ist so verbreitet, dass es eigene Werkzeuge dafür gibt. Der Address-Sanitizer erkennt ihn zur Laufzeit, indem er freigegebene Bereiche vergiftet und jeden Zugriff darauf meldet.

Wer räumt die Halde auf

Verfahren	Wer entscheidet	Gefahr
Von Hand	der Programmierer	Lecks, doppelte Freigabe
Zählen	die Laufzeit	Zyklen
Sammeln	die Laufzeit	Pausen
Eigentum	**der Übersetzer**	Lernaufwand

Zählen von Verweisen.
Jedes Objekt führt Buch darüber, wie viele Zeiger auf es zeigen. Fällt der Zähler auf null, wird freigegeben.

Vorteil	Sofortige Freigabe, keine Pausen, vorhersagbar
Nachteil	Jede Zuweisung kostet; bei Fäden sogar einen atomaren Befehl
Grenze	**Zyklen werden nie frei**

Zwei Objekte, die aufeinander zeigen, halten sich gegenseitig am Leben, auch wenn sonst niemand sie kennt. Python löst das mit einem zusätzlichen Sammler, der von Zeit zu Zeit nach Zyklen sucht.

Erreichbarkeit bestimmen.
Ein Sammler geht von den Wurzeln aus und markiert alles Erreichbare. Der Rest ist Müll.

Wurzeln	Globale Variablen, Register, alle Stapelblöcke aller Fäden
Markieren	Von dort aus allen Zeigern folgen
Kehren	Unmarkierte Bereiche freigeben

Das Verfahren ist zuverlässig, weil es die richtige Frage stellt. Es fragt nicht „wird dieses Objekt noch gebraucht“, was unentscheidbar wäre, sondern „ist es noch erreichbar“, was sich ausrechnen lässt. Unerreichbares kann sicher weg; Erreichbares wird vorsichtshalber behalten.

Warum Sammler nach Generationen trennen

Eine Beobachtung, die sich immer wieder bestätigt: Die meisten Objekte sterben sehr jung.

Lebensdauer	Anteil	
stirbt sofort	≈ 80–95 %	Zwischenergebnisse
überlebt eine Runde	≈ 5 %	
lebt lange	< 1 %	Zwischenspeicher, Konfiguration

Daraus folgt eine sehr wirksame Strategie: Man teilt die Halde in eine junge und eine alte Zone. Die junge wird häufig und schnell durchsucht, die alte selten.

Zone	Häufigkeit	Dauer
jung	alle paar ms	≈ 1 ms
alt	alle paar Minuten	10–1000 ms

Beim Durchsuchen der jungen Zone wird nur das wenige Überlebende kopiert. Der Aufwand hängt also von den lebenden Objekten ab, nicht vom Müll, und das ist der entscheidende Punkt: Je mehr Müll, desto billiger je Byte.

Ein Zeiger von einem alten in ein junges Objekt macht das kompliziert, denn dann ist die alte Zone doch eine Wurzel. Die Laufzeit merkt sich solche Zeiger in einer Nebentabelle, die bei jeder Zuweisung gepflegt wird.

Eigentum im Typsystem

Rust geht einen dritten Weg. Der Übersetzer verfolgt, wem ein Wert gehört, und fügt die Freigabe an der richtigen Stelle selbst ein.

Ein Eigentümer	Jeder Wert gehört zu jedem Zeitpunkt genau einer Variablen
Übergabe	Bei einer Zuweisung wandert das Eigentum; die alte Variable ist danach unbrauchbar
Ausleihen	Man kann Verweise vergeben, die kürzer leben als der Wert
Regel	Entweder **ein** schreibender oder **beliebig viele** lesende Verweise

Die letzte Zeile löst zwei Probleme auf einmal. Sie verhindert, dass jemand einen Verweis benutzt, während ein anderer den Wert verändert oder freigibt, und sie verhindert damit zugleich die Wettläufe aus Band 5, Kapitel 6.

	von Hand	**Sammler**	**Eigentum**
Laufzeitkosten	keine	Pausen	keine
Fehler möglich	ja	nein	nein
Lernaufwand	mittel	gering	hoch
Vorhersagbar	ja	nein	ja

Die dritte Zeile ist der Preis. Rust verlangt, dass man über Lebensdauern nachdenkt, bevor das Programm läuft, und das ist ungewohnt. Wer aus C kommt, hat dieselben Gedanken schon immer gedacht, nur ohne Hilfe.

Aufgabe 6.1.

(a) Verfolgen Sie einen Aufruf auf dem Stapel vollständig.
(b) Rechnen Sie die Kosten verschiedener Speicherverwaltungen aus.
(c) Untersuchen Sie, wie Abschlüsse Speicher am Leben halten.

Lösung Schritt für Schritt.

(a) Ein Aufruf auf dem Stapel.

```
int fak(int n) {
  if (n <= 1) return 1;
  return n * fak(n - 1); }
```

Der Stapel bei `fak(3)`, tiefster Punkt.

Adresse	Inhalt	gehört zu	
$0x7000$	`n = 3`	Aufruf 1	
$0x6ff8$	Rücksprung	Aufruf 1	zum Aufrufer
$0x6ff0$	`n = 2`	Aufruf 2	
$0x6fe8$	Rücksprung	Aufruf 2	in `fak`
$0x6fe0$	`n = 1`	Aufruf 3	
$0x6fd8$	Rücksprung	Aufruf 3	in `fak`

Das Abwickeln.

Schritt	Aufruf	liefert	Stapel
1	`fak(1)`	1	-16 B
2	`fak(2)`	$2 \cdot 1 = 2$	-16 B
3	`fak(3)`	$3 \cdot 2 = 6$	-16 B

Der Speicherbedarf in Abhängigkeit von n.

$$\text{Tiefe} = n, \qquad \text{Speicher} = 16n \text{ Byte}.$$

n	Stapel	Bemerkung
100	1,6 KB	unproblematisch
10^5	1,6 MB	
$5 \cdot 10^5$	8 MB	Standardgrenze erreicht
10^6	16 MB	Stapelüberlauf

Die Endrekursion.
Schreibt man die Funktion so um, dass der rekursive Aufruf das Letzte ist, was geschieht, kann der Übersetzer den alten Block wiederverwenden.

```
int fak2(int n, int acc) {
  if (n <= 1) return acc;
  return fak2(n - 1, n * acc); }
```

Fassung	Stapel	bei $n = 10^6$
`fak`	$\mathcal{O}(n)$	Absturz
`fak2`, optimiert	$\mathcal{O}(1)$	16 Byte

Der Unterschied liegt darin, dass bei `fak` nach dem Aufruf noch eine Multiplikation folgt. Der Block muss deshalb erhalten bleiben. Bei `fak2` folgt nichts mehr, und aus dem Aufruf kann ein Sprung werden.

Nicht jede Sprache garantiert das

Sprache	Endrekursion	Bemerkung
Scheme	**garantiert**	Teil der Norm
Haskell, OCaml	garantiert	
C, C++	ab `-O2` meist	keine Zusage
Java, Python	nein	bewusste Entscheidung

Bei C ist das unangenehm: Ein Programm, das mit Optimierung läuft, stürzt ohne sie ab. Wer sich darauf verlässt, sollte es prüfen.

Python verzichtet bewusst darauf. GUIDO VAN ROSSUM hat das damit begründet, dass die Aufrufliste im Fehlerfall vollständig sichtbar bleiben soll. Bei entfernten Blöcken fehlt sie, und die Fehlersuche wird schwerer.

Das ist ein echter Kompromiss und keine Nachlässigkeit: Man tauscht die Möglichkeit tiefer Rekursion gegen bessere Fehlerberichte.

(b) Was Speicherverwaltung kostet.
Der Fall. Ein Programm legt 10^7 kleine Objekte an, von denen 90 Prozent sofort wieder sterben.

Verfahren	Takte je Objekt	gesamt
Stapel	2	0,005 s
Halde von Hand	$50 + 40$	0,23 s
Zählen	$50 + 40 + 20$	0,28 s
Generationen-sammler	**5** + Sammeln	$\approx$ 0,04 s

Das überrascht viele: Ein guter Sammler kann schneller sein als Zuteilung von Hand. Das Anlegen kostet nur das Verschieben eines Zeigers, und die Freigabe kostet für die 90 Prozent Müll überhaupt nichts, weil sie nie angefasst werden.

Was die Rechnung nicht zeigt.

Speicherbedarf	Ein Sammler braucht Reserve; üblich sind Faktor zwei bis fünf
Pausen	Der Durchsatz stimmt, einzelne Anfragen können warten
Lokalität	Ein kopierender Sammler ordnet Objekte neu und verbessert damit die Cachenutzung

Die Pausen im Vergleich.

Sammler	Pause	Jahr
Anhalten und kehren	100–1000 ms	1960er
Generationen	1–50 ms	1980er
G1 (Java)	$\approx$ 10 ms	2012
ZGC, Shenandoah	$<$ 1 ms	2018
Go	$<$ 0,5 ms	seit 2016

Die Pausen sind kein grundsätzliches Problem mehr. Der Preis dafür ist, dass die Sammler nebenläufig zum Programm arbeiten, was Durchsatz kostet und ihre Umsetzung erheblich verkompliziert.

(c) Abschlüsse und Lebensdauer.
Der Fall.

```
function zaehler() {
  let n = 0;
  return function() { return ++n; };
}
const c = zaehler();
c(); c();  // liefert 1, dann 2
```

Die Variable `n` gehört zu einem Aufruf, der längst zurückgekehrt ist, und lebt trotzdem weiter. Sie kann also nicht auf dem Stapel liegen.

Was der Übersetzer tut.

1. Er stellt fest, welche äußeren Variablen die innere Funktion benutzt.
2. Diese werden nicht auf dem Stapel angelegt, sondern auf der Halde.
3. Die innere Funktion wird zu einem Paar aus Code und Umgebung.
4. Die Umgebung hält die Variablen am Leben, solange die Funktion lebt.

Abschluss	Ein Paar (Code, Umgebung), oft **Closure** genannt
Hochziehen	Die Verlagerung einer Variablen vom Stapel auf die Halde

Wie verschiedene Sprachen damit umgehen.

Sprache	Vorgehen	Preis
JavaScript, Python	alles auf der Halde	Sammler nötig
Go	**Fluchtanalyse**	bleibt auf dem Stapel, wenn möglich
Rust	Eigentum ausdrücklich	`move` verlangt
C	gibt es nicht	Funktionszeiger ohne Umgebung

Die Fluchtanalyse ist eine schöne Optimierung: Der Übersetzer prüft, ob ein Wert die Funktion überlebt. Wenn nicht, bleibt er auf dem Stapel und kostet nichts. Nur was tatsächlich entkommt, landet auf der Halde.

Fall	entkommt?	liegt
lokal, nur gelesen	nein	Stapel
Zeiger zurückgegeben	**ja**	Halde
in einem Abschluss benutzt	ja	Halde
an eine Goroutine übergeben	ja	Halde

Go zeigt die Entscheidungen mit `go build -gcflags=-m` an. Das ist eines der nützlichsten Werkzeuge zur Leistungsuntersuchung, weil Zuteilungen auf der Halde der häufigste vermeidbare Aufwand sind.

Jones und Mitautoren[13] sind die maßgebliche Darstellung der Speicherbereinigung; Appel[2] behandelt Aktivierungsblöcke und Abschlüsse gründlich; Scott[25] vergleicht die Modelle verschiedener Sprachen.

7 Abstraktionsmittel

Wozu Abstraktion dient

Jedes Abstraktionsmittel einer Sprache verfolgt dasselbe Ziel: einen Namen für etwas einzuführen, damit man es benutzen kann, ohne es zu kennen.

Mittel	verbirgt	seit
Prozedur	einen Ablauf	1950er
Verbund	die Anordnung von Feldern	1960er
Modul	eine Gruppe von Namen	1970er
Abstrakter Typ	die Darstellung	1970er
Generizität	den Elementtyp	1970er
Objekt	Daten und Verhalten zugleich	1960er/1980er

Die vierte Zeile ist die folgenreichste. Wenn niemand außerhalb weiß, wie ein Wert dargestellt ist, kann man die Darstellung später ändern, ohne dass etwas bricht. Das ist der eigentliche Gewinn, und alles andere folgt daraus.

Was Generizität kostet

Eine Liste soll für jeden Elementtyp funktionieren. Dafür gibt es zwei Wege, die sich grundsätzlich unterscheiden.

	Ausprägung	Löschung
Vorgehen	je Typ eigener Code	ein Code für alle
Sprachen	C++, Rust	Java, Haskell
Laufzeit	wie handgeschrieben	Zeiger und Umwege
Codegröße	wächst	konstant
Übersetzungszeit	hoch	gering
Fehlermeldungen	berüchtigt	besser

Die vorletzte Zeile ist ein realer Kostenpunkt. Ein C++-Projekt, das stark mit Schablonen arbeitet, kann seine Übersetzungszeit vervielfachen, weil derselbe Code für zwanzig Typen erzeugt und optimiert wird.

Liste für n Typen	Codegröße	Zugriff
Ausprägung	$n \cdot 4$ KB	1 Takt
Löschung	4 KB	3–5 Takte

Die Fehlermeldungen bei Ausprägung

Wenn ein Fehler erst beim Einsetzen des konkreten Typs auffällt, meldet der Übersetzer ihn tief im Inneren der Bibliothek.

> *Eine falsche Vergleichsfunktion beim Sortieren erzeugte in älteren C++-Fassungen Meldungen von mehreren hundert Zeilen, die auf Bibliothekscode zeigten, den der Programmierer nie gesehen hatte.*

Die Antwort darauf sind **Anforderungen**: Man schreibt hin, was ein Typ können muss, und der Übersetzer prüft das an der Aufrufstelle.

Sprache	Jahr	Bezeichnung
Haskell	1988	Typklassen
Rust	2015	`trait`-Schranken
C++	2020	`concept`

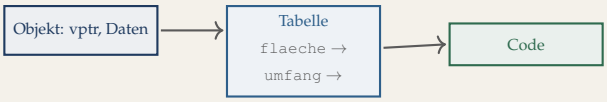
Der Unterschied ist bemerkenswert: Statt „Fehler in Zeile 4711 von `stl_algo.h`“ sagt der Übersetzer „`Punkt` erfüllt `Ord` nicht, weil ein Vergleich fehlt“. Dieselbe Prüfung, an der richtigen Stelle gemeldet.

Wie ein Methodenaufruf gefunden wird

Bei `f.flaeche()` muss zur Laufzeit entschieden werden, welcher Code gemeint ist.

Verfahren	Aufwand	Sprachen
Statisch	1 Takt	C, nicht virtuelle Methoden
Methodentabelle	≈ 3 Takte	C++, Java
Suche in der Kette	≈ 20 Takte	Python, Ruby
Zwischenspeicher	≈ 2 Takte	JavaScript mit JIT

Die Methodentabelle.
Jedes Objekt trägt einen Zeiger auf eine Tabelle, in der die Adressen seiner Methoden stehen. Der Aufruf ist dann ein Tabellenzugriff und ein indirekter Sprung.



Der indirekte Sprung ist das eigentlich Teure, nicht der Tabellenzugriff. Der Prozessor muss vorhersagen, wohin es geht (Band 4, Kapitel 7), und liegt er falsch, kostet das fünfzehn bis zwanzig Takte.

Aufrufstelle	Vorhersage	Kosten
immer derselbe Typ	sehr gut	≈ 3 Takte
zwei Typen im Wechsel	mäßig	≈ 8 Takte
viele Typen	schlecht	≈ 20 Takte

In der Praxis ist die erste Zeile der Normalfall. Über 90 Prozent aller virtuellen Aufrufstellen sehen zur Laufzeit nur einen einzigen Typ. Genau darauf beruht das Verfahren aus Kapitel 13.

Sprachen der vierten Generation

In den 1980er Jahren tauchte eine Klasse von Sprachen auf, die einen anderen Weg gingen als C oder Pascal. Sie zielten nicht darauf, allgemein verwendbar zu sein, sondern darauf, eine bestimmte Art von Anwendung mit sehr wenig Code zu erledigen: kaufmännische Programme mit Bildschirmmasken und Datenbankzugriff.

> *Der Name „vierte Generation“ sollte an die Reihe Maschinencode, Assembler, Hochsprache anschließen. Er war Werbung und beschrieb doch etwas Reales: Diese Sprachen hatten Sprachmittel, die anderswo Bibliotheken waren.*

Was das konkret bedeutet.
In einer kaufmännischen Anwendung geht der größte Teil der Arbeit in die Dialoggestaltung: Masken aufbauen, Eingaben prüfen, Felder nachladen, Fehler melden. Genau dieser Teil ist zugleich die häufigste Fehlerquelle.

	Allgemeine Sprache	4GL
Maske	Bibliotheksaufrufe	**Sprachkonstruktion**
Datenzugriff	Schnittstelle	eingebaut
Dezimalrechnung	Zusatzbibliothek	eingebauter Typ
Zielgruppe	alles	**ein Anwendungsgebiet**

Zu dieser Familie gehören die Business-BASIC-Dialekte der mittleren Datentechnik (Band 5, Kapitel 3), aber auch eigenständige Entwicklungen.

> *Ein deutsches Beispiel ist ROSI-SQL, das die Halstenbach ACT GmbH in Wiehl Anfang der 1990er Jahre entwickelte: eine 4GL mit eingebauter SQL-Schnittstelle und einem Dialogverwaltungssystem, verfügbar für Unix, Linux und Windows. Die Nachfolgefassung ROSI 2000 bindet Datenbanken wie Informix, Oracle oder MySQL an.*

Bemerkenswert am Entwurf solcher Sprachen ist die ausdrückliche Zielsetzung: eine „sprechende“, selbstdokumentierende Syntax, damit der Schulungsaufwand klein bleibt und man das Programm auch nach fünf Jahren noch lesen kann.

Stärke	Für ihr Gebiet sehr wenig Code, sehr schnelle Entwicklung
Schwäche	Außerhalb des Gebiets unbrauchbar oder umständlich
Bindung	Sprache, Werkzeug und Hersteller sind dasselbe

> *Die dritte Zeile erklärt, warum die 4GL-Welle abebbte. Wer eine solche Sprache wählte, band sich an einen Anbieter. Als die Anwendungen ins Netz wanderten, mussten viele dieser Sprachen einen Übergang schaffen, der ihnen nicht immer gelang.*

Verschwunden ist der Gedanke trotzdem nicht. Er heißt heute **fachspezifische Sprache** und ist gut angesehen.

SQL	Anfragen, seit 1974 und unverändert erfolgreich
Reguläre Ausdrücke	Textmuster
HTML, CSS	Aufbau und Gestaltung von Seiten
Terraform, Ansible	Beschreibung von Anlagen

Der Unterschied zu den 4GL der 1980er Jahre liegt weniger in der Idee als im Umfang. Eine fachspezifische Sprache deckt heute meist einen Ausschnitt ab und wird in eine allgemeine Sprache eingebettet, statt zu versuchen, alles selbst zu erledigen.

Aufgabe 7.1.

(a) Rechnen Sie die Kosten von Vererbung und Methodentabellen aus.
(b) Vergleichen Sie Schnittstellen und Typklassen.
(c) Untersuchen Sie, wann Abstraktion nichts kostet.

Lösung Schritt für Schritt.

(a) Vererbung und Tabellen.
Das Speicherbild bei einfacher Vererbung.

Abstand	Inhalt	aus	
0	Tabellenzeiger	—	8 B
8	`x`	Basis	4 B
12	`y`	Basis	4 B
16	`r`	abgeleitet	8 B

Die Felder der Basis stehen vorn und an denselben Stellen wie beim Basisobjekt. Deshalb kann man einen Zeiger auf das abgeleitete Objekt überall dort verwenden, wo ein Basiszeiger erwartet wird, ohne etwas umzurechnen.

Was ein virtueller Aufruf kostet.

Schritt	Takte	Bemerkung
Tabellenzeiger laden	1	meist im Cache
Methodenadresse laden	1	
indirekter Sprung	1	bei richtiger Vorhersage
günstig	**3**	
bei Fehlvorhersage	**18**	

Was Mehrfachvererbung daraus macht.
Wenn eine Klasse von zwei Basisklassen erbt, können nicht beide am Abstand null stehen.

Zwei Tabellen	Das Objekt trägt mehrere Tabellenzeiger
Zeigerkorrektur	Bei der Umwandlung auf die zweite Basis wird eine Konstante addiert
Sprungpolster	Kleine Zwischenroutinen richten den Zeiger vor dem Aufruf zurecht

Aus einem Takt Zeigerumwandlung werden mehrere, und aus einem einfachen Sprung wird einer über ein Polster. Der Aufwand bleibt klein, die Umsetzung wird deutlich komplizierter.

Sprache	Mehrfachvererbung	stattdessen
C++	**ja**, mit virtuellen Basen	
Java	nur Schnittstellen	Vorgabemethoden
Rust	keine Vererbung	`trait` und Komposition
Go	keine Vererbung	Einbettung, Schnittstellen

Die letzten beiden Zeilen sind eine bewusste Entscheidung neuerer Sprachen. Die Erfahrung mit tiefen Klassenhierarchien war überwiegend schlecht, weil eine Änderung an der Wurzel alles darunter betrifft.

(b) Schnittstellen und Typklassen.
Der Unterschied in einem Satz. Bei einer Schnittstelle sagt der Typ, welche er erfüllt. Bei einer Typklasse sagt es jemand anderes, auch nachträglich.

	Schnittstelle	**Typklasse**
Erklärt bei	der Klasse	**getrennt**
Fremde Typen	nicht nachrüstbar	nachrüstbar
Auswahl	über das Objekt	über den **statischen** Typ
Rückgabetyp	schwierig	möglich

Warum Zeile 2 wichtig ist.
Wenn eine Bibliothek einen Typ liefert und eine andere eine Schnittstelle, kann man beide nicht verbinden, ohne eine davon zu ändern.

Java	Ein Adapter, der den fremden Typ umhüllt
Haskell, Rust	Eine Instanz für den fremden Typ schreiben; fertig

*Rust schränkt das mit der **Waisenregel** ein: Entweder der Typ oder die Eigenschaft muss aus dem eigenen Modul stammen. Sonst könnten zwei Bibliotheken widersprüchliche Instanzen liefern, und niemand wüsste, welche gilt.*

Warum Zeile 4 wichtig ist.

```
trait Standard { fn standard() ->
Self; }
```

Diese Funktion hat kein Objekt, an dem man sie aufrufen könnte. Sie liefert eines. Bei einer Schnittstelle im Java-Sinn ist das unmöglich, weil die Auswahl über ein vorhandenes Objekt läuft. Bei einer Typklasse geht es, weil der gewünschte Typ die Auswahl bestimmt.

(c) Wann Abstraktion nichts kostet.
Der Grundsatz. Was der Übersetzer zur Übersetzungszeit auflösen kann, kostet zur Laufzeit nichts.

Abstraktion	Laufzeitkosten	Bedingung
Prozedur	null	wird eingesetzt
Verbund	null	Felder direkt erreichbar
Generizität, ausgeprägt	null	Typ steht fest
Schnittstelle, ein Typ	fast null	JIT kann spezialisieren
Schnittstelle, viele Typen	3–20 Takte	Sprung nicht vorhersagbar
Dynamische Suche	≈ 20 Takte	

Der Nachweis am Beispiel.

```
struct Meter(f64);

fn addiere(a: Meter, b: Meter) ->
Meter { Meter(a.0 + b.0) }
```

Fassung	Befehle	Laufzeit
mit `f64` unmittelbar	1	1,00
mit `Meter`	**1**	1,00

Der Typ `Meter` verschwindet vollständig. Er existiert nur für den Übersetzer, der damit verhindert, dass jemand Meter zu Sekunden addiert. Zur Laufzeit ist es eine Gleitkommazahl.

*Man nennt das **aufwandsfreie Abstraktion**. Sie ist der Leitgedanke von C++ und Rust: Man soll nicht dafür bezahlen, was man nicht benutzt, und was man benutzt, soll nicht teurer sein, als wenn man es von Hand geschrieben hätte.*

Die Grenzen der Aufwandsfreiheit

Sie gilt für den erzeugten Code, nicht für alles Übrige.

Übersetzungszeit	Wächst erheblich; ein großes C++-Projekt übersetzt Stunden
Codegröße	Jede Ausprägung erzeugt eigenen Code, was den Befehlscache belastet
Fehlersuche	Ohne Optimierung sind es viele kleine Funktionsaufrufe; mit Optimierung ist der Bezug zum Quelltext verloren
Lesbarkeit	Eine Zeile kann sehr viel bedeuten

Die zweite Zeile ist ein realer Effekt. Codegröße ist keine kostenlose Größe: Wenn der Befehlscache überläuft, wird das Programm langsamer, obwohl jeder einzelne Teil optimal ist.

Die dritte Zeile trifft jeden, der schon einmal optimierten C++-Code im Fehlersuchprogramm betrachtet hat. Der Zähler springt scheinbar wahllos, weil zwanzig kleine Funktionen in eine verschmolzen wurden.

Deshalb gibt es die Zwischenstufe `-Og`: optimieren, aber nur so weit, dass die Zuordnung zum Quelltext erhalten bleibt.

Hoare[11] nennt die Grundsätze des Sprachentwurfs; Scott[25] vergleicht die Abstraktionsmittel systematisch; Wirth[27] zeigt, wozu sie eigentlich da sind.

Part III

Vom Baum zum Befehl

8 Zwischendarstellungen

Warum nicht gleich Maschinencode

Der Syntaxbaum ist zu hoch, der Maschinencode zu niedrig. Dazwischen braucht man eine Form, in der sich umformen lässt.

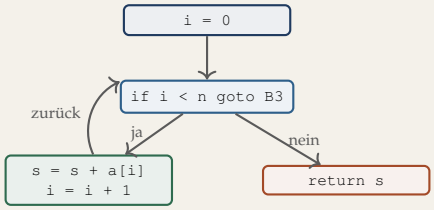
	Syntaxbaum	Zwischenform
Struktur	verschachtelt	**flach**
Kontrollfluss	implizit	ausdrücklich, Sprünge
Ausdrücke	beliebig tief	höchstens ein Operator
Variablen	Namen	**beliebig viele Hilfsnamen**

Dreiadresscode.
Jede Anweisung hat höchstens einen Operator und drei beteiligte Größen.

`a = b + c`	binäre Operation
`a = -b`	unäre Operation
`a = b`	Kopie
`goto M`	unbedingter Sprung
`if a < b goto M`	bedingter Sprung
`a = f(b, c)`	Aufruf
`a = *p, *p = a`	Speicherzugriff

Sieben Formen genügen, um jedes Programm auszudrücken. Das ist der Punkt: Ein Optimierer muss nur diese sieben Fälle behandeln statt der hundert Knotenarten eines Syntaxbaums.

Der Kontrollflussgraph.
Man teilt den Code in Blöcke, die keine Sprünge enthalten, und verbindet sie mit Kanten.



Damit ist das Programm ein Graph, und die Optimierung wird zu einer Aufgabe der Graphentheorie (Band 1, Kapitel 12). Schleifen sind Zyklen, unerreichbarer Code sind Knoten ohne eingehende Kante.

Die SSA-Form

Eine Zwischenform, in der jede Variable genau einmal geschrieben wird. Wird sie mehrfach zugewiesen, bekommt jede Zuweisung eine neue Nummer.

gewöhnlich	in SSA
`x = 1`	`x1 = 1`
`x = x + 2`	`x2 = x1 + 2`
`y = x * 3`	`y1 = x2 * 3`

Der Gewinn: Zu jeder Benutzung gibt es genau eine Definition, und man sieht sie am Namen. Fragen wie „welchen Wert hat x hier“ beantworten sich, ohne dass man den Kontrollfluss verfolgen muss.

Das Problem an Zusammenflüssen.
Wenn zwei Wege zusammenkommen, hängt der Wert davon ab, woher man kommt.

```
if (b) x1 = 1; else x2 = 2;
x3 = φ(x1, x2);      welches von beiden?
y1 = x3 + 1;
```

Die φ-Funktion	Wählt den Wert je nachdem, über welche Kante man den Block betreten hat
Sie ist fiktiv	Es gibt keinen Maschinenbefehl dafür; sie wird später aufgelöst
Auflösung	Am Ende jedes Vorgängerblocks wird eine Kopie eingefügt

Die φ-Funktion ist eine Buchführungshilfe. Sie macht die Datenflussfragen leicht, und wenn die Optimierungen fertig sind, wird sie in gewöhnliche Zuweisungen zurückverwandelt.

Was dadurch einfach wird.

Optimierung	**ohne SSA**	**mit SSA**
Konstanten weiterreichen	Datenflussanalyse	Definition ansehen
Toten Code entfernen	Analyse je Block	keine Benutzung, weg
Gemeinsame Teilausdrücke	Vergleich nötig	gleicher Name
Aufwand	$\mathcal{O}(n^2)$ typisch	$\mathcal{O}(n)$ typisch

Deshalb hat sich die Form seit 1991 durchgesetzt. GCC, LLVM, die Java-VM, V8 und praktisch jeder andere ernsthafte Übersetzer arbeiten damit.

C als Zwischensprache

Es gibt eine Abkürzung, die viele Übersetzer genommen haben: Man erzeugt gar keinen Maschinencode, sondern C, und überlässt den Rest einem vorhandenen C-Übersetzer.

Vorteil	Alle Zielrechner, für die es einen C-Übersetzer gibt, und das sind alle
Vorteil	Man erbt die gesamte Optimierung, ohne sie zu schreiben
Vorteil	Anbindung an vorhandene Bibliotheken ist trivial
Nachteil	Fehlermeldungen zeigen auf erzeugten C-Code
Nachteil	Was C nicht ausdrücken kann, wird umständlich

System	**Jahr**	**Bemerkung**
cfront (C++)	1983	die erste C++-Umsetzung
Eiffel	1986	
FlagShip	**1992**	xBase nach C
Nim	2008	bis heute
Emscripten	2011	C nach JavaScript, umgekehrte Richtung

Die dritte Zeile ist ein deutsches Beispiel und ein besonders klarer Fall. FlagShip von multisoft Datentechnik in Landshut übersetzte xBase-Quelltext nach C und ließ dann den vorhandenen Übersetzer arbeiten. Damit ließen sich Anwendungen, die für DOS geschrieben waren, auf Unix-Systeme bringen.

Der vierte Punkt der Tabelle oben ist der eigentliche Preis. Wenn der erzeugte C-Code einen Fehler enthält, meldet der C-Übersetzer ihn an einer Stelle, die im ursprünglichen Programm gar nicht existiert.

`#line`	C erlaubt es, Zeilennummern umzubiegen, sodass Meldungen auf den ursprünglichen Text zeigen
Was fehlt	Endrekursion, eigene Aufrufkonventionen, genaue Kontrolle über den Stapel

Die zweite Zeile ist der Grund, warum funktionale Sprachen diesen Weg meist verlassen haben. Wer garantierte Endrekursion braucht, kann sie in C nicht ausdrücken und muss zu Kunstgriffen greifen, die den Gewinn wieder auffressen.

Der Gedanke selbst ist trotzdem lebendig. Heute heißt die Zwischensprache oft LLVM oder WebAssembly statt C, und das Argument ist dasselbe: Man schreibt einen Vorderteil und erbt zwanzig Zielarchitekturen.

Aufgabe 8.1.

(a) Übersetzen Sie eine Schleife vollständig in Zwischencode.
(b) Bringen Sie ein Programm in SSA-Form.
(c) Vergleichen Sie Zwischenformen verschiedener Übersetzer.

Lösung Schritt für Schritt.

(a) Eine Schleife in Zwischencode.
Der Quelltext.

```
s = 0;
for (i = 0; i < n; i++)
  s = s + a[i];
```

Schritt 1 – Wörtliche Übersetzung.

```
1     s = 0
2     i = 0
3     L1:  if i >= n goto L2
4     t1 = i * 4      Elementgröße
5     t2 = a + t1     Adresse
6     t3 = *t2
7     s = s + t3
8     i = i + 1
9     goto L1
10    L2:
```

Drei Quellzeilen sind zu zehn Anweisungen geworden. Die Adressrechnung, die im Quelltext als `a[i]` dasteht, ist jetzt ausdrücklich.

Schritt 2 – Die Multiplikation entfernen.

Bei jedem Durchlauf wächst `i` um eins, also `t1` um vier. Man kann fortlaufend addieren statt jedes Mal zu multiplizieren.

```
1       s = 0
2       i = 0
3       t2 = a      Zeiger auf das erste Element
4       L1:  if i >= n goto L2
5       t3 = *t2
6       s = s + t3
7       t2 = t2 + 4
8       i = i + 1
9       goto L1
```

Eine Multiplikation und eine Addition je Durchlauf sind zu einer Addition geworden. Man nennt das ***Ersetzung induktiver Variablen****, und es ist die wirksamste Einzeloptimierung für Schleifen über Felder.*

Schritt 3 – Die Zählvariable loswerden.
`i` wird nur noch für den Vergleich gebraucht. Man kann stattdessen `t2` vergleichen.

```
1       s = 0
2       t2 = a
3       tend = a + n * 4      einmal, vor der
        Schleife
4       L1:  if t2 >= tend goto L2
5       t3 = *t2
6       s = s + t3
7       t2 = t2 + 4
8       goto L1
```

Die Bilanz.

Fassung	Anweisungen je Durchlauf	Verhältnis
wörtlich	6	1,00
ohne Multiplikation	5	0,83
ohne Zähler	**4**	**0,67**

Ein Drittel weniger Arbeit, ohne dass am Quelltext etwas geändert wurde. Und der C-Programmierer, der stattdessen von Hand mit Zeigern gearbeitet hätte, bekommt heute nichts Besseres.

(b) Ein Programm in SSA-Form.
Der Ausgangscode.

```
x = 1;
if (b) { x = 2; y = x + 1; }
else { y = x + 5; }
z = x + y;
```

Schritt 1 – Blöcke bilden.

Block	Inhalt	Nachfolger
B_0	`x = 1`, Test	B_1, B_2
B_1	`x = 2`, `y = x + 1`	B_3
B_2	`y = x + 5`	B_3
B_3	`z = x + y`	—

Schritt 2 – Nummerieren.

B_0	`x1 = 1`
B_1	`x2 = 2;  y1 = x2 + 1`
B_2	`y2 = x1 + 5`
B_3	**hier fließt beides zusammen**

Schritt 3 – φ-Funktionen einsetzen.

B_3	`x3 = φ(x2 aus B1, x1 aus B2)`
	`y3 = φ(y1 aus B1, y2 aus B2)`
	`z1 = x3 + y3`

Schritt 4 – Was man jetzt sofort sieht.

`y1 = x2 + 1`	`x2` ist die Konstante 2, also ist `y1` gleich 3
`y2 = x1 + 5`	`x1` ist 1, also ist `y2` gleich 6

Man musste dafür nirgends nachsehen, welcher Zweig gerade gilt. Der Name `x2` sagt bereits, welche Definition gemeint ist, und die steht unmittelbar daneben.

Das Ergebnis nach der Konstantenweitergabe.

B_1	`y1 = 3`
B_2	`y2 = 6`
B_3	`x3 = φ(2, 1);  y3 = φ(3, 6);` `z1 = x3 + y3`

Zwei Additionen sind verschwunden. Ein Optimierer, der noch weiter geht, erkennt, dass die beiden φ-Funktionen von derselben Bedingung abhängen, und kann `z1` zu `φ(5, 7)` zusammenziehen.

(c) Zwischenformen im Vergleich.

System	Form	SSA	Ebene
GCC	GIMPLE, RTL	ja	zwei Stufen
LLVM	LLVM IR	ja	eine, typisiert
Rust	HIR, MIR, LLVM	MIR ja	**drei**
Java-VM	Bytecode	nein, Stapel	für Ausführung
V8	Turbofan-Graph	ja	Datenfluss
WebAssembly	Stapelmaschine	nein	für Ausführung

Zwei Arten von Zwischenform.

Zum Optimieren	Registerbasiert, in SSA, beliebig viele Hilfsnamen. Nur im Übersetzer
Zum Ausliefern	Stapelbasiert, kompakt, leicht zu prüfen. Wird gespeichert und verschickt

Der Unterschied erklärt, warum Java-Bytecode und WebAssembly stapelbasiert sind. Sie sollen klein sein und sich schnell auf Gültigkeit prüfen lassen, nicht optimiert werden. Die Optimierung geschieht später, in einer anderen Form.

Warum Rust drei Ebenen hat.

Ebene	**nah an**	**wofür**
HIR	dem Quelltext	Typprüfung, Fehlermeldungen
MIR	dem Kontrollfluss	**Eigentumsprüfung**
LLVM IR	der Maschine	Optimierung, Codeerzeugung

Die mittlere Ebene wurde 2016 eingeführt, weil sich die Lebensdauerprüfung im Syntaxbaum nicht sauber machen ließ. Sie brauchte einen Kontrollflussgraphen, und den gab es vorher erst in LLVM, wo alle Typinformation schon verloren war.

Cytron und Mitautoren[7] führten die SSA-Form ein; Muchnick[20] behandelt Zwischenformen ausführlich; Lattner und Adve[16] beschreiben den Entwurf von LLVM.

9 Optimierung

Was erlaubt ist

Eine Umformung ist zulässig, wenn das Programm danach dasselbe tut. Die Frage ist, was „dasselbe“ bedeutet.

Beobachtbar	Ein- und Ausgabe, Rückgabewerte, Zugriffe auf flüchtige Speicherstellen
Nicht beobachtbar	Laufzeit, Speicherverbrauch, Reihenfolge unabhängiger Rechnungen, Registerbelegung

> *Der Übersetzer darf also alles ändern, was man von außen nicht sieht. Die Sprachnormen beschreiben das mit dem Begriff der **abstrakten Maschine**: Ein Programm muss sich verhalten, als liefe es auf ihr, und wie es das erreicht, ist seine Sache.*

Der Werkzeugkasten.

Umformung	Reichweite	Gewinn
Konstanten falten	lokal	klein
Konstanten weiterreichen	global	mittel
Toten Code entfernen	global	klein bis mittel
Gemeinsame Teilausdrücke	global	mittel
Stärke mindern	lokal	klein
Einsetzen von Funktionen	**zwischen Funktionen**	**groß**
Schleifeninvariantes herausziehen	Schleife	groß
Schleife entrollen	Schleife	mittel
Vektorisieren	Schleife	sehr groß

Die hervorgehobene Zeile ist die wichtigste, und zwar nicht wegen des eingesparten Aufrufs. Sie ist wichtig, weil sie alle anderen Optimierungen erst ermöglicht: Erst wenn der Code der aufgerufenen Funktion an Ort und Stelle steht, kann der Übersetzer über beide zusammen nachdenken.

Ohne Einsetzen	**Gewinn**	
alle Optimierungen	1,0	Grundlage
ohne Einsetzen	0,6	40 % verloren
nur Einsetzen	0,8	

Datenflussanalyse

Viele Optimierungen brauchen die Antwort auf Fragen der Form: Was gilt an dieser Stelle, auf allen Wegen dorthin?

Erreichende Definitionen	Welche Zuweisungen können den Wert hier gesetzt haben
Lebendige Variablen	Wird dieser Wert später noch gebraucht
Verfügbare Ausdrücke	Wurde das schon einmal ausgerechnet

Das Verfahren.
Man stellt für jeden Block eine Gleichung auf und löst das System durch Wiederholung, bis sich nichts mehr ändert.

1. Jeder Block bekommt eine Anfangsmenge.
2. Aus den Vorgängern ergibt sich, was hereinkommt.
3. Der Block verändert das: Er tötet manches und erzeugt anderes.
4. Wiederholen, bis sich nichts mehr ändert.

$$\text{aus}(B) = \text{erzeugt}(B) \cup \big(\text{ein}(B) \setminus \text{tötet}(B)\big).$$

Dass dieses Verfahren anhält, ist nicht selbstverständlich und lässt sich beweisen: Die Mengen wachsen nur und sind nach oben beschränkt, also muss es einen Fixpunkt geben. Das ist derselbe Gedanke wie bei den monotonen Abbildungen in der Mathematikreihe.

Warum die Antwort immer ungenau ist

Der Übersetzer weiß nicht, welche Wege wirklich durchlaufen werden. Er muss also alle annehmen, die im Graphen existieren.

```
if (x > 0) a = 1;
if (x > 0) b = a;      ist a gesetzt?
```

Ein Mensch sieht, dass beide Bedingungen dasselbe prüfen und `a` deshalb gesetzt ist, wenn es gebraucht wird. Der Graph enthält trotzdem einen Weg, auf dem der erste Zweig übersprungen und der zweite genommen wird, und der Übersetzer muss ihn berücksichtigen.

Solche Wege heißen **unausführbar**. Sie festzustellen ist im Allgemeinen unentscheidbar (Band 2, Kapitel 7), und deshalb bleibt jede Datenflussanalyse eine Näherung nach der sicheren Seite.

	sichere Seite	**Folge**
Ist der Wert lebendig?	**ja** annehmen	Register wird gehalten
Zeigen zwei Zeiger auf dasselbe?	**ja** annehmen	keine Umordnung
Kann die Funktion Nebenwirkungen haben?	**ja** annehmen	kein Einsetzen

Die zweite Zeile ist der größte Hemmschuh bei C und C++. Solange der Übersetzer nicht ausschließen kann, dass zwei Zeiger auf dieselbe Stelle zeigen, darf er Zugriffe nicht vertauschen und Werte nicht in Registern halten.

Das Schlüsselwort `restrict` ist das Versprechen des Programmierers, dass es nicht so ist. Es kann eine Schleife um Faktor zwei bis drei beschleunigen und ist zugleich eine Zusage, die niemand prüft.

Undefiniertes Verhalten als Optimierungsgrundlage

Die C-Norm erklärt bestimmte Situationen für undefiniert. Der Übersetzer darf dann annehmen, dass sie nicht eintreten.

Überlauf bei `int`	Der Übersetzer darf annehmen, dass `x + 1 > x` gilt
Nullzeiger	Wer einen Zeiger benutzt, hat damit erklärt, dass er nicht null ist
Feldgrenzen	Zugriffe liegen innerhalb

Das ist kein Versehen, sondern Absicht. Ohne diese Annahmen müsste der Übersetzer bei jeder Addition mit einem Überlauf rechnen und könnte kaum etwas umformen.

```
for (int i = 0; i <= n; i++) ...
```

Weil ein Überlauf undefiniert ist, darf der Übersetzer annehmen, dass `i` irgendwann größer als `n` wird, und die Schleife entrollen. Wäre der Überlauf definiert, könnte `i` umlaufen und die Schleife nie enden.

Wenn die Annahme nicht stimmt

Der berüchtigtste Fall stammt aus dem Linux-Kern.

```
struct sock *sk = tun->sk;      Zugriff
if (!tun) return -1;            Prüfung, zu spät
```

1. Zeile 1 greift auf `tun` zu.
2. Also darf der Übersetzer annehmen, `tun` sei nicht null.
3. Dann ist die Bedingung in Zeile 2 immer falsch.
4. Also wird die Prüfung **entfernt**.

Aus einer Prüfung, die den Fehler abgefangen hätte, wurde nichts. Auf Systemen, auf denen sich Speicher an Adresse null abbilden lässt, entstand daraus 2009 eine ausnutzbare Lücke.

Jeder Schritt für sich ist korrekt. Der Übersetzer hat sich an die Norm gehalten; der Code hat gegen sie verstoßen, indem er den Zeiger vor der Prüfung benutzte.

Werkzeuge	UBSan meldet undefiniertes Verhalten zur Laufzeit
Schalter	Der Linux-Kern übersetzt mit `-fno-strict-overflow` und `-fno-delete-null-pointer-checks`
Sprachwahl	Rust definiert den Überlauf: Abbruch beim Testen, Umlauf im Betrieb

Der Streit darüber, wie weit ein Übersetzer solche Annahmen treiben darf, ist nicht entschieden. Die Norm erlaubt es; viele halten es für unklug, weil es Programme bricht, die jahrzehntelang funktioniert haben.

Aufgabe 9.1.

(a) Optimieren Sie eine Schleife Schritt für Schritt und messen Sie den Gewinn.
(b) Führen Sie eine Datenflussanalyse vollständig durch.
(c) Untersuchen Sie, wann Einsetzen schadet.

Lösung Schritt für Schritt.

(a) Eine Schleife optimieren.
Der Ausgangscode.

```
for (int i = 0; i < n; i++)
  y[i] = x[i] * 2.0 + laenge(v) *
z[i];
```

Schritt 1 – Invariantes herausziehen.
`laenge(v)` hängt nicht von `i` ab und wird n-mal berechnet.

```
double L = laenge(v);
for (int i = 0; i < n; i++)
  y[i] = x[i] * 2.0 + L * z[i];
```

Das darf der Übersetzer nur, wenn er weiß, dass `laenge` keine Nebenwirkungen hat und `v` sich nicht ändert. Bei einer Funktion aus einer anderen Übersetzungseinheit weiß er das nicht, und dann bleibt der Aufruf in der Schleife.

Fall	Herausziehen?	
`static` in derselben Datei	ja	Übersetzer sieht den Rumpf
andere Datei	nein	es sei denn mit LTO
mit dem Zusatz `pure`	ja	Zusage des Programmierers

Schritt 2 – Entrollen.

```
for (i = 0; i + 4 <= n; i += 4) {
  y[i] = x[i] * 2.0 + L * z[i];
  y[i+1] = x[i+1] * 2.0 + L * z[i+1];
  y[i+2] = x[i+2] * 2.0 + L * z[i+2];
  y[i+3] = x[i+3] * 2.0 + L * z[i+3];
}
for (; i < n; i++) ...     Rest
```

Anteil	**vorher je Element**	**nachher**
Rechnen	3 Befehle	3
Zähler und Sprung	2 Befehle	0,5
Summe	5	**3,5**

Schritt 3 – Vektorisieren.
Vier Gleitkommazahlen passen in ein 256-Bit-Register (Band 4, Kapitel 10).

Fassung	**Befehle je 4 Elemente**	**Verhältnis**
ursprünglich	$20 + 4$ Aufrufe	1,0
invariantes heraus	20	≈ 4
entrollt	14	5,7
vektorisiert	**5**	**16**

Faktor sechzehn, davon der größte Einzelbeitrag aus dem Herausziehen des Funktionsaufrufs. Die Vektorisierung, über die am meisten geredet wird, bringt hier nur den letzten Faktor knapp drei.

Wann die Vektorisierung scheitert.

Überlappung	Zeigen `x` und `y` auf denselben Speicher, ist die Reihenfolge wesentlich
Abhängigkeit	`y[i] = y[i-1] + 1` lässt sich nicht parallelisieren
Verzweigungen	Ein `if` im Rumpf erschwert es erheblich
Gleitkomma	Umordnen ändert das Ergebnis (Band 0, Kapitel 9)

Die letzte Zeile ist der Grund, warum Summen über Gleitkommazahlen ohne `-ffast-math` nicht vektorisiert werden. Die Addition ist nicht assoziativ, und der Übersetzer darf das Ergebnis nicht ändern, auch nicht in der letzten Stelle.

(b) Eine Datenflussanalyse.
Die Frage. Welche Variablen sind an jeder Stelle noch lebendig, werden also später gelesen?
Der Code.

```
1   a = 1
2   b = 2
3   c = a + b
4   a = 5
5   d = a + c
6   return d
```

Rückwärts durchgehen.

Zeile	lebendig danach	tötet	erzeugt
6	∅	—	`d`
5	{`d`}	`d`	`a`, `c`
4	{`a`, `c`}	`a`	—
3	{`c`}	`c`	`a`, `b`
2	{`a`, `b`}	`b`	—
1	{`a`}	`a`	—

Das Ergebnis vor jeder Zeile.

vor Zeile	lebendig	
1	∅	
2	{a}	
3	{a, b}	**Höchstzahl: 2**
4	{c}	
5	{a, c}	
6	{d}	

Was man damit anfängt.

Register sparen	Nie sind mehr als zwei Werte gleichzeitig lebendig; zwei Register genügen
Toten Code finden	Eine Zuweisung an eine Variable, die danach nicht lebendig ist, kann weg
Wiederverwenden	Nach Zeile 3 ist `a` tot; sein Register kann für `c` dienen

Hier ist `a` in Zeile 1 nach Zeile 3 tot, wird aber in Zeile 4 neu gesetzt. Die Zuweisung in Zeile 1 ist also nötig, die in Zeile 4 ebenfalls. Wäre Zeile 5 nicht da, wäre Zeile 4 toter Code.

(c) Wann Einsetzen schadet.
Der Gewinn.

Aufruf gespart	Etwa 5–20 Takte
Ermöglicht anderes	Konstanten wandern hinein, tote Zweige verschwinden
Bessere Register	Kein Retten und Wiederherstellen an der Grenze

Der Preis.

Codegröße	Jede Aufrufstelle bekommt eine eigene Kopie
Befehlscache	Größerer Code passt schlechter hinein
Registerdruck	Mehr gleichzeitig lebendige Werte, mehr Auslagerung
Übersetzungszeit	Wächst überproportional

Die Rechnung.

Funktionsgröße	Aufruf spart	Code wächst	lohnt?
3 Befehle	10 Takte	+2 je Stelle	immer
20 Befehle	10 Takte	+19	meist
200 Befehle	10 Takte	+199	nur wenn heiß
2000 Befehle	10 Takte	+1999	nie

Übersetzer benutzen deshalb eine Kostenschätzung mit einer Schranke. Sie bewerten die Größe der Funktion, die Zahl der Aufrufstellen und ob es sich um eine Schleife handelt, und setzen ein, solange die Punktzahl unter der Schranke bleibt.

Der Fall, in dem alles langsamer wird

Ein Programm mit einer eng geschriebenen Kernschleife und vielen kleinen Hilfsfunktionen kann durch aggressives Einsetzen langsamer werden.

Codegröße Schleife	L1-Befehlscache	Wirkung
20 KB	32 KB	passt
40 KB	32 KB	ständige Fehlgriffe

Der Übersetzer hat jede einzelne Entscheidung richtig getroffen und die Summe falsch. Er kennt die Größe des Befehlscaches nicht und weiß nicht, welche Schleife wirklich heiß ist.

Die Antwort darauf heißt **Optimierung nach Profil**: Man lässt das Programm einmal mit Messung laufen und übersetzt dann noch einmal, mit Kenntnis der tatsächlichen Häufigkeiten.

Verfahren	Gewinn	Aufwand
`-O2`	Grundlage	einmal übersetzen
`-O3`	0–5 %	
mit Profil	**10–20** %	zweimal, plus Messlauf

Die letzte Zeile bringt mehr als alle Feineinstellungen zusammen, und sie wird selten benutzt, weil der Ablauf umständlich ist und man einen repräsentativen Messlauf braucht. Genau diesen Nachteil hat die Übersetzung zur Laufzeit nicht (Kapitel 13).

Muchnick[20] ist die gründlichste Darstellung der Optimierungen; Aho und Mitautoren[1] behandeln die Datenflussanalyse; Ferrante und Mitautoren[9] führten den Abhängigkeitsgraphen ein.

10 Codeerzeugung

Drei Aufgaben

Befehlsauswahl	Welche Maschinenbefehle setzen die Zwischenform um
Registerzuteilung	Welcher Wert liegt in welchem Register
Anordnung	In welcher Reihenfolge stehen die Befehle

Alle drei hängen voneinander ab, und alle drei sind in ihrer allgemeinen Form NP-schwer (Band 3). Man löst sie deshalb nacheinander mit guten Näherungsverfahren und nimmt in Kauf, dass das Ergebnis nicht optimal ist.

Die Befehlsauswahl.
Ein Zielrechner bietet oft mehrere Wege für dasselbe.

Zwischenform	Möglichkeit	Takte
`a = b * 2`	`imul eax, 2`	3
	`shl eax, 1`	1
	`add eax, eax`	**1, besser für die Ausführung**
	`lea eax,[ebx+ebx]`	1, ohne Flaggen zu ändern

Der Befehl `lea` war ursprünglich für Adressrechnungen gedacht und wird von Übersetzern gern als allgemeiner Rechenbefehl benutzt, weil er drei Operanden hat und die Statusflaggen unangetastet lässt.

Das Verfahren: Kacheln legen.
Man fasst die Zwischenform als Baum auf und deckt ihn mit Mustern ab, denen je ein Befehl entspricht.

1. Jeder Maschinenbefehl entspricht einem Baummuster mit Kosten.
2. Gesucht ist die Überdeckung des Baums mit den geringsten Gesamtkosten.
3. Das lässt sich mit dynamischer Programmierung in linearer Zeit lösen.

Für Bäume ist das Problem also leicht. Schwer wird es erst, wenn ein Zwischenwert mehrfach benutzt wird, denn dann ist es kein Baum mehr, sondern ein Graph.

Registerzuteilung durch Graphfärbung

GREGORY CHAITIN beschrieb 1981 ein Verfahren, das bis heute die Grundlage bildet.

1. Für jeden Wert einen Knoten anlegen.
2. Zwei Knoten verbinden, wenn ihre Werte gleichzeitig lebendig sind.
3. Den Graphen mit k Farben färben, wobei k die Zahl der Register ist.
4. Jede Farbe ist ein Register.

Die Übersetzung ist genau richtig: Zwei Werte, die gleichzeitig lebendig sind, dürfen nicht dasselbe Register belegen, und zwei benachbarte Knoten dürfen nicht dieselbe Farbe haben.

Das Problem	Graphfärbung ist NP-vollständig (Band 3, Kapitel 8)
Der Ausweg	Ein gieriges Verfahren, das fast immer gut genug ist

Chaitins Heuristik.

1. Solange es einen Knoten mit weniger als k Nachbarn gibt: entfernen und auf einen Stapel legen.
2. Bleiben nur Knoten mit $\geq k$ Nachbarn übrig, einen davon zum **Auslagern** wählen.
3. Wenn der Graph leer ist, den Stapel abarbeiten und jedem Knoten eine freie Farbe geben.

Schritt 1 ist der Einfall. Ein Knoten mit weniger als k Nachbarn kann immer gefärbt werden, gleichgültig was mit den anderen geschieht, denn eine Farbe bleibt garantiert übrig. Man darf ihn also getrost beiseitelegen.

Was Auslagern kostet

Wenn die Register nicht reichen, muss ein Wert in den Speicher.

Zugriff	Takte
Register	0–1
Stapel, im L1	4–5
Stapel, im L2	≈ 14

Die Wahl des Auszulagernden ist deshalb wesentlich. Man wählt den Wert mit den meisten Nachbarn und den wenigsten Zugriffen, und ein Wert innerhalb einer Schleife wird zehnfach gewichtet.

Architektur	Register	Auslagerungsdruck
x86, 32 Bit	8	sehr hoch
x86-64	16	mittel
ARM64	31	gering
RISC-V	32	gering

Die erste Zeile war einer der Gründe, warum der Übergang zu 64 Bit auf x86 mehr Geschwindigkeit brachte, als die breiteren Zahlen erklären. Es kamen acht Register hinzu, und das reduzierte die Auslagerung erheblich.

Bei acht Registern sind zwei bereits fest belegt, für Stapel- und Rahmenzeiger. Es blieben also sechs für alles Übrige, und ein mäßig komplexer Ausdruck braucht schon vier.

Aufgabe 10.1.

(a) Führen Sie eine Registerzuteilung vollständig durch.
(b) Untersuchen Sie die Befehlsanordnung.
(c) Erzeugen Sie Code für einen vollständigen Ausdruck.

Lösung Schritt für Schritt.

(a) Registerzuteilung durch Färbung.
Der Code.

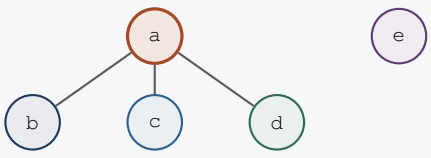
```
1       a = 1
2       b = 2
3       c = a + b
4       d = c * 2
5       e = a + d
6       return e
```

Schritt 1 – Lebensspannen bestimmen.

Wert	lebt ab	bis	Länge
`a`	1	5	4
`b`	2	3	1
`c`	3	4	1
`d`	4	5	1
`e`	5	6	1

Schritt 2 – Den Konfliktgraphen aufstellen.

Paar	gleichzeitig?	Kante
`a`, `b`	ja, Zeile 2–3	✓
`a`, `c`	ja, Zeile 3–4	✓
`a`, `d`	ja, Zeile 4–5	✓
`b`, `c`	nein	—
`c`, `d`	nein	—
`d`, `e`	nein	—



Schritt 3 – Färben mit $k = 2$.

Schritt	Knoten	Nachbarn	Aktion
1	`e`	0	auf den Stapel
2	`b`	1	auf den Stapel
3	`c`	1	auf den Stapel
4	`d`	1	auf den Stapel
5	`a`	0	auf den Stapel

Schritt 4 – Rückwärts färben.

Knoten	Nachbarfarben	Wahl	Register
`a`	—	1	`r1`
`d`	{1}	2	`r2`
`c`	{1}	2	`r2`
`b`	{1}	2	`r2`
`e`	—	1	`r1`

Fünf Werte in zwei Registern, ohne einen einzigen Speicherzugriff. Der Grund ist, dass `b`, `c` und `d` nie gleichzeitig leben und sich deshalb ein Register teilen können.

Der erzeugte Code.

```
mov r1, 1     a
mov r2, 2     b
add r2, r1    c = a + b
add r2, r2    d = c * 2
add r1, r2    e = a + d
ret
```

(b) Befehlsanordnung.

Warum die Reihenfolge zählt.
Ein Befehl, der auf das Ergebnis des vorigen wartet, kann nicht gleichzeitig ausgeführt werden (Band 4, Kapitel 6).

schlecht angeordnet	gut angeordnet
`load r1, [a]`	`load r1, [a]`
`add r1, 1` *wartet*	`load r2, [b]` *unabhängig*
`load r2, [b]`	`add r1, 1`
`add r2, 1` *wartet*	`add r2, 1`

Fassung	Takte	bei 4 Takten Ladezeit
schlecht	10	zweimal warten
gut	**6**	Ladevorgänge überlappen

Auf einem Prozessor mit Umordnung (Band 4, Kapitel 6) fällt der Unterschied geringer aus, weil die Hardware das selbst richtet. Er verschwindet aber nicht, denn das Fenster der Umordnung ist begrenzt.

Der Zielkonflikt mit der Registerzuteilung.

Gute Anordnung	Will viele unabhängige Befehle nebeneinander, also viele gleichzeitig lebendige Werte
Gute Zuteilung	Will wenige gleichzeitig lebendige Werte, um Auslagerung zu vermeiden

Beide Ziele widersprechen sich unmittelbar. Übersetzer ordnen deshalb meist zweimal an: einmal vor der Zuteilung, um Freiheit zu haben, und einmal danach, um die eingefügten Speicherzugriffe zu berücksichtigen.

(c) Code für einen vollständigen Ausdruck.
Der Quelltext.

```
r = (a + b) * (c - d) / 2;
```

Schritt 1 – Zwischenform.

```
t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = t3 / 2
r = t4
```

Schritt 2 – Die Division ersetzen.
Eine Division kostet 20 bis 40 Takte, eine Verschiebung einen.

Fall	Ersetzung	Bedingung
`x / 2` vorzeichenlos	`x » 1`	immer
`x / 2` vorzeichenbehaftet	**drei Befehle**	Rundung zur Null
`x / 3`	Multiplikation mit Kehrwert	ganzzahlig gerechnet

Die zweite Zeile ist eine hübsche Feinheit. Bei negativen Zahlen rundet die Verschiebung nach unten, die Division aber zur Null hin. Aus $-7/2$ wird -3, aus $-7 >> 1$ wird -4. Der Übersetzer fügt deshalb eine Korrektur ein.

```
mov r5, r3
shr r5, 31      Vorzeichenbit isolieren
add r3, r5      bei negativen Zahlen um 1 erhöhen
sar r3, 1
```

Schritt 3 – Der fertige Code.

```
mov eax, [a]
add eax, [b]     t1
mov ecx, [c]
sub ecx, [d]     t2
imul eax, ecx    t3
mov edx, eax
shr edx, 31
add eax, edx
sar eax, 1       t4
mov [r], eax
```

Die Bilanz.

Fassung	Takte	
mit `idiv`	≈ 30	Division beherrscht alles
mit Verschiebung	$\approx$ **9**	

Faktor drei für eine Umformung, die aus einem Befehl vier macht. Das ist ein gutes Beispiel dafür, dass die Zahl der Befehle wenig über die Laufzeit sagt.

Wer die Divisionen wirklich braucht, kann sie oft ganz vermeiden: Modulo mit einer Zweierpotenz ist ein `AND`, und eine Schleife mit Division im Rumpf lässt sich oft so umschreiben, dass die Division vor die Schleife wandert.

Chaitin[5] führte die Registerzuteilung durch Färbung ein; Appel[2] stellt sie besonders klar dar; Muchnick[20] behandelt die Befehlsanordnung ausführlich.

11 Binden und Laden

Warum es einen Binder gibt

Ein Übersetzer sieht immer nur eine Datei. Er kann deshalb nicht wissen, wo eine Funktion aus einer anderen Datei liegt, und lässt eine Lücke.

Symbole sammeln	Welche Namen definiert jede Datei, welche braucht sie
Auflösen	Jedem gebrauchten Namen eine Definition zuordnen
Zusammenlegen	Gleichartige Abschnitte aller Dateien aneinanderreihen
Verschieben	Die Lücken mit den nun bekannten Adressen füllen

Abschnitt	Inhalt	Rechte
`.text`	Code	lesen, ausführen
`.rodata`	Konstanten	nur lesen
`.data`	vorbelegte Daten	lesen, schreiben
`.bss`	Nullen	lesen, schreiben

Der Abschnitt `.bss` nimmt in der Datei keinen Platz ein. Es steht nur seine Größe dort, und das Betriebssystem legt beim Laden entsprechend viele Nullseiten an. Ein Feld von einem Megabyte kostet damit null Byte auf der Platte.

Was bei der Verschiebung geschieht.

Vor dem Binden	`call 00000000`	Lücke, mit Eintrag
Eintrag sagt	„hier gehört `f` hin“	Art: relativ, 32 Bit
Nach dem Binden	`call 0x1a2f`	Abstand eingesetzt

Statisch oder dynamisch

	Statisch	**Dynamisch**
Bibliothek liegt	**im Programm**	getrennt
Programmgröße	groß	klein
Speicher bei 100 Programmen	100 Kopien	eine
Aktualisierung	neu binden	Datei ersetzen
Startzeit	sofort	Auflösung nötig
Aufruf	1 Takt	2–3 Takte
Abhängigkeiten	keine	müssen passen

Die vierte Zeile ist das stärkste Argument für dynamisches Binden. Wenn eine Sicherheitslücke in einer Bibliothek behoben wird, genügt ein Austausch der Datei, und alle Programme sind repariert. Bei statischer Bindung muss jedes einzeln neu gebaut werden.

Wie ein Aufruf aufgelöst wird.

Sprungtabelle	Jeder Aufruf geht an einen festen Eintrag im eigenen Programm
Beim ersten Mal	Der Eintrag springt zum Auflöser, der die echte Adresse sucht und einträgt
Danach	Der Eintrag springt unmittelbar zum Ziel

*Man nennt das **verzögerte Bindung**. Nur die tatsächlich benutzten Funktionen werden aufgelöst, und das verkürzt den Start eines großen Programms erheblich.*

Programm	Symbole	davon benutzt
kleines Werkzeug	≈ 200	≈ 40
Browser	≈ 50 000	≈ 8000

Wenn die Abhängigkeiten nicht passen

Ein Programm verlangt eine Bibliotheksfassung, eine andere Bibliothek eine andere. Beide lassen sich nicht gleichzeitig laden.

Fassungsnummern	Die Datei heißt `libfoo.so.2`, und Programme verlangen die 2
Symbolfassungen	Eine Datei kann mehrere Fassungen desselben Symbols enthalten
Alles mitliefern	Container, Flatpak, statisches Binden

Die dritte Zeile ist die Antwort, die sich seit etwa 2015 durchgesetzt hat, und sie gibt genau den Vorteil auf, um dessentwillen man dynamisch bindet: Eine behobene Lücke muss wieder überall einzeln nachgezogen werden.

Go ist hier besonders deutlich. Programme werden statisch gebunden und bringen alles mit, was sie brauchen. Das Ergebnis ist eine Datei von zehn Megabyte, die man überall hinkopieren kann und die keine Abhängigkeiten hat.

Ob das ein Fortschritt ist, hängt davon ab, was man vergleicht: mit dem Betriebssystem als gepflegter Grundlage ist es ein Rückschritt, mit dem Zustand „es läuft auf meinem Rechner“ ein Fortschritt.

Aufgabe 11.1.

(a) Verfolgen Sie das Binden zweier Dateien vollständig.
(b) Untersuchen Sie lageunabhängigen Code.
(c) Analysieren Sie die Kosten der Bindungsarten.

Lösung Schritt für Schritt.

(a) Zwei Dateien binden.
Die Quelldateien.

haupt.c	**hilf.c**
`extern int f(int);`	`int zaehler = 0;`
`int main() {`	`int f(int x) {`
`  return f(5); }`	`  zaehler++;`
	`  return x * 2; }`

Schritt 1 – Die Symboltabellen der Objektdateien.

Datei	Symbol	Art	Abschnitt
`haupt.o`	`main`	definiert	`.text` +0
`haupt.o`	`f`	**gebraucht**	—
`hilf.o`	`f`	definiert	`.text` +0
`hilf.o`	`zaehler`	definiert	`.bss` +0

Schritt 2 – Abschnitte zusammenlegen.

Abschnitt	aus	Größe	Adresse
`.text`	`haupt.o`	16 B	`0x1000`
`.text`	`hilf.o`	24 B	`0x1010`
`.bss`	`hilf.o`	4 B	`0x4000`

Schritt 3 – Adressen zuordnen.

Symbol	Adresse
`main`	`0x1000`
`f`	`0x1010`
`zaehler`	`0x4000`

Schritt 4 – Die Verschiebung ausführen.
In `haupt.o` stand an Abstand 6 ein Eintrag: „hier gehört `f` hin, relativ, 32 Bit“.

$$\text{Wert} = \text{Ziel} - (\text{Ort} + 4) = \texttt{0x1010} - (\texttt{0x1006} + 4) = \texttt{0x0A}.$$

vorher	`e8 00 00 00 00`	`call ...`
nachher	`e8 0a 00 00 00`	`call 0x1010`

Der Abstand wird vom Ende *des Befehls aus gerechnet, weil der Prozessor den Befehlszähler bereits weitergesetzt hat, wenn er den Sprung ausführt. Das ist eine häufige Fehlerquelle beim Schreiben eines Binders.*

Was schiefgehen kann.

Undefiniertes Symbol	Niemand definiert `f`; meist ein vergessenes `-l`
Doppelte Definition	Zwei Dateien definieren `f`
Abstand zu groß	Bei ± 2 GB Reichweite in sehr großen Programmen

Die dritte Zeile klingt entlegen und ist bei großen Programmen real. Man löst sie mit Sprungpolstern, die der Binder selbst einfügt, oder mit einem anderen Codemodell, das absolute 64-Bit-Adressen benutzt.

(b) Lageunabhängiger Code.
Das Problem. Eine gemeinsam genutzte Bibliothek liegt in jedem Prozess an einer anderen Adresse, weil die Adressraumverwürfelung sie verschiebt (Band 5, Kapitel 5).

Absolute Adressen	Müssten beim Laden alle angepasst werden; dann ist der Code nicht mehr teilbar
Relative Adressen	Funktionieren unabhängig von der Lage, solange alles zusammen verschoben wird

Der Zugriff auf globale Daten.

ohne PIC	mit PIC
`mov eax, [0x4000]`	`mov rax, [rip + 0x2ff8]`
1 Befehl, absolut	1 Befehl, relativ zum Zähler

Auf x86-64 kostet das gar nichts, weil der Prozessor seit 2003 die Adressierung relativ zum Befehlszähler beherrscht. Auf x86 mit 32 Bit gab es sie nicht, und man musste die eigene Adresse mit einem Trick ermitteln und ein Register dauerhaft dafür opfern.

Architektur	Kosten von PIC	
x86, 32 Bit	5–10 %	ein Register weniger
x86-64	$< 1\,\%$	`rip`-relativ
ARM64	$< 1\,\%$	

Deshalb ist lageunabhängiger Code heute überall Standard, während er in den 1990er Jahren als spürbarer Nachteil galt. Die Sicherheitseigenschaft war schon damals erwünscht, und erst die Hardware machte sie billig.

(c) Die Kosten der Bindungsarten.
Der Fall. Ein Programm von 2 MB benutzt eine Bibliothek von 8 MB. Auf dem Rechner laufen 50 solche Programme.

	statisch	dynamisch
Datei je Programm	10 MB	2 MB
Platte, 50 Programme	500 MB	**108** MB
Speicher, 50 laufend	500 MB	**108** MB
Startzeit	5 ms	12 ms

Die dritte Zeile setzt voraus, dass die Bibliothek wirklich nur einmal im Speicher liegt. Das gilt, weil ihre Codeseiten nur gelesen werden und das Betriebssystem sie deshalb in alle Adressräume einblenden kann (Band 5, Kapitel 5).

Warum der Vorteil kleiner ist, als er aussieht.

Nur Benutztes zählt	Ein statischer Binder nimmt nur die Teile mit, die gebraucht werden; oft ein Bruchteil
Toter Code fliegt	Mit `-gc-sections` entfernt der Binder ungenutzte Abschnitte
Nur wenige teilen	Bei einer Bibliothek, die nur ein Programm benutzt, gibt es nichts zu teilen

Fall	**statisch**	**Bemerkung**
naiv	10 MB	alles mitgenommen
nur Benutztes	3,2 MB	
plus Abschnitts-bereinigung	**2,4** MB	

Optimierung über Dateigrenzen hinweg.
Ein Binder, der die Zwischenform statt fertigen Maschinencode bekommt, kann noch optimieren.

Verfahren	**Gewinn**	**Kosten**
gewöhnlich	Grundlage	
LTO	5–15 %	Bindezeit vervielfacht
LTO, dünn	5–12 %	parallelisierbar

Der Gewinn kommt fast vollständig daher, dass jetzt Funktionen aus anderen Dateien eingesetzt werden können. Genau das war in Kapitel 9 die wichtigste Einzeloptimierung, und ohne LTO endet sie an der Dateigrenze.

Die zweite Zeile ist der Grund, warum LTO nicht überall eingeschaltet ist: Der Binder muss auf einmal das ganze Programm optimieren, und das ist ein einziger, schlecht parallelisierbarer Schritt am Ende des Bauvorgangs.

Levine[17] ist die maßgebliche Darstellung des Bindens und Ladens; Aho und Mitautoren[1] behandeln die Grundlagen; Kernighan und Ritchie[14] zeigen das Modell der getrennten Übersetzung.

Part IV

Andere Wege

12 Interpreter und virtuelle Maschinen

Wie ein Interpreter arbeitet

Statt Maschinencode zu erzeugen, führt ein Interpreter das Programm unmittelbar aus.

Art	arbeitet auf	Verlangsamung
Text	dem Quelltext	1000-fach
Syntaxbaum	dem Baum	50–200-fach
Bytecode	einer Befehlsfolge	**3–30**-fach
Mit Sprungfäden	dasselbe, anders verkettet	2–20-fach

Die erste Zeile ist der Grund, warum kein ernsthaftes System so arbeitet. Wer bei jedem Schleifendurchlauf den Quelltext neu zerlegt, zahlt die Zerlegung tausendfach.

Die Schleife eines Bytecode-Interpreters.

```
while (true) {
  Befehl b = code[zaehler++];
  switch (b.art) {
    case LADE: stapel[sp++] = b.wert;
break;
    case ADD: sp-; stapel[sp-1] +=
stapel[sp]; break;
    case SPRINGE: zaehler = b.ziel;
break;
    ...
  } }
```

Anteil je Befehl	**Takte**	
Befehl holen	1	
Verteilersprung	≈ 15	**Fehlvorhersage**
Arbeit	1–3	
Summe	≈ 18	für einen Takt Nutzarbeit

Die mittlere Zeile ist der eigentliche Aufwand, und sie ist die Wurzel fast aller Beschleunigungsversuche. Der Sprung in der `switch`-Anweisung geht bei jedem Durchlauf woandershin, und der Prozessor kann ihn nicht vorhersagen (Band 4, Kapitel 7).

Der Verteilersprung

Es gibt einen Kunstgriff, der ohne große Umbauten viel bringt: Statt eines gemeinsamen Verteilers bekommt jeder Befehl seinen eigenen.

```
#define WEITER goto
*tabelle[code[zaehler++].art]
```

```
L_LADE: stapel[sp++] = ...; WEITER;

L_ADD: sp-; stapel[sp-1] += ...;
WEITER;
```

Jetzt gibt es einen Sprung je Befehlsart statt eines gemeinsamen. Der Prozessor kann für jeden davon eine eigene Vorhersage lernen, und weil bestimmte Befehlspaare häufig aufeinanderfolgen, trifft sie oft zu.

Verfahren	Trefferquote	Gewinn
gemeinsamer Verteiler	≈ 30 %	Grundlage
Sprungfäden	≈ 70 %	20–40 % schneller

Der Preis ist, dass man eine Spracherweiterung braucht: Zeiger auf Marken gibt es in Norm-C nicht, wohl aber in GCC und Clang. Der CPython-Interpreter benutzt das seit Jahren.

Ein zweiter Kunstgriff geht in dieselbe Richtung: Man fasst häufige Befehlspaare zu einem einzigen zusammen, etwa „lade lokale Variable und addiere“. Das halbiert die Zahl der Verteilersprünge auf einen Schlag.

Stapel oder Register

Eine virtuelle Maschine kann ihre Zwischenwerte auf einem Stapel halten oder in nummerierten Registern.

	Stapel	Register
Beispiele	Java, .NET, WebAssembly	Lua, Dalvik
Befehl	kurz, ohne Operanden	länger, mit Nummern
Anzahl Befehle	mehr	weniger
Codegröße	kleiner	größer
Erzeugung	trivial	Zuteilung nötig

Dasselbe in beiden Formen.

Stapel	Register
`lade a`	
`lade b`	
`addiere`	`add r3, r1, r2`
`speichere c`	
4 Befehle, 8 Byte	1 Befehl, 4 Byte
4 Verteilersprünge	**1 Verteilersprung**

Weil der Verteilersprung der teuerste Anteil ist, gewinnt die Registerform. Untersuchungen an Lua und Dalvik ergaben je nach Last zwanzig bis vierzig Prozent.

Java und WebAssembly haben sich trotzdem für den Stapel entschieden, weil dort andere Ziele im Vordergrund standen: kompakter Code für die Übertragung und eine Form, die sich schnell auf Gültigkeit prüfen lässt.

xBase: eine Sprache, die aus einem Dateiformat wuchs

Anfang der 1980er Jahre war dBASE das verbreitetste Datenbankprogramm für Kleinrechner. Es brachte eine eigene Sprache mit, in der sich Auswertungen und ganze Anwendungen schreiben ließen.

Für damalige Verhältnisse war das sehr fortgeschritten. Man beschrieb in wenigen Zeilen, was man von den Daten wollte, und musste sich um Dateiverwaltung und Indexpflege nicht kümmern.

Der wunde Punkt war die Geschwindigkeit. dBASE interpretierte die Programme, und das war für größere Anwendungen zu langsam.
Im Winter 1985 gründeten ehemalige Mitarbeiter von Ashton-Tate die Firma Nantucket und brachten **Clipper** heraus: einen Übersetzer für dieselbe Sprache, der ausführbare DOS-Programme erzeugte.

	dBASE	**Clipper**
Ausführung	interpretiert	**übersetzt**
Geschwindigkeit	mäßig	deutlich besser
Weitergabe	Kunde braucht dBASE	**eigenständige Datei**

Die dritte Zeile war wirtschaftlich mindestens so wichtig wie die zweite. Wer eine Anwendung verkaufte, musste dem Kunden nicht mehr eine Lizenz für die Entwicklungsumgebung mitverkaufen.

Aus diesem Ursprung entstand eine ganze Sprachfamilie, die man nach der Dateiendung der Datenbanken **xBase** nennt. Zu ihr gehören dBASE, FoxPro, Clipper und deren Nachfolger.
Für dieses Buch ist die Familie aus zwei Gründen interessant.
Erstens: Sie zeigt, dass „interpretiert" und „übersetzt" keine Eigenschaften einer Sprache sind. Dieselbe Sprache lief in dBASE interpretiert und in Clipper übersetzt, mit demselben Ergebnis und sehr verschiedener Geschwindigkeit. Genau das war die Behauptung aus Kapitel 1.
Zweitens: FlagShip. Das Produkt von multisoft Datentechnik in Landshut übersetzte ab 1992 xBase-Quelltext nach C und ließ dann den C-Übersetzer des Zielsystems arbeiten (Kapitel 8).

1992	Erste Fassung; Portierung von DOS-Anwendungen auf Unix-Systeme wie AIX, Solaris, HP-UX und SINIX
1995	Linux kommt hinzu
2002	Visual FlagShip mit grafischer Oberfläche

> *Der Weg über C war hier genau die richtige Entscheidung. Das Ziel war nicht ein weiterer Übersetzer, sondern die Rettung vorhandener Anwendungen auf neue Systeme, und dafür braucht man vor allem viele Zielplattformen bei überschaubarem Aufwand.*

Die Familie lebt weiter, inzwischen überwiegend in freien Umsetzungen wie Harbour, die sich wiederum als xBase-nach-C-Übersetzer verstehen.

Das ist eine bemerkenswerte Langlebigkeit für eine Sprache, die niemand entworfen hat. Sie ist um ein Dateiformat herum gewachsen, und ihr Erfolg beruhte darauf, dass sie für kaufmännische Datenverwaltung genau die richtigen Sprachmittel anbot (Kapitel 7).

Aufgabe 12.1.

(a) Bauen Sie einen kleinen Interpreter und führen Sie ihn aus.
(b) Vergleichen Sie die Ausführungsarten quantitativ.
(c) Untersuchen Sie, wie eine virtuelle Maschine Code prüft.

Lösung Schritt für Schritt.

(a) Ein kleiner Interpreter.
Der Befehlssatz.

Befehl	Wirkung	Stapel
`PUSH n`	Zahl ablegen	$+1$
`LOAD i`	Variable ablegen	$+1$
`STORE i`	oben nach Variable	-1
`ADD`	zwei zu einem	-1
`JLT m`	springen wenn kleiner	-2
`JMP m`	springen	0

Das Programm. Summe von 1 bis 3.

0	`PUSH 0`	
1	`STORE s`	`s = 0`
2	`PUSH 1`	
3	`STORE i`	`i = 1`
4	`LOAD s`	
5	`LOAD i`	
6	`ADD`	
7	`STORE s`	`s = s + i`
8	`LOAD i`	
9	`PUSH 1`	
10	`ADD`	
11	`STORE i`	`i = i + 1`
12	`LOAD i`	
13	`PUSH 4`	
14	`JLT 4`	zurück, wenn `i < 4`

Der Lauf, erste Runde.

PC	Befehl	Stapel	Variablen
0	`PUSH 0`	$[0]$	
1	`STORE s`	$[\,]$	`s`$= 0$
2	`PUSH 1`	$[1]$	
3	`STORE i`	$[\,]$	`s`$= 0$, `i`$= 1$
4	`LOAD s`	$[0]$	
5	`LOAD i`	$[0, 1]$	
6	`ADD`	$[1]$	
7	`STORE s`	$[\,]$	`s`$= 1$
8–11		$[\,]$	`i`$= 2$
12–14	`JLT 4`	$[\,]$	$2 < 4$, springt

Die weiteren Runden.

Runde	`i` vorher	`s` nachher	`i` nachher
1	1	1	2
2	2	3	3
3	3	**6**	4

$$1 + 2 + 3 = 6. \qquad \checkmark$$

Die Kosten.

	Bytecode	**Maschinencode**
Befehle je Runde	11	≈ 4
Takte je Befehl	≈ 18	≈ 1
Takte je Runde	**198**	**4**

Faktor fünfzig. Davon kommt Faktor drei aus der größeren Zahl von Befehlen und Faktor achtzehn aus dem Verteilersprung. Wer einen Interpreter beschleunigen will, muss am zweiten Faktor ansetzen.

(b) Die Ausführungsarten im Vergleich.
Die Aufgabe. Eine Schleife mit 10^9 einfachen Rechnungen.

System	**Laufzeit**	**Verhältnis**	**Start**
C, `-O2`	0,9 s	1,0	sofort
Rust, Go	0,9–1,1 s	1,0–1,2	sofort
Java, nach Anlauf	1,0 s	1,1	50–200 ms
JavaScript (V8)	1,3 s	1,4	30 ms
Python, mit JIT	4 s	4,4	20 ms
Python, ohne	45 s	50	20 ms
Baumdurchlauf	180 s	200	sofort

Die Tabelle gilt für reine Rechenarbeit und ist damit der ungünstigste Fall für Interpreter. Bei einem Programm, das die meiste Zeit auf Datenbank oder Netz wartet, verschwindet der Unterschied vollständig.

Wann was passt.

Kurze Läufe	Interpreter; der Anlauf einer Laufzeitübersetzung lohnt nicht
Lange Läufe	Übersetzung, vorab oder zur Laufzeit
Wartend	Gleichgültig; die Sprache bestimmt die Entwicklungszeit, nicht die Laufzeit
Rechnend	Übersetzt, oder eine Bibliothek, die es ist

Die letzte Zeile beschreibt die übliche Praxis in der numerischen Arbeit: Man schreibt in Python und ruft Bibliotheken, die in C oder Fortran übersetzt sind. Der Interpreter verteilt nur die Arbeit und ist deshalb kein Engpass.

(c) Wie eine virtuelle Maschine Code prüft.
Die Aufgabe. Fremder Bytecode soll ausgeführt werden, ohne dass er die Maschine beschädigen kann.

Stapelhöhe	An jeder Stelle muss sie eindeutig sein, unabhängig vom Weg dorthin
Typen	Ein `ADD` für Ganzzahlen darf keine Zeiger bekommen
Sprungziele	Müssen innerhalb des Codes und auf Befehlsanfängen liegen
Zugriffe	Feldindizes und Variablennummern im gültigen Bereich

Das Verfahren.
Der Prüfer führt den Code abstrakt aus: Er rechnet nicht mit Werten, sondern mit Typen.

Befehl	Stapel vorher	Stapel nachher
`iconst_1`	[]	[int]
`iload_0`	[int]	[int,int]
`iadd`	[int,int]	[int]
`aload_1`	[int]	[int,ref]
`iadd`	[int,ref]	**abgelehnt**

Das ist eine Datenflussanalyse wie in Kapitel 9, nur mit einem anderen Ziel. Statt zu optimieren, weist sie nach, dass nichts Unerlaubtes geschehen kann.

An Zusammenflüssen.
Kommen zwei Wege zusammen, müssen die Stapelbilder verträglich sein.

Weg 1	Weg 2	Ergebnis
[int]	[int]	[int]
[Auto]	[LKW]	[Fahrzeug]
[int]	[ref]	abgelehnt
[int]	[]	abgelehnt, Höhe

Die zweite Zeile zeigt, dass der Prüfer die Klassenhierarchie kennen muss. Er nimmt den gemeinsamen Obertyp, und das ist der Grund, warum die Prüfung Klassen nachladen kann.

Die Prüfung ist notwendig und nicht hinreichend

Sie zeigt, dass der Bytecode wohlgeformt ist. Über das, was das Programm tut, sagt sie nichts.

Prüfung fängt ab	Speicherzugriffe außerhalb, Typverwechslungen, Stapelfehler
Prüfung fängt nicht ab	Dateien löschen, Netzverbindungen aufbauen, Endlosschleifen

Für die zweite Zeile braucht man eine Rechteverwaltung, und das ist eine völlig andere Aufgabe (Band 5, Kapitel 14). Java hatte dafür einen Sicherheitsmanager, der sich als zu kompliziert erwies und 2021 für veraltet erklärt wurde.

WebAssembly zieht daraus eine Konsequenz und macht es umgekehrt: Ein Modul kann von sich aus überhaupt nichts. Es hat keinen Zugriff auf Dateien, Netz oder Uhrzeit, sondern nur auf die Funktionen, die ihm die Umgebung ausdrücklich übergibt.

Das ist der Grundsatz des geringsten Rechts aus Band 5, angewandt auf eine virtuelle Maschine. Die Voreinstellung sperrt, und jede Fähigkeit muss einzeln erteilt werden.

Nystrom[23] baut einen vollständigen Bytecode-Interpreter Schritt für Schritt; multisofts Handbuch[21] zeigt den Weg über C an einem realen System; Scott[25] ordnet die Ausführungsarten ein.

13 Übersetzen zur Laufzeit

Der Grundgedanke

Man übersetzt nicht vorab, sondern während das Programm läuft, und nur die Teile, die sich lohnen.

	Vorab	Zur Laufzeit
Weiß über die Daten	nichts	**alles**
Weiß über den Rechner	wenig	**genau**
Zeitbudget	Minuten	Millisekunden
Übersetzt	alles	nur das Heiße

Die erste Zeile ist der eigentliche Vorteil. Ein Übersetzer, der vorab arbeitet, muss für alle möglichen Eingaben richtigen Code erzeugen. Einer, der zur Laufzeit arbeitet, sieht die tatsächlichen Werte und kann darauf wetten.

Die Stufen.

Stufe	Ausführung	Aufwand	Güte
0	Interpreter	null	1×
1	schnelle Übersetzung	gering	10×
2	optimierende	hoch	50×

Der Wechsel geschieht nach Zählern. Eine Methode, die tausendmal aufgerufen wurde, oder eine Schleife mit zehntausend Durchläufen wandert eine Stufe höher. Die meisten Methoden bleiben auf Stufe null, und das ist richtig so.

Anteil der Methoden	Anteil der Laufzeit	
≈ 1 %	≈ 90 %	**lohnt sich**
≈ 99 %	≈ 10 %	Interpreter genügt

Spekulieren und zurücknehmen

Der entscheidende Kunstgriff: Man erzeugt Code unter Annahmen, die man nicht beweisen kann, und prüft sie zur Laufzeit.

```
function summe(a, b) { return a + b; }
```

In JavaScript kann + Zahlen addieren, Zeichenketten verbinden oder Objekte umwandeln. Ein vorab arbeitender Übersetzer müsste alle Fälle erzeugen. Ein Beobachter sieht dagegen, dass hier bisher immer Ganzzahlen kamen.

```
if (typ(a) != INT || typ(b) != INT)
goto ruecknahme;
add eax, ebx
jo ruecknahme      bei Überlauf
```

Fall	Takte	
Annahme stimmt	≈ 3	Prüfung plus Addition
Annahme stimmt nicht	≈ 10^4	Rücknahme

Was Rücknahme bedeutet.

1. Die Ausführung des optimierten Codes wird abgebrochen.
2. Der Zustand wird in die Form gebracht, die der Interpreter erwartet.
3. Die Ausführung wird im Interpreter an derselben Stelle fortgesetzt.
4. Der optimierte Code wird verworfen und später neu erzeugt.

Schritt 2 ist der aufwendige Teil und der Grund, warum eine Laufzeitübersetzung schwerer zu bauen ist als ein gewöhnlicher Übersetzer. Der optimierte Code hat Werte in Registern und Variablen wegoptimiert, die der Interpreter braucht. Die Zuordnung muss für jede Rücknahmestelle mitgeführt werden.

Wenn die Wetten dauernd verloren gehen

Ein Programm, das an einer heißen Stelle wechselnde Typen liefert, kann in eine Schleife geraten: optimieren, zurücknehmen, wieder optimieren.

```
for (let i = 0; i < n; i++)
  s = summe(s, i % 2 ?  1 :  "x");
```

Typen an der Stelle	Zustand	Leistung
immer derselbe	optimiert	1,0
zwei, gemischt	beide behandelt	0,6
viele, wechselnd	aufgegeben	0,1

Die Laufzeit merkt sich, wie oft eine Stelle schon zurückgenommen wurde, und gibt nach einigen Versuchen auf. Sie erzeugt dann allgemeinen Code, der alle Fälle behandelt und entsprechend langsam ist.

Daraus folgt eine praktische Regel für Sprachen mit dynamischen Typen: Man sollte an jeder Stelle immer denselben Typ verwenden, auch wenn die Sprache es nicht verlangt. Ein Feld, das mal Zahlen und mal Zeichenketten enthält, kostet ein Vielfaches.

Dasselbe gilt für Objekte. Wer Eigenschaften nachträglich hinzufügt, ändert die verborgene Klasse, und alle Zwischenspeicher, die auf die alte Form gewettet hatten, werden ungültig.

Von Self zu HotSpot

Anfang der 1980er Jahre galt es als ausgemacht, dass rein objektorientierte Sprachen langsam sein müssen. In Smalltalk ist jede Zahl ein Objekt und jede Addition ein Nachrichtenversand, und den kann man nicht vorab auflösen.
An der Stanford University arbeiteten David Ungar und Randall Smith ab 1986 an Self, einer Sprache, die noch radikaler war als Smalltalk. Sie hatte nicht einmal Klassen, sondern nur Objekte, die voneinander erbten.

Die Sprache war damit maximal dynamisch und, nach allem was man wusste, hoffnungslos langsam. Die Gruppe machte daraus ein Forschungsprogramm: Wie schnell kann man so etwas machen?

Die Antwort bestand aus einer Reihe von Ideen, die heute überall stecken.

Verborgene Klassen	Objekte mit gleicher Struktur bekommen intern dieselbe Beschreibung
Zwischenspeicher am Aufruf	Man merkt sich an der Aufrufstelle, welcher Typ zuletzt kam
Mehrfach	Bei zwei bis vier Typen werden alle gespeichert
Rücknahme	Optimierter Code darf falsch liegen, wenn er es merkt

URS HÖLZLE *und Ungar beschrieben 1994, wie viel der zweite und dritte Punkt bringen. Weil über neunzig Prozent aller Aufrufstellen nur einen einzigen Typ sehen, wird aus einer Suche in der Vererbungskette ein Vergleich und ein direkter Sprung.*

Verfahren	**Takte je Aufruf**	
Suche in der Kette	≈ 20	
Zwischenspeicher	≈ 2	
plus Einsetzen	**0**	der Aufruf verschwindet

Self erreichte damit etwa die Hälfte der Geschwindigkeit von optimiertem C, für eine Sprache ohne statische Typen und ohne Klassen. Das galt zuvor als ausgeschlossen.

Die Firma Sun übernahm die Gruppe. Aus dem Self-Projekt wurde über den Zwischenschritt Strongtalk die virtuelle Maschine HotSpot, die 1999 in Java einzog und dort dieselben Verfahren anwandte.

1986	Self beginnt
1994	Hölzle und Ungar veröffentlichen die Messungen
1999	HotSpot in Java
2008	V8 bringt dieselben Verfahren zu JavaScript
2009	LuaJIT, PyPy

Ein Forschungsprojekt zu einer Sprache, die kaum jemand benutzt hat, liegt damit der Ausführung praktisch jeder Webseite zugrunde. Das kommt in der Informatik häufiger vor, als man denkt.

Aufgabe 13.1.

(a) Rechnen Sie aus, wann sich eine Laufzeitübersetzung lohnt.
(b) Untersuchen Sie den Zwischenspeicher an der Aufrufstelle.
(c) Vergleichen Sie Vorabübersetzung und Laufzeitübersetzung.

Lösung Schritt für Schritt.

(a) Wann sich die Übersetzung lohnt.
Die Größen.

Größe	**Wert**	
Interpretiert je Durchlauf	500 ns	
Übersetzt je Durchlauf	20 ns	
Übersetzungsaufwand	2 ms	optimierend

Die Gewinnschwelle.

$$n \cdot 500 = 2\,000\,000 + n \cdot 20 \quad \Longrightarrow \quad n = \frac{2 \cdot 10^6}{480} \approx 4167.$$

n	**interpretiert**	**übersetzt**	**besser**
100	0,05 ms	2,0 ms	interpretieren
4167	2,08 ms	2,08 ms	gleich
10^5	50 ms	**4** ms	übersetzen
10^7	5000 ms	**202** ms	übersetzen

Die Schwelle von etwa viertausend erklärt, warum die üblichen Zähler in dieser Größenordnung liegen. HotSpot übersetzt eine Methode nach etwa zehntausend Aufrufen und eine Schleife früher, weil dort mehr Durchläufe zu erwarten sind.

Warum es mehrere Stufen gibt.

Stufe	**Aufwand**	**je Durchlauf**	**Schwelle**
Interpreter	0	500 ns	—
schnell	0,05 ms	60 ns	≈ 110
optimierend	2 ms	20 ns	≈ 4900

Die mittlere Stufe lohnt sich schon nach hundert Durchläufen und fängt damit den großen Bereich zwischen den beiden Extremen ab. Sie sammelt zugleich die Messwerte, die die dritte Stufe für ihre Wetten braucht.

(b) Der Zwischenspeicher an der Aufrufstelle.
Die Ausgangslage. Ein Aufruf `obj.flaeche()` in einer Schleife.
Ohne Zwischenspeicher.

Schritt	**Takte**	
Klasse des Objekts holen	2	
Methode suchen	≈ 15	Streutabelle, Vererbung
Sprung	2	
Summe	≈ 19	

Mit einfachem Zwischenspeicher.

```
if (obj.klasse == Kreis) {
  ...  Rumpf von Kreis::flaeche,
eingesetzt ...
} else { langsamerWeg(); }
```

Fall	Takte	Häufigkeit
Treffer	**2**	$\approx 93\,\%$
Fehlgriff	≈ 25	$\approx 7\,\%$
Mittel	$\approx 3{,}6$	

$$\text{Gewinn} = \frac{19}{3{,}6} \approx 5{,}3.$$

Der eigentliche Gewinn liegt aber nicht in den gesparten Takten, sondern darin, dass der Rumpf jetzt eingesetzt ist. Damit kann der Optimierer über den Aufruf hinweg arbeiten, Konstanten hineinschieben und tote Zweige entfernen.

Die Verteilung in der Praxis.

Typen je Stelle	Anteil	Behandlung
genau einer	$\approx 93\,\%$	einfacher Vergleich
zwei bis vier	$\approx 5\,\%$	Kette von Vergleichen
viele	$\approx 2\,\%$	Tabelle, kein Einsetzen

Diese Verteilung ist über Sprachen und Programme hinweg erstaunlich stabil. Sie ist der Grund, warum das ganze Verfahren funktioniert: Man wettet auf einen sehr häufigen Fall und zahlt nur selten drauf.

(c) Vorab oder zur Laufzeit.

	Vorab	Zur Laufzeit
Startzeit	sofort	Anlauf
Höchstleistung	sehr gut	teils besser
Speicher	gering	Code plus Übersetzer
Vorhersagbarkeit	gut	schwankt
Zielrechner	festgelegt	der tatsächliche

Warum die zweite Zeile stimmen kann.

Tatsächliche Befehle	Der Code darf alles benutzen, was dieser Prozessor kann, ohne Rücksicht auf ältere
Tatsächliche Zweige	Selten genommene Zweige werden ausgelagert
Tatsächliche Typen	Nur die vorkommenden Fälle werden erzeugt
Tatsächliche Ziele	Virtuelle Aufrufe werden zu direkten

Ein vorab arbeitender Übersetzer könnte all das nur mit einem Messlauf erreichen. Genau das ist die Optimierung nach Profil aus Kapitel 9, und sie wird selten benutzt, weil sie umständlich ist. Die Laufzeitübersetzung bekommt sie geschenkt.

Wo die Vorabübersetzung gewinnt.

Kurze Läufe	Ein Werkzeug, das 50 ms läuft, kommt nie in die optimierte Stufe
Kleine Geräte	Der Übersetzer selbst braucht Speicher
Echtzeit	Eine Rücknahme zur Unzeit ist nicht hinnehmbar
Ausschließbarer Speicher	Wo Seiten nicht zugleich schreibbar und ausführbar sein dürfen

Die letzte Zeile ist ein realer Konflikt mit der Sicherheit. Eine Laufzeitübersetzung muss Code erzeugen und dann ausführen. Systeme, die das grundsätzlich verbieten, schließen sie damit aus, und deshalb gibt es auf manchen Mobilplattformen für fremde Programme keine.

Die Mischformen.

Vorab plus Laufzeit	Android übersetzt bei der Installation vorab und verbessert später anhand der Nutzung
Zwischenspeicher	Erzeugter Code wird gespeichert und beim nächsten Start wiederverwendet
Aufteilung	Die Bibliothek vorab, die Anwendung zur Laufzeit

Die erste Zeile ist der interessanteste Fall. Android sammelt beim Gebrauch ein Profil, übersetzt in Ruhezeiten neu und wird damit über Tage hinweg schneller. Das Gerät lernt, wie sein Besitzer die Anwendung benutzt.

Aycock[3] gibt einen Überblick über die Geschichte der Laufzeitübersetzung; Hölzle und Ungar[12] beschreiben den Zwischenspeicher an der Aufrufstelle; Appel[2] behandelt die Grundlagen.

14 Übersetzer in der Praxis

Was einen Übersetzer brauchbar macht

Die Lehrbuchdarstellung endet bei der Codeerzeugung. Für den täglichen Gebrauch entscheiden andere Dinge.

Richtigkeit	Der erzeugte Code muss bedeuten, was der Quelltext bedeutet
Fehlermeldungen	Sie kosten mehr Arbeitszeit als alles andere
Übersetzungsdauer	Sie bestimmt, wie oft jemand ausprobiert
Auskunftsfähigkeit	Der Editor will wissen, was ein Name bedeutet
Fehlersuche	Der erzeugte Code muss sich auf den Quelltext zurückführen lassen

Die zweite Zeile wird in Lehrbüchern kaum behandelt und ist die häufigste Begegnung zwischen Übersetzer und Mensch. Ein Programmierer sieht am Tag vielleicht zwanzig Fehlermeldungen und ein einziges Mal den erzeugten Code, nämlich nie.

Was eine gute Fehlermeldung ausmacht.

1. **Wo**: Datei, Zeile, Spalte, und die Stelle im Text markiert.
2. **Was**: Was der Übersetzer erwartet hat und was er fand.
3. **Warum**: Welche Regel verletzt ist, in verständlichen Worten.
4. **Wie weiter**: Ein konkreter Vorschlag, wenn er möglich ist.
5. **Der Zusammenhang**: Bei einem Typfehler auch die Stelle, an der der Typ herkam.

Schlecht	`parse error before '}'`
Besser	`Zeile 42: '}' erwartet, aber Dateiende gefunden`
Gut	`Zeile 42: der in Zeile 17 mit '{' geöffnete Block wurde nie geschlossen`

Der Unterschied zwischen der zweiten und der dritten Zeile ist erheblich. Die zweite sagt, wo der Übersetzer gescheitert ist. Die dritte sagt, wo der Mensch den Fehler gemacht hat, und das sind meist zwei verschiedene Stellen.

Weiterarbeiten nach einem Fehler

Ein Übersetzer, der beim ersten Fehler aufhört, zwingt zu einem Durchlauf je Fehler. Also muss er sich erholen und weitersuchen.

Panikmodus	Symbole überspringen bis zu einem Wiedereinstiegspunkt, meist `;` oder `}`
Reparatur	Ein fehlendes Symbol gedanklich einfügen und weitermachen
Fehlerregeln	Häufige Fehler stehen als eigene Regeln in der Grammatik

Die dritte Zeile ist die aufwendigste und die beste. Wenn man weiß, dass Anfänger in einer bestimmten Sprache = statt == schreiben, nimmt man diesen Fall ausdrücklich auf und meldet ihn mit einem passenden Text.

Die Gefahr der Folgefehler.
Nach einem Fehler ist der Zustand des Übersetzers unsicher. Meldet er munter weiter, entstehen Meldungen, die nur Folgen der ersten sind.

Verhalten	Meldungen	davon echt
Keine Erholung	1	1
Naiv weiter	180	3
Mit Unterdrückung	7	3

Ein vergessenes Semikolon in einer Kopfdatei kann in C++ hunderte Meldungen erzeugen. Deshalb unterdrücken gute Übersetzer weitere Meldungen aus derselben Gegend und brechen nach einer Höchstzahl ab.

Wie man einen Übersetzer prüft

Ein Übersetzer ist ein Programm, dessen Fehler sich in allen anderen Programmen zeigen. Das macht die Prüfung besonders wichtig und besonders schwer.

Testfälle	Sammlungen von Programmen mit bekanntem Ergebnis
Rückfalltests	Jeder behobene Fehler wird zu einem dauerhaften Test
Selbstaufbau	Der Übersetzer übersetzt sich selbst (Kap. 1)
Zufallsprogramme	Programme erzeugen und die Ergebnisse vergleichen
Beweise	Die Übersetzung wird maschinell verifiziert

Die vierte Zeile im Detail.
Das Werkzeug `Csmith` erzeugt zufällige C-Programme, die genau ein Ergebnis haben und keine undefinierten Konstruktionen enthalten. Dann übersetzt man dasselbe Programm mit mehreren Übersetzern und mit mehreren Optimierungsstufen.

> *Alle Ergebnisse müssen übereinstimmen. Wo sie es nicht tun, hat einer der Übersetzer einen Fehler, und man muss nur noch herausfinden, welcher. Man braucht dazu kein Wissen über die richtige Antwort.*

Übersetzer	**gefundene Fehler**	**Bemerkung**
GCC	über 100	
LLVM/Clang	über 100	
Kommerzielle	etliche	
CompCert	**0 im geprüften Teil**	mit Korrektheitsbeweis

> *Die letzte Zeile ist bemerkenswert. CompCert ist ein C-Übersetzer, für den ein maschinell geprüfter Beweis existiert, dass der erzeugte Code dasselbe bedeutet wie der Quelltext. Csmith fand darin Fehler im Vorderteil und im nicht bewiesenen Umfeld, aber keinen in den bewiesenen Umformungen.*

Warum das nicht alle so machen.

	CompCert	GCC
Optimierungen	wenige	sehr viele
Erzeugter Code	10–20 % langsamer	Bezugsgröße
Zielarchitekturen	wenige	über 40
Aufwand	Jahrzehnte	

Für Flugsteuerungen und Kernkraftwerke sind zwanzig Prozent Geschwindigkeitsverlust ein guter Preis für die Gewissheit. Für alles andere nicht.

Vertraue niemandem, den du nicht selbst gebaut hast

In seiner Dankesrede zum Turing-Award beschrieb KEN THOMPSON 1984 eine Möglichkeit, die sich unmittelbar aus dem Selbstaufbau ergibt und die seither niemand mehr vergessen hat.
Der Gedankengang hat drei Stufen.
Stufe 1. Man baut in einen C-Übersetzer eine Hintertür ein: Erkennt er, dass er gerade das Anmeldeprogramm übersetzt, so fügt er ein zusätzliches Passwort ein, das immer funktioniert.

Das ist entdeckbar. Wer den Quelltext des Übersetzers liest, findet die Stelle.

Stufe 2. Man erweitert die Hintertür: Erkennt der Übersetzer, dass er gerade *sich selbst* übersetzt, so fügt er beide Hintertüren in das Ergebnis ein, also die für das Anmeldeprogramm und diese hier.
Stufe 3. Man übersetzt den Übersetzer einmal mit dieser Fassung und wirft den veränderten Quelltext weg.

Quelltext	vollkommen sauber, nichts zu finden
Ausführbarer Übersetzer	enthält beide Hintertüren
Jede neue Fassung	bekommt sie wieder eingebaut

Von nun an ist der Angriff unsichtbar. Man kann den Quelltext des Übersetzers und den des Anmeldeprogramms Zeile für Zeile prüfen und wird nichts finden, denn dort steht nichts. Die Hintertür lebt allein in der ausführbaren Datei und pflanzt sich bei jedem Selbstaufbau fort.

Thompsons Schlusssatz lautete sinngemäß, man könne keinem Programm trauen, das man nicht vollständig selbst geschrieben habe, und das schließe den Übersetzer ein.

Es ist dieselbe Art von Selbstbezug, die in Band 2 beim Halteproblem auftauchte, nur mit umgekehrtem Vorzeichen: Dort war die Selbstanwendung ein Beweismittel, hier ist sie ein Werkzeug.

Gibt es einen Ausweg?
DAVID A. WHEELER beschrieb 2005 ein Verfahren, das er *Diverse Double-Compiling* nannte.

1. Man übersetzt den Quelltext des verdächtigen Übersetzers mit einem **zweiten**, unabhängigen Übersetzer. Ergebnis A.
2. Mit A übersetzt man den Quelltext erneut. Ergebnis B.
3. Mit dem verdächtigen Übersetzer übersetzt man denselben Quelltext. Ergebnis C.
4. Sind B und C gleich, so enthält der verdächtige Übersetzer die Hintertür nicht.

Die Begründung: B ist aus dem geprüften Quelltext entstanden, über einen Übersetzer, der von der Hintertür nichts weiß. Wenn C damit übereinstimmt, hat der verdächtige Übersetzer nichts hinzugefügt.

Das Verfahren verlangt zweierlei: einen zweiten Übersetzer, der die Hintertür nicht ebenfalls enthält, und eine reproduzierbare Übersetzung, wie sie in Kapitel 1 beschrieben wurde.

Genau deshalb ist die Reproduzierbarkeit von Übersetzungen seit etwa 2015 zu einem ernsthaften Anliegen geworden. Sie ist die Voraussetzung dafür, überhaupt prüfen zu können, ob ein ausgeliefertes Programm dem entspricht, was im Quelltext steht.

Aufgabe 14.1.

(a) Untersuchen Sie, wovon die Übersetzungsdauer abhängt.
(b) Analysieren Sie, wie Fehlersuchinformationen entstehen und was Optimierung damit anstellt.

(c) Verkleinern Sie ein fehlerauslösendes Programm systematisch.

Lösung Schritt für Schritt.

(a) Die Übersetzungsdauer.
Warum sie zählt.

Unter 1 s	Man bleibt bei der Sache und probiert weiter
Bis 10 s	Man wartet, aber die Aufmerksamkeit hält
Über 1 min	Man wendet sich etwas anderem zu und kommt schwer zurück

Diese Schwellen sind der Grund, warum die Übersetzungsdauer beim Entwurf von Go ausdrücklich als Ziel genannt wurde. Ein Team, das zwanzig Minuten auf eine Übersetzung wartet, arbeitet anders als eines, das zwei Sekunden wartet.

Wo die Zeit hingeht.

Stufe	**C**	**C++**	**Go**
Kopfdateien lesen	10 %	60 %	2 %
Syntax	8 %	10 %	12 %
Semantik	12 %	15 %	25 %
Optimierung	45 %	10 %	35 %
Erzeugung	25 %	5 %	26 %

Warum C++ so heraussticht.
Eine Kopfdatei wird in jeder Übersetzungseinheit erneut gelesen und ausgewertet.

Größe	**Wert**	
Quelldateien	500	
Zeilen je Datei	300	
Zeilen aus Kopfdateien	40 000	je Datei
Selbst geschrieben	150 000	
Tatsächlich gelesen	**$2 \cdot 10^7$**	**Faktor 133**

Der Übersetzer liest zwanzig Millionen Zeilen, um hundertfünfzigtausend zu übersetzen. Das ist kein Fehler in der Umsetzung, sondern eine unmittelbare Folge des Textersetzungsmodells, das C 1972 eingeführt hat.

Vorübersetzte Köpfe	Der Zustand nach den Kopfdateien wird einmal gespeichert; hilft, wenn alle dieselben benutzen
Zusammenfassen	Mehrere Quelldateien werden zu einer verbunden; spart Wiederholungen
Module	Seit C++20: eine Schnittstelle wird einmal ausgewertet und als Binärform abgelegt

Die dritte Zeile ist die eigentliche Lösung und war fünfzig Jahre überfällig. Andere Sprachen haben Module von Anfang an gehabt; deshalb steht in der Go-Spalte oben eine 2 statt einer 60.

(b) Fehlersuchinformationen.
Was gespeichert wird.

Zeilentabelle	Welche Maschinenadresse gehört zu welcher Quellzeile
Variablenorte	Wo liegt eine Variable gerade: Register, Stapel, oder nirgends
Typangaben	Wie ist ein Speicherbereich zu deuten
Rahmenauskunft	Wie kommt man vom aktuellen Rahmen zum aufrufenden

Programm	**ohne**	**mit**
Ausführbare Datei	2 MB	14 MB
Laufzeit	unverändert	unverändert

Die zweite Zeile ist wichtig: Fehlersuchinformationen kosten nur Platz, keine Geschwindigkeit. Sie liegen in eigenen Abschnitten, die beim Laden gar nicht in den Speicher kommen. Man kann sie sogar in eine getrennte Datei auslagern.

Was die Optimierung damit anstellt.

Verschobene Zeilen	Anweisungen werden umgeordnet; der Haltepunkt liegt nicht mehr dort, wo man ihn erwartet
Verschwundene Variablen	Ein Wert lebt nur in einem Register oder wurde ganz wegoptimiert
Verschwundene Aufrufe	Eine eingesetzte Funktion taucht im Aufrufkeller nicht mehr auf
Springende Anzeige	Der Debugger zeigt Zeile 20, dann 17, dann 23

Die zweite Zeile führt zu der berüchtigten Meldung „Wert wegoptimiert“. Die Variable steht im Quelltext, und im laufenden Programm gibt es sie nicht. Beides ist richtig.

Einstellung	Geschwindigkeit	Fehlersuche
`-O0 -g`	1,0	sehr gut
`-O1 -g`	3,2	brauchbar
`-O2 -g`	4,5	mühsam
`-Og -g`	2,5	**gut**

Die letzte Zeile ist ein eigens dafür geschaffener Kompromiss: alle Optimierungen, die die Fehlersuche nicht stören, und keine anderen. Sie wird zu selten benutzt.

Der Fehler, der bei eingeschalteter Optimierung verschwindet

Ein Programm stürzt mit `-O2` ab und läuft mit `-O0`. Die Versuchung ist groß, den Übersetzer zu verdächtigen.

Meist	Das Programm enthält eine undefinierte Konstruktion, und der Übersetzer nutzt sie aus
Manchmal	Ein Wettlauf zwischen Fäden, den die Umordnung sichtbar macht (Band 5)
Selten	Ein Fehler im Übersetzer

Die erste Zeile ist der häufigste Fall. Der Übersetzer darf annehmen, dass kein undefiniertes Verhalten vorkommt, und rechnet auf dieser Grundlage. Bei `-O0` rechnet er weniger und die Annahme fällt nicht auf.

Was hilft.

`-fsanitize=address`	Findet Zugriffe außerhalb von Feldern und auf freigegebenen Speicher
`-fsanitize=undefined`	Findet Überläufe, Verschiebungen um zu viele Stellen, Nullzeiger
`-Wall -Wextra`	Ein erheblicher Teil solcher Fehler wird schon gewarnt

Die letzte Zeile ist die billigste und wird am häufigsten übergangen. Wer Warnungen abschaltet, statt sie zu beheben, hat die kostenlose Hälfte der Fehlersuche verschenkt.

(c) Ein Programm systematisch verkleinern.
Die Lage. Ein Programm von 40 000 Zeilen bringt den Übersetzer zum Absturz. Für eine Fehlermeldung braucht man ein kleines Beispiel.
Das Verfahren. Man halbiert und prüft, ob der Fehler bleibt. Es ist eine binäre Suche über Programmteile und heißt **Delta-Debugging**.

1.	Die Eingabe in zwei Hälften teilen.
2.	Jede Hälfte für sich prüfen. Bleibt der Fehler, mit dieser weitermachen.
3.	Bleibt er in keiner, feiner unterteilen und Teile paarweise entfernen.
4.	Wiederholen, bis nichts mehr wegfallen kann.

Ein Verlauf.

Runde	Zeilen	Versuche	Fehler?
0	40 000	—	ja
1	20 000	2	ja
2	10 000	2	ja
5	1250	8	ja
10	39	26	ja
14	**6**	18	ja

Aufwand $\approx \mathcal{O}(\log n)$ Runden, insgesamt etwa 60 Übersetzungsläufe.

Aus 40 000 Zeilen werden sechs, vollautomatisch, in wenigen Minuten. Wer das von Hand versucht, braucht einen Tag und gibt vorher auf.

Die Schwierigkeit.
Ein zufällig verkleinertes Programm ist meist syntaktisch falsch oder enthält undefiniertes Verhalten. Dann stürzt der Übersetzer aus einem anderen Grund ab, und die Suche läuft in die Irre.

Sprachbewusst	Das Werkzeug entfernt nur ganze Sprachkonstruktionen, nicht einzelne Zeilen
Prüfschritt	Nach jeder Verkleinerung wird geprüft, ob das Programm noch gültig ist
Genaue Bedingung	Nicht „stürzt ab“, sondern „stürzt mit dieser Meldung an dieser Stelle ab“

Die dritte Zeile ist entscheidend. Eine zu weite Bedingung führt zu einem kleinen Programm, das einen ganz anderen Fehler auslöst. Das Werkzeug `C-Reduce` *setzt alle drei Punkte um und ist bei Übersetzerfehlern der Standardweg.*

Yang und Mitautoren[28] beschreiben Csmith und die gefundenen Fehler; Thompsons Dankesrede[26] ist vier Seiten lang und sollte jeder einmal gelesen haben; Cooper und Torczon[6] gehen auf die praktischen Fragen ein.

15 Der Stand im Jahr 2026

Was gelöst ist und was nicht

Frage	Stand	Bemerkung
Lexikalische Analyse	gelöst	seit den 1970er Jahren
Syntaktische Analyse	gelöst	mehr als genug Verfahren
Registerzuteilung	gelöst	Färbung und Nachfolger
Zwischenformen	gelöst	SSA hat sich durchgesetzt
Fehlermeldungen	in Arbeit	sehr ungleich verteilt
Übersetzungsdauer	in Arbeit	Module, Zwischenspeicher
Nebenläufigkeit übersetzen	in Arbeit	Speichermodelle sind schwer
Übersetzer beweisen	in Arbeit	CompCert zeigt, dass es geht
Beste Optimierungsreihenfolge	offen	**niemand weiß sie**
Undefiniertes Verhalten	offen	ein Entwurfsfehler von 1989
Automatische Parallelisierung	offen	seit vierzig Jahren

Die erste Zeile des unteren Abschnitts verdient eine Erläuterung. Optimierungen beeinflussen einander: Die eine schafft Gelegenheiten für die andere und zerstört Gelegenheiten für eine dritte. Welche Reihenfolge die beste ist, hängt vom Programm ab, und es gibt keine Theorie dazu.

Optimierungen	Reihenfolgen	
5	120	noch prüfbar
10	$3{,}6 \cdot 10^6$	
50	$3 \cdot 10^{64}$	aussichtslos

GCC und LLVM haben je über hundert Durchgänge. Ihre Reihenfolge ist über Jahrzehnte durch Ausprobieren entstanden und steht als feste Liste im Quelltext. Niemand behauptet, sie sei optimal.

Fünf Entwicklungen

Erstens: Der Übersetzer ist in den Editor gewandert.
Bis etwa 2015 hatte jeder Editor seine eigene, halbrichtige Vorstellung davon, was ein Programm bedeutet. Heute fragt er den Übersetzer.

Sprachserver	Ein Prozess, der eine Sprache versteht und Fragen des Editors beantwortet
Ein Protokoll	Ein Server je Sprache, ein Anschluss je Editor, statt $m \cdot n$ Erweiterungen
Dieselbe Antwort	Editor und Übersetzer sind sich einig, weil es derselbe Code ist

Das ist genau die Rechnung aus Kapitel 1, nur auf Werkzeuge angewandt. Und es hat Folgen für den Bau von Übersetzern: Sie müssen jetzt mit unvollständigem, fehlerhaftem Text umgehen können, denn während des Tippens ist er das immer.

	klassisch	für den Editor
Eingabe	vollständig	**halb fertig**
Fehler	abbrechen genügt	weiterarbeiten
Antwortzeit	Sekunden	**Millisekunden**
Wiederholung	alles neu	nur das Geänderte

Zweitens: Übersetzer als Prüfwerkzeug.

Die Grenze zwischen Übersetzer und Prüfprogramm verschwindet. Was früher ein getrenntes Werkzeug war, steht heute im Übersetzer.

Eigentum	Rust prüft zur Übersetzungszeit, wem ein Wert gehört
Nullwerte	Kotlin, Swift und C# unterscheiden Typen, die null sein dürfen, von solchen, die es nicht dürfen
Nebenläufigkeit	Go meldet Wettläufe zur Laufzeit, Rust verhindert sie zur Übersetzungszeit
Erschöpfung	Der Übersetzer prüft, ob eine Fallunterscheidung alle Fälle abdeckt

Die letzte Zeile ist unscheinbar und sehr wirksam. Wer eine Aufzählung um einen Wert erweitert, bekommt an jeder Stelle im Programm einen Fehler, an der der neue Fall fehlt. In Sprachen ohne diese Prüfung findet man sie im Betrieb.

Drittens: Ein Zwischenformat für das Netz.
WebAssembly ist eine Zwischenform, die nicht in einem Übersetzer bleibt, sondern ausgeliefert wird.

Eigenschaft	**Wert**	
Übersetzt aus	C, C++, Rust, Go	und weiteren
Läuft in	Browser, Server, eingebettet	
Geschwindigkeit	$\approx 80\,\%$ von nativ	
Prüfung	**vor der Ausführung**	Typen und Sprungziele

Die letzte Zeile ist der Grund, warum das Format so aussieht, wie es aussieht. Es ist absichtlich so entworfen, dass sich in einem Durchlauf prüfen lässt, ob ein Modul harmlos ist. Ausdruckskraft wurde gegen Prüfbarkeit eingetauscht.

Viertens: Undefiniertes Verhalten wird zurückgedrängt.
Der C-Standard von 1989 ließ eine große Zahl von Fällen offen, damit Übersetzer auf ungewöhnlichen Rechnern effizient bleiben konnten. Optimierer haben daraus mit der Zeit weitreichende Annahmen abgeleitet.

Damals gemeint	„Der Übersetzer muss sich nicht darum kümmern“
Heute genutzt	„Der Übersetzer darf annehmen, dass es nie vorkommt“

Aus dieser Verschiebung folgt, dass ein Übersetzer eine Prüfung entfernen darf, die der Programmierer ausdrücklich hingeschrieben hat, wenn sie nur bei undefiniertem Verhalten anschlagen könnte. Das hat zu echten Sicherheitslücken geführt.

Neue Sprachen	legen Überlaufverhalten fest, statt es offenzulassen
C und C++	streichen Fälle aus der Liste; Vorzeichenzahlen sind seit C++20 im Zweierkomplement festgelegt
Werkzeuge	Die Prüfer finden zur Laufzeit, was der Standard offenlässt

Fünftens: Sprachmodelle im Werkzeugkasten.
Seit etwa 2022 gehören Programme, die Code vorschlagen, zum Alltag vieler Entwickler. Für den Übersetzerbau folgt daraus weniger, als man vermuten könnte, und etwas Bestimmtes.

Unverändert	Der Übersetzer prüft weiterhin; ein Vorschlag ist erst Code, wenn er übersetzt und läuft
Wichtiger geworden	Typprüfung und Warnungen, weil mehr Code entsteht, den niemand Zeile für Zeile gelesen hat
Neu	Fehlermeldungen werden auch von Programmen gelesen und müssen maschinell auswertbar sein

Die mittlere Zeile ist die eigentliche Verschiebung. Ein strenges Typsystem war immer nützlich; wenn ein erheblicher Teil des Codes nicht von Hand geschrieben wurde, wird es zur wichtigsten verbliebenen Prüfinstanz.

Was aus den Sprachen dieses Bandes geworden ist

Die Sprachen, die in diesem Band vorkamen, sind sehr unterschiedlich gealtert.

Sprache	Stand	Erbe
FORTRAN	lebt	Feldrechnung, Numerik
LISP	lebt	Programme als Daten, Abschlüsse
ALGOL 60	tot	**fast alles, was danach kam**
Pascal	fast tot	Lehre, klare Struktur
C	lebt	alle Systemsoftware
xBase	Nischen	Satzorientierung, Codeblöcke
4GL-Sprachen	Nischen	Dialog und Datenzugriff in der Sprache

Die dritte Zeile ist der interessanteste Fall. ALGOL 60 wurde kaum benutzt und hat trotzdem gewonnen: Blockstruktur, Rekursion, lexikalische Sichtbarkeit und die formale Sprachbeschreibung stammen von dort und stecken heute in praktisch jeder Sprache.

Die beiden letzten Zeilen verdienen eine Anmerkung.
Sprachen wie die xBase-Familie oder ROSI-SQL werden gern als überholt abgetan. Aus der Sicht dieses Bandes ist das zu einfach. Sie haben eine Entscheidung getroffen, die heute wieder diskutiert wird: die Aufgabenstellung eines Anwendungsgebiets unmittelbar in die Sprache zu nehmen, statt sie auf Bibliotheken zu verteilen.

	4GL, damals	heute
Datenzugriff	Sprachbefehl	eingebettetes SQL, ORM
Dialoge	Sprachbefehl	Rahmenwerk
Dezimalrechnung	eingebauter Typ	Bibliothekstyp
Zielgruppe	ein Gebiet	**fachspezifische Sprachen**

Der Begriff hat gewechselt, die Idee nicht. Was in den 1980er Jahren „Sprache der vierten Generation“ hieß, heißt heute „domänenspezifische Sprache“, und die Abwägung ist dieselbe geblieben: Ausdruckskraft für ein Gebiet gegen Brauchbarkeit für alle anderen.

Drei Einsichten

Erstens: Der Übersetzer ist der Ort, an dem sich Sprache und Maschine treffen. Band 4 beschrieb, was ein Prozessor kann. Band 5 beschrieb, wer die Ausführung verwaltet. Dieser Band beschreibt, wie aus dem, was ein Mensch meint, das wird, was eine Maschine tut.

> *Und er zeigt, dass diese Übersetzung nicht wörtlich ist. Der Übersetzer erhält die Bedeutung und ändert alles andere: die Reihenfolge, die Namen, die Zahl der Anweisungen, gelegentlich die Rechenart. Was erhalten bleiben muss, legt die Sprachbeschreibung fest, und deshalb ist sie wichtiger, als sie aussieht.*

Zweitens: Fast jede Entscheidung im Sprachentwurf ist ein Tausch.

Man gewinnt	Man verliert	Beispiel
Sicherheit	Ausdrucksfreiheit	Rusts Eigentumsmodell
Geschwindigkeit	Vorhersagbarkeit	undefiniertes Verhalten
Ausdruckskraft	Übersetzbarkeit	dynamische Typen
Bequemlichkeit	Durchschaubarkeit	implizite Umwandlungen
Gebietsnähe	Allgemeinheit	4GL, fachspezifische Sprachen

Hoare hat 1973 aufgeschrieben, worauf es dabei ankommt, und sein wichtigster Punkt lautet: Eine Sprache soll den Programmierer nicht überraschen. Sprachen, die diesen Punkt verletzt haben, sind daran nicht gestorben, aber sie haben ihre Benutzer viel Zeit gekostet.

Drittens: Der Selbstbezug ist hier ein Werkzeug.
Ein Übersetzer, der sich selbst übersetzt, ist die praktischste Form eines Kreises, den man in Band 2 noch als Grenze kennengelernt hat.

	Band 2	Band 6
Selbstanwendung	beweist eine Grenze	**baut ein Werkzeug**
Ergebnis	Halteproblem unlösbar	Übersetzer trägt sich selbst
Kehrseite	—	Thompsons Hintertür

Dieselbe Struktur, drei verschiedene Folgen. Das ist der Leitgedanke dieser Reihe: Universalität und Grenze sind nicht zwei Eigenschaften, sondern eine, von zwei Seiten betrachtet.

Band 7 wendet sich der Informationstheorie zu: der Frage, wie viel in einer Nachricht überhaupt steckt, wie weit sich Daten verdichten lassen und warum es dabei eine harte Grenze gibt. Backus' Aufsatz zur Geschichte von FORTRAN[4], Dijkstras Streitschrift von 1968[8] und McCarthys LISP-Aufsatz[18] sind drei kurze Texte, mit denen sich die Wartezeit gut überbrücken lässt.

Der Übersetzer ist der einzige Leser, der einen Text
vollständig versteht und ihn trotzdem
nicht beurteilen kann.

— eine Zuspitzung, die auf eine Beobachtung von
NIKLAUS WIRTH *zurückgeht: Ein Übersetzer prüft, ob ein Programm*
den Regeln entspricht, nicht ob es das Richtige tut.
Das bleibt Aufgabe des Menschen, und dabei hilft nur
ein Text, den auch ein Mensch lesen kann.

Part V

Anhang

A Zahlen und Kenngrößen

Größenordnungen

Größe	typisch	Bemerkung
Zeichen je Symbol	3	im Quelltext
Symbole je Baumknoten	1,2	
Zwischenanweisungen je Zeile	2–4	
Maschinenbefehle je Zeile	3–8	bei `-O2`
Scanner	≈ 2 % des Codes	≈ 5 % der Zeit
Syntaxanalyse	≈ 5 %	≈ 10 %
Semantik	≈ 15 %	≈ 15 %
Optimierung	≈ 50 %	≈ 50 %
Codeerzeugung	≈ 28 %	≈ 20 %
GCC	≈ 15 Mio. Zeilen	über 40 Zielarchitekturen
LLVM	≈ 30 Mio. Zeilen	
Optimierungsdurchgänge	über 100	feste Reihenfolge

Was Optimierung bringt

Stufe	Laufzeit	Codegröße	Übers.-dauer
`-O0`	1,00	1,00	1,0
`-O1`	0,31	0,72	1,8
`-O2`	0,22	0,78	2,6
`-O3`	0,19	1,15	3,4
`-Os`	0,26	**0,61**	2,4

Der Sprung von `-O0` auf `-O1` bringt den größten Teil des Gewinns. Von `-O2` auf `-O3` kommt wenig hinzu, und der Code wird größer, was über den Befehlscache gelegentlich sogar schadet.

Die Zerlegungsverfahren im Überblick

Klasse	Richtung	Umfang	Praxis
LL(1)	oben nach unten	klein	von Hand
LL(k)	oben nach unten	mittel	**verbreitet**
LR(0)	unten nach oben	klein	—
SLR(1)	unten nach oben	mittel	—
LALR(1)	unten nach oben	groß	`yacc`, `bison`
LR(1)	unten nach oben	größer	selten
GLR	unten nach oben	alle	mehrdeutige Sprachen
PEG	oben nach unten	—	keine Mehrdeutigkeit

> *Fast alle großen Übersetzer benutzen heute einen handgeschriebenen Zerleger nach dem Verfahren des rekursiven Abstiegs, kombiniert mit Vorrangkletterei für Ausdrücke. Der Grund sind die Fehlermeldungen, nicht die Sprachklasse.*

B Landkarte des Bandes

Wo was steht

Kap.	Thema	Verweise
1	Aufbau, Selbstaufbau	Band 2 (Selbstbezug)
2	Lexikalische Analyse	Band 2 (reguläre Sprachen)
3	Syntaktische Analyse	Band 2 (kontextfreie Sprachen)
4	Namen und Bedeutung	Band 1 (Streutabellen)
5	Typsysteme	Band 2 (Entscheidbarkeit)
6	Speicher und Lebensdauer	Band 5 (Halde, Stapel)
7	Abstraktionsmittel	Band 5 (mittlere Datentechnik)
8	Zwischendarstellungen	Band 1 (Graphen)
9	Optimierung	Band 3 (Komplexität)
10	Codeerzeugung	Band 4 (Befehlssatz, Register)
11	Binden und Laden	Band 5 (Adressraum, Bibliotheken)
12	Interpreter	Band 4 (Sprungvorhersage)
13	Laufzeitübersetzung	Band 4 (Caches)
14	Praxis	Band 2 (Trusting Trust)
15	Stand 2026	Band 7 (Informationstheorie)

Sechs Verbindungen zur Mathematikreihe

Endliche Automaten	Algebra der Zustandsübergänge
Grammatiken	Formale Sprachen, Induktion über Ableitungen
Datenflussanalyse	Verbände, Fixpunkte, monotone Abbildungen
Registerzuteilung	Graphfärbung
Schleifenumformung	Ganzzahlige lineare Optimierung
Typinferenz	Unifikation, Substitutionen

Die dritte Zeile ist die überraschendste. Jede Datenflussanalyse ist die Berechnung eines kleinsten Fixpunkts auf einem Verband, und der Satz von Knaster und Tarski garantiert, dass er existiert und dass die übliche Iteration ihn findet.

C Glossar

Begriffe

Abschluss	Funktion samt der Umgebung, die sie eingefangen hat
Basisblock	Anweisungsfolge ohne Sprung hinein oder hinaus
Bytecode	Befehle für eine gedachte Maschine
Codeblock	Anonyme Funktion als Wert; in xBase seit Clipper 5
Einsetzen	Ein Funktionsaufruf wird durch den Rumpf ersetzt
Grammatik	Regelwerk, das die gültigen Programme beschreibt
Kellermaschine	Ausführungsmodell mit Stapel statt Registern
Lebendigkeit	Ob ein Wert später noch gebraucht wird
Phi-Funktion	Zusammenführung mehrerer Werte in der SSA-Form
Rekursiver Abstieg	Zerlegung mit einer Funktion je Grammatikregel
Sichtbarkeit	Bereich, in dem ein Name gilt
Sprachserver	Übersetzertechnik im Dienst des Editors
SSA-Form	Zwischenform, in der jede Variable einmal geschrieben wird
Stärkeminderung	Ein teurer Operator wird durch einen billigeren ersetzt
Symboltabelle	Zuordnung von Namen zu ihrer Bedeutung
Syntaxbaum	Struktur des Programms ohne überflüssige Zeichen
Typinferenz	Typen werden hergeleitet statt hingeschrieben
Undefiniertes Verhalten	Fälle, die der Standard offenlässt
Vorderteil	Der sprachabhängige Teil des Übersetzers
Zwischenform	Darstellung zwischen Sprache und Maschine

D Zeittafel

Von 1951 bis 2026

1951	Hopper: Programme, die Programme erzeugen
1957	**Backus u. a.: FORTRAN**
1958	McCarthy: LISP
1960	**ALGOL 60: BNF, Blöcke, Rekursion**
1964	BASIC
1965	Knuth: LR-Analyse
1968	Dijkstra: „Go To Statement Considered Harmful“
1970	Wirth: Pascal
1971	Wirth: schrittweise Verfeinerung
1972	**Ritchie: C**
1973	Hoare: Hinweise zum Sprachentwurf
1973	Unix-Kern in C neu geschrieben
1975	Steele, Sussman: Scheme
1978	Milner: Typinferenz
1979	`lex` und `yacc`
1980	dBASE II
1982	Chaitin: Registerzuteilung durch Färbung
1983	Stroustrup: C++
1984	**Thompson: „Reflections on Trusting Trust“**
1985	Nantucket Clipper: Übersetzer für die dBASE-Sprache
1987	Ferrante u. a.: Programmabhängigkeitsgraph
1990	Clipper 5.0 mit Codeblöcken

1991	**Cytron u. a.: SSA-Form**
1991	Python
1992	FlagShip: xBase nach C, für Unix
1992	Halstenbach: ROSI-SQL für Unix
1994	Hölzle, Ungar: Aufrufstellen zur Laufzeit optimieren
1995	Java: Bytecode und virtuelle Maschine
1995	FlagShip auch für Linux
1999	HotSpot: Laufzeitübersetzung im Massengebrauch
2003	**Lattner: LLVM**
2005	Wheeler: Diverse Double-Compiling
2008	V8: JavaScript wird schnell
2009	CompCert: bewiesener C-Übersetzer
2011	Yang u. a.: Csmith findet Übersetzerfehler
2012	Go 1.0
2015	Rust 1.0
2016	Sprachserver-Protokoll
2017	WebAssembly
2020	Module in C++
2022	Rust im Linux-Kern
2026	Übersetzer als wichtigste Prüfinstanz

E Kommentierte Quellen

[1] Aho, A., Lam, M., Sethi, R., Ullman, J.: *Compilers: Principles, Techniques, and Tools*. 2. Aufl., Pearson 2006.

Das Drachenbuch, seit vierzig Jahren die Standardreferenz. Sehr gründlich bei Analyse und Datenfluss, etwas altmodisch bei der Codeerzeugung. Zum Nachschlagen unentbehrlich, zum Durchlesen schwer.

[2] Appel, A.: *Modern Compiler Implementation in ML* (auch in Java und C). Cambridge University Press.

Baut einen vollständigen Übersetzer über das ganze Buch hinweg auf. Besonders klar bei Aktivierungsblöcken, Abschlüssen und der Registerzuteilung. Die ML-Fassung ist die beste.

[3] Aycock, J.: *A Brief History of Just-In-Time*. ACM Computing Surveys 35(2), 2003.

Zwanzig Seiten Überblick über eine Technik, die viel älter ist, als die meisten glauben. Reicht bis in die 1960er Jahre zurück und ordnet die Verfahren sauber ein.

[4] Backus, J.: *The History of FORTRAN I, II, and III*. ACM SIGPLAN Notices 13(8), 1978.

Der Erbauer des ersten ernsthaften Übersetzers erzählt, wie es zuging. Erfrischend offen über Fehlschläge und über den Widerstand, gegen den das Projekt anzukommen hatte. Kurz und sehr lesenswert.

[5] Chaitin, G.: *Register Allocation and Spilling via Graph Coloring*. SIGPLAN 1982.

Die Arbeit, die ein praktisches Problem auf ein bekanntes graphentheoretisches zurückführte. Seither ist die Registerzuteilung ein Färbungsproblem, und alles Weitere sind Verfeinerungen.

[6] Cooper, K., Torczon, L.: *Engineering a Compiler*. 3. Aufl., Morgan Kaufmann 2022.

Die beste Wahl für den Einstieg unter den Lehrbüchern. *Moderner als das Drachenbuch, mit mehr Gewicht auf Zwischenformen und Optimierung, und durchweg besser erklärt.*

[7] Cytron, R., u. a.: *Efficiently Computing Static Single Assignment Form and the Control Dependence Graph*. ACM TOPLAS 13(4), 1991.

Die Arbeit, mit der die SSA-Form praktisch wurde. Ohne sie sähe jeder heutige Optimierer anders aus. Nicht leicht zu lesen, aber jede Seite lohnt sich.

[8] Dijkstra, E.: *Go To Statement Considered Harmful*. CACM 11(3), 1968.

Eineinhalb Seiten, die eine jahrzehntelange Debatte auslösten. Die Überschrift stammt vom Herausgeber. Dijkstras eigentliches Argument ist feiner, als die Rezeption vermuten lässt: Es geht darum, den Programmtext auf den Programmablauf abbilden zu können.

[9] Ferrante, J., Ottenstein, K., Warren, J.: *The Program Dependence Graph and Its Use in Optimization*. ACM TOPLAS 9(3), 1987.

Eine Darstellung, die Steuerfluss und Datenfluss in einer Struktur vereint. Grundlage für Umformungen, die sich in keiner der beiden Sichten allein ausdrücken lassen.

[10] Grune, D., van Reeuwijk, K., Bal, H., Jacobs, C., Langendoen, K.: *Modern Compiler Design*. 2. Aufl., Springer 2012.

Deckt die Zerlegungsverfahren vollständiger ab als jedes andere Lehrbuch und behandelt auch funktionale und logische Sprachen. Gut geschrieben und weniger bekannt, als es verdient.

[11] Hoare, C.A.R.: *Hints on Programming Language Design*. 1973.

Zwanzig Seiten gesammelte Urteilskraft zum Sprachentwurf. Der Kerngedanke, dass eine Sprache den Programmierer nicht überraschen soll, ist seither hundertfach zitiert und ebenso oft verletzt worden.

[12] Hölzle, U., Ungar, D.: *Optimizing Dynamically-Dispatched Calls with Run-Time Type Feedback*. PLDI 1994.

Die Technik, die dynamische Sprachen schnell gemacht hat. Alles, was heutige JavaScript- und Java-Umgebungen an Aufrufstellen treiben, geht auf diese Arbeit und ihre Vorläufer aus dem Self-Projekt zurück.

[13] Jones, R., Hosking, A., Moss, E.: *The Garbage Collection Handbook*. 2. Aufl., CRC Press 2023.

Die maßgebliche Darstellung der Speicherbereinigung, vollständig und gut geordnet. Wer wissen will, warum ein Verfahren kurze Pausen hat und ein anderes hohen Durchsatz, findet die Antwort hier.

[14] Kernighan, B., Ritchie, D.: *The C Programming Language*. 2. Aufl., Prentice Hall 1988.

Zweihundert Seiten für eine ganze Sprache. Ein Vorbild an Knappheit, das seither niemand erreicht hat. Zeigt nebenbei das Modell der getrennten Übersetzung, das ganze Werkzeugketten geprägt hat.

[15] Knuth, D.: *On the Translation of Languages from Left to Right*. Information and Control 8(6), 1965.

Die Begründung der LR-Analyse. Knuth zeigte, welche Sprachen sich in einem Durchgang von links nach rechts ohne Rücksetzen zerlegen lassen, und wie. Formal und bis heute maßgeblich.

[16] Lattner, C., Adve, V.: *LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation*. CGO 2004.

Der Entwurf, der den Übersetzerbau organisatorisch verändert hat. Die Zwischenform als erstklassiges, dauerhaft gespeichertes Format ist der eigentliche Einfall.

[17] Levine, J.: *Linkers and Loaders*. Morgan Kaufmann 1999.

Das einzige gute Buch über einen Gegenstand, den alle benutzen und kaum jemand versteht. Erklärt Objektformate, Relokation, dynamische Bibliotheken und Symbolauflösung gründlich.

[18] McCarthy, J.: *Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I*. CACM 3(4), 1960.

Der Aufsatz, mit dem LISP entstand. Enthält nebenbei den ersten selbstinterpretierenden Interpreter, also ein Programm, das seine eigene Sprache auswertet. Hofstadter-Leser werden sich hier wohlfühlen.

[19] Milner, R.: *A Theory of Type Polymorphism in Programming*. Journal of Computer and System Sciences 17(3), 1978.

Der Algorithmus, der Typen herleitet, statt sie zu verlangen. Grundlage von ML, Haskell, OCaml und in abgeschwächter Form von `auto` in C++ und `var` in Java.

[20] Muchnick, S.: *Advanced Compiler Design and Implementation*. Morgan Kaufmann 1997.

Die gründlichste Darstellung der Optimierungsverfahren, die es gibt. Über sechshundert Seiten allein dazu. Kein Einstiegsbuch, aber das Nachschlagewerk, wenn man ein bestimmtes Verfahren genau wissen will.

[21] multisoft Datentechnik, Landshut: *FlagShip Reference Manual*. Laufend fortgeführt.

Die Dokumentation eines Systems, das xBase-Quelltext nach C übersetzt und dann den C-Übersetzer des Zielsystems bemüht. Als Lehrbuch nicht gedacht und trotzdem lehrreich, weil sich hier die Zwischensprache C an einem realen, kommerziell genutzten Beispiel beobachten lässt. Der Abschnitt über die Unterschiede zu Clipper zeigt, wie viel Sprachsemantik in solchen Details steckt.

[22] Naur, P. (Hrsg.): *Revised Report on the Algorithmic Language ALGOL 60*. CACM 6(1), 1963.

Der Bericht, der die formale Sprachbeschreibung begründete. Die Backus-Naur-Form steht hier zum ersten Mal in voller Anwendung. Ein Dokument, das mehr Nachwirkung hatte als die Sprache, die es beschreibt.

[23] Nystrom, R.: *Crafting Interpreters*. Genever Benning 2021.

*Frei im Netz verfügbar und ein Vergnügen zu lesen. Baut zwei vollständige Umsetzungen derselben Sprache auf: einen Baumdurchlauf in Java und eine Bytecode-Maschine in C. **Der beste Einstieg überhaupt**, wenn man selbst etwas bauen will.*

[24] Pierce, B.: *Types and Programming Languages*. MIT Press 2002.

Die maßgebliche Einführung in die Typtheorie. Streng, vollständig und mit sorgfältig aufgebauten Beweisen. Anspruchsvoll, und die einzige Quelle, die das Thema wirklich abdeckt.

[25] Scott, M.: *Programming Language Pragmatics*. 4. Aufl., Morgan Kaufmann 2015.

Vergleicht die Entwurfsentscheidungen vieler Sprachen nebeneinander: Namensregeln, Typsysteme, Speichermodelle, Abstraktionsmittel. Wer wissen will, warum eine Sprache etwas anders macht als eine andere, findet es hier.

[26] Thompson, K.: *Reflections on Trusting Trust*. CACM 27(8), 1984.

Vier Seiten, die man einmal im Leben gelesen haben sollte. Zeigt, wie sich eine Hintertür in einen Übersetzer einbauen lässt, die in keinem Quelltext auftaucht und sich bei jedem Selbstaufbau fortpflanzt. Der Schluss ist unbequem und bis heute nicht widerlegt.

[27] Wirth, N.: *Compiler Construction*. Addison-Wesley 1996.

Hundertfünfzig Seiten für einen vollständigen Übersetzer. Wirth war der Meinung, dass Übersetzerbau nicht kompliziert sein muss, und beweist es. Der Gegenentwurf zum Drachenbuch und in seiner Knappheit lehrreich.

[28] Yang, X., Chen, Y., Eide, E., Regehr, J.: *Finding and Understanding Bugs in C Compilers*. PLDI 2011.

Beschreibt Csmith und die über dreihundert gefundenen Fehler in gängigen C-Übersetzern. Ernüchternd und methodisch vorbildlich. Der Abschnitt über CompCert ist die beste vorhandene Begründung für formal verifizierte Übersetzer.

Wenn Sie nur drei Bücher kaufen

[23]	**Nystrom**	Zum Selbermachen. Frei verfügbar, unterhaltsam, und am Ende hat man zwei laufende Umsetzungen
[6]	**Cooper/Torczon**	Zum Lernen. Das beste Lehrbuch, besonders bei Zwischenformen und Optimierung
[20]	**Muchnick**	Zum Nachschlagen, sobald es um ein bestimmtes Optimierungsverfahren geht

Wer danach noch etwas möchte: [11] Hoare und [26] Thompson sind zusammen fünfundzwanzig Seiten und gehören zum Bildungskanon des Fachs. [27] Wirth zeigt, wie klein ein vollständiger Übersetzer sein kann, wenn man auf Vollständigkeit im Detail verzichtet.